

PROCESO DE DECODIFICACIÓN EN CÓDIGOS BINARIOS USANDO
LÍDERES DE CLASES LATERALES

LISBETH DANYELI DELGADO ORDOÑEZ

UNIVERSIDAD DEL CAUCA
FACULTAD DE CIENCIAS NATURALES, EXACTAS Y DE LA EDUCACIÓN
DEPARTAMENTO DE MATEMÁTICAS
PROGRAMA DE MATEMÁTICAS
POPAYÁN

2021

**PROCESO DE DECODIFICACIÓN EN CÓDIGOS BINARIOS USANDO
LÍDERES DE CLASES LATERALES**

LISBETH DANYELI DELGADO ORDOÑEZ

**TRABAJO DE INVESTIGACIÓN PRESENTADO COMO REQUISITO PARCIAL
PARA OPTAR AL TÍTULO DE MATEMÁTICA**

DIRIGIDO POR:

DR. JOHN HERMES CASTILLO GÓMEZ

CODIRIGIDO POR:

DR. JHON JAIRO BRAVO GRIJALBA

UNIVERSIDAD DEL CAUCA

FACULTAD DE CIENCIAS NATURALES, EXACTAS Y DE LA EDUCACIÓN

DEPARTAMENTO DE MATEMÁTICAS

PROGRAMA DE MATEMÁTICAS

POPAYÁN

2021

Nota de aceptación

Director _____

Dr. John Hermes Castillo Gómez

Codirector _____

Dr. Jhon Jairo Bravo Grijalba

Jurado _____

Paula Andrea Cadavid

Jurado _____

Diana Haidive Bueno

Fecha de sustentación: __ de 2021, Popayán.

Dedicado a Jacob, Luz y Emmy.

Agradecimientos

En este espacio quiero expresar mi agradecimiento a todas las personas que me han acompañado y apoyado en este proceso.

Al Dr. John Castillo, director de esta investigación, por todas las horas de dedicación, orientación, paciencia y supervisión en este tiempo. También, por todas sus palabras de aliento y disposición transmitiendo siempre su amor a la investigación. Además quiero expresar mi admiración, por todo el trabajo que realiza en su universidad y grupo de investigación.

Al Dr. Jhon Jairo Bravo, también director de esta investigación, por tener siempre la disposición de ayudarme a trabajar en lo que quería, por resolver todos los tramites y preguntas que tenía para él. Por los conocimientos transmitidos en las aulas de clase, por su buena disposición en todo momento. Por poner semillas de amor a la investigación en matemáticas.

A mi familia por su confianza y apoyo, especialmente a mi papá por darme la oportunidad de estudiar, a mi mamá por que nunca me dejo rendirme. A mis hermanos por ser mi inspiración en todo momento, a mi abuela por todo su amor. A mi abuelo que sembró las ganas de siempre seguir. A Eduardo por darme claridad. A mi hija Emmy por ser mi esperanza y luz.

A mis amigas y amigos, todos esos que encontré en el camino académico, porque aprendí de ellas y ellos, tanto en las aulas, como fuera de ellas, por todo lo que hemos vivido aprendiendo de nuestros profesores como de la vida. A mi dúo, porque entramos juntas y saldremos juntas, gracias por todos los sueños compartidos.

A Camila, Diana y Maria Paula, por tantos años y tantos aprendizajes juntas.

A mis profesores del Departamento de Matemáticas, todos son seres humanos maravillosos y profesionales que admiro y respeto. Infinitas gracias por estar siempre para sus estudiantes, no solo en las aulas.

A la Universidad del Cauca por la formación académica y personal infundida.

Resumen

En este trabajo se presentan algunos de los conceptos básicos de la teoría de códigos correctores de errores y se hace un estudio del cálculo de líderes en códigos lineales binarios, concepto fundamental para el proceso de decodificación por síndrome. A partir de los resultados expuestos en esta tesis se realizaron implementaciones en el software de álgebra computacional **SageMath**, algoritmos que se espera sean utilizados en futuras investigaciones. Estos algoritmos fueron contruidos para facilitar el entendimiento de la teoría estudiada a través de diversos ejemplos.

Los conceptos estudiados permiten definir una representación gráfica para cualquier código lineal binario. A partir de esta nueva definición se establecen algunas propiedades para este grafo relacionadas con las propiedades del código. Hasta donde se tiene conocimiento, esta es una nueva conexión entre la teoría de códigos y la teoría de grafos.

Índice general

Resumen	vi
Índice de figuras	ix
Introducción	1
1. Preliminares	3
1.1. Códigos	3
1.2. Códigos lineales binarios	9
1.3. Orden de términos	12
2. Líderes de clases de un código binario	16
2.1. Arreglo estándar	16
2.2. Diagrama de Hasse de un código	23
2.3. Cálculo de líderes	46
3. Decodificación por distancia mínima	53
3.1. Clases laterales y síndromes	53
3.2. Ejemplos de decodificación	57
4. Un grafo asociado a un código lineal binario	63
4.1. Teoría de grafos: definiciones y ejemplos	63
4.2. Algunas familias de grafos	65
4.3. Resultados	67

A. Elementos básicos de SageMath (Phyton)	74
B. Algoritmos y su implementación en SageMath	78
B.1. Algoritmo arreglo estándar	78
B.2. Algoritmo Diagrama de Hasse	80
B.2.1. Funciones auxiliares	83
B.3. Algoritmo cálculo de líderes	85
B.3.1. Funciones Auxiliares	87
B.4. Algoritmos auxiliares para contar	90
B.5. Algoritmos de decodificación	91
Referencias	94

Índice de figuras

2.1. Representación gráfica de la relaciones en el código del Ejemplo 2.9.	25
2.2. Diagrama de Hasse del código del Ejemplo 2.21.	33
2.3. Diagrama de Hasse del código del Ejemplo 2.23.	35
2.4. Diagrama de Hasse del código del Ejemplo 2.24.	36
2.5. Diagrama de Hasse del código del Ejemplo 2.25.	39
4.1. Ejemplo de grafo y un subgrafo.	64
4.2. Grafo conexo y grafo no conexo.	65
4.3. Grafo completo K_5	66
4.4. Ejemplo de grafo camino y grafo ciclo.	66
4.5. Ejemplos grafo bipartito y grafo estrella.	67
4.6. Grafo del código $\mathbb{F}_2^n$	68
4.7. Grafo $\Gamma(\mathcal{C}) \simeq P_2$	68
4.8. Grafo estrella.	72

Introducción

Claude Shannon fue un matemático, ingeniero electrónico y criptógrafo estadounidense, quien en el año 1948 publicó un artículo llamado *A mathematical theory of communication* [7], en el cual demuestra que los canales de información tienen una unidad de medida, es decir se puede determinar la capacidad del canal, de las bases para la corrección de errores, supresión de ruidos y redundancia, originando así una nueva rama en las matemáticas llamada Teoría de Códigos.

Richard Hamming trabajó al igual que Shannon en los Laboratorios Bell, interesándose también por la corrección de errores, por lo que ayudó a la construcción de esta nueva rama con grandes contribuciones y que en su honor algunos conceptos llevan su nombre: la distancia de Hamming, peso de Hamming, ventana de Hamming y código de Hamming, definiciones que siguen siendo fundamentales para el desarrollo de nuevos resultados en la disciplina.

Uno de los principales objetivos en la teoría de códigos es construir códigos capaces de transmitir un gran número de palabras y que tengan mayor corrección de errores a un bajo costo computacional. En la actualidad existen diversas ramas de las matemáticas auxiliares para ello, entre las cuales se destacan: álgebra, combinatoria y geometría.

Una estrategia útil para la construcción de algoritmos para la decodificación es el cálculo de los líderes de clases laterales. En este aspecto, en este trabajo de grado se hace una revisión del artículo *Computing coset leaders and leader codewords of binary codes* [1], en la cual se detallan algunas de sus demostraciones y a partir de estas ideas se realizaron implementaciones de parte del algoritmo para el cálculo de líderes que se presenta en el mencionado artículo. Para los algoritmos se usó el sistema de álgebra computacional **SageMath**, el cual fue creado por William Stein en 2005 y es un software libre basado en el lenguaje de programación **Python**.

Este trabajo se divide en 4 capítulos y un apéndice. En el primer capítulo se encuentran los conceptos básicos de teoría de códigos y teoría necesaria para abordar los temas presentados

en el desarrollo de los siguientes capítulos. En el segundo se desarrolla el cálculo de líderes de clases laterales, la teoría para la construcción de los algoritmos más adelante presentados y mediante ejemplos se muestran algunas implementaciones en **SageMath**. En el tercer capítulo, se desarrolla el proceso de decodificación con ayuda de los algoritmos mencionados en el Capítulo 2. En el capítulo 4 se muestra la forma de construir un grafo asociado a un código lineal binario. En este capítulo se demuestran algunos resultados acerca de las propiedades de este grafo. Hasta donde se tiene conocimiento, esta es una nueva conexión entre la teoría de códigos y la teoría de grafos. Los Teoremas [4.18](#), [4.21](#), [4.22](#), [4.27](#) y [4.28](#) y el Corolario [4.19](#) son resultados propios que se encontraron durante el desarrollo de este trabajo. Este trabajo no es autocontenido, sin embargo a lo largo del texto algunos resultados aparecen con la referencia del texto en dónde el lector puede encontrarlos junto con sus respectivas demostraciones. Con el animo de hacer la lectura más completa se presentan algunas de estas demostraciones. Para los resultados que no cuentan con una referencia exacta se presenta una demostración. Finalmente, en el apéndice se encuentran los algoritmos usados en el trabajo, su pseudocódigo y su implementación en **SageMath**.

Capítulo 1

Preliminares

En este capítulo se presentan algunas definiciones importantes en la teoría de códigos para el desarrollo y entendimiento del trabajo presentado a continuación, además de algunos resultados importantes que ayudarán a lo largo del desarrollo de este documento.

1.1. Códigos

En este trabajo se considera el espacio vectorial $\mathbb{F}_q^n$ de todas las n -uplas sobre el campo finito $\mathbb{F}_q$ con las operaciones usuales de suma y producto por un escalar:

- Suma: $(x_1, x_2, \dots, x_n) + (y_1, y_2, \dots, y_n) = (x_1 + y_1, x_2 + y_2, \dots, x_n + y_n)$.
- Producto: $\alpha(x_1, x_2, \dots, x_n) = (\alpha x_1, \alpha x_2, \dots, \alpha x_n)$,

para todo $(x_1, x_2, \dots, x_n), (y_1, y_2, \dots, y_n) \in \mathbb{F}_q^n$ y cualquier $\alpha \in \mathbb{F}_q$.

Además, $\mathbb{F}_q^n$ es un espacio vectorial con producto punto o producto interno dado por $(x_1, x_2, \dots, x_n) \cdot (y_1, y_2, \dots, y_n) = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$.

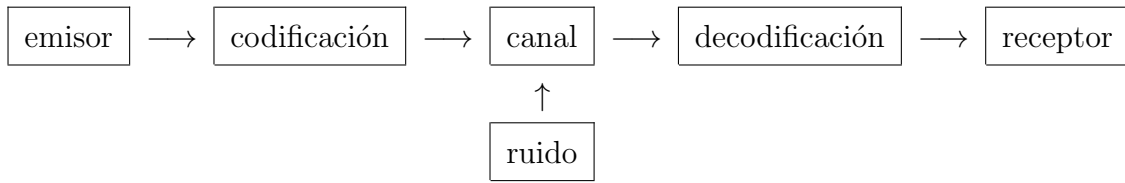
Un código es un sistema de señales o signos que se usan para la transmisión de mensajes. Es importante tener en cuenta que para ello se necesita de un canal de transmisión y tener el mismo lenguaje para que cuando el emisor envíe un mensaje, el receptor pueda entenderlo. Matemáticamente un código se puede describir de la siguiente manera.

Definición 1.1. Se dice que $\mathcal{C}$ es un código de bloque de longitud n o simplemente un código $\mathcal{C}$ de longitud n sobre $\mathbb{F}_q$ si es un subconjunto no vacío de $\mathbb{F}_q^n$; es decir si $\mathcal{C} \subseteq \mathbb{F}_q^n$. Los vectores

$(a_1, a_2, \dots, a_n)$ o $a_1 a_2 \dots a_n$ de $\mathcal{C}$, se llaman palabras código. El número de palabras del código se denota con $|\mathcal{C}|$. Si $|\mathcal{C}| = M$ se dice que el código tiene tamaño M , o es un código de longitud n y tamaño M , o que es un (n, M) -código.

Ejemplo 1.2. Sea $\mathcal{C} = \{01010, 01101, 11101\}$ sobre $\mathbb{F}_2$. Entonces $\mathcal{C}$ es un código de longitud 5 y tamaño 3, es decir es un $(5, 3)$ -código.

El objeto de estudio de la teoría de códigos es la transmisión de datos a través de canales ruidosos y la recuperación de mensajes corruptos. Estos últimos son mensajes en los cuales se han introducido errores. El siguiente esquema muestra cómo se transmite la información.



El mensaje que se desea enviar se transmite por un medio denominado canal, el cual depende del tipo de información a comunicar. El proceso se inicia con un emisor, quien en ese caso debe ingresar el mensaje teniendo en cuenta las capacidades y el tipo de canal; el segundo paso es la codificación, en el cual el mensaje transmitido se convierte a palabras del código. Luego el mensaje viaja por el canal hasta el receptor. En este proceso puede aparecer ruido y se espera que posteriormente pueda corregirse. En la decodificación se corrigen errores y se convierten las palabras del código a un lenguaje que el receptor pueda entender. Finalmente, el mensaje transmitido llega al receptor.

Para el anterior proceso Richard Hamming, uno de los trabajadores de los laboratorios Bell, estudió en particular la distancia de Hamming y otras definiciones iniciales de Teoría de Códigos. A continuación se da la definición de distancia de Hamming.

Definición 1.3. Sean $\mathbf{x}$ y $\mathbf{y}$ vectores de $\mathbb{F}_q^n$. La distancia de Hamming entre $\mathbf{x}$ y $\mathbf{y}$, denotada por $d(\mathbf{x}, \mathbf{y})$, se define como el número de posiciones en las cuales $\mathbf{x}$ y $\mathbf{y}$ difieren, es decir, si $\mathbf{x} = x_1 x_2 \dots x_n$ y $\mathbf{y} = y_1 y_2 \dots y_n$, entonces

$$d(\mathbf{x}, \mathbf{y}) = |\{i : 1 \leq i \leq n, x_i \neq y_i\}|.$$

Teorema 1.4. La distancia de Hamming es una métrica, es decir, para $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}_q^n$ se satisfacen las siguientes propiedades:

- (i) $d(\mathbf{x}, \mathbf{y}) \geq 0$ y $d(\mathbf{x}, \mathbf{y}) = 0$ si y solo si $\mathbf{x} = \mathbf{y}$;
- (ii) $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$;
- (iii) $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$.

Demostración. Los dos primeros items son una consecuencia directa de la definición de distancia de Hamming. Para el último, considere los siguientes conjuntos

$$A = \{i : x_i = z_i\} \text{ y } B = \{i : x_i = y_i \wedge y_i = z_i\}.$$

Es claro que $B \subseteq A$ y en consecuencia $A^C \subseteq B^C$. Observe que $d(\mathbf{x}, \mathbf{z}) = |A^C|$ y

$$B^C = \{i : x_i \neq y_i \vee y_i \neq z_i\} = \{i : x_i \neq y_i\} \cup \{i : y_i \neq z_i\} = C \cup D,$$

donde $C = \{i : x_i \neq y_i\}$ y $D = \{i : y_i \neq z_i\}$. De esta manera,

$$\begin{aligned} d(\mathbf{x}, \mathbf{z}) &= |A^C| \leq |B^C| = |C \cup D| = |C| + |D| - |C \cap D| \\ &\leq |C| + |D| = d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z}). \end{aligned}$$

□

A partir de la Definición 1.3 surgen los siguientes conceptos.

Definición 1.5. En un código $\mathcal{C}$ que contiene al menos dos palabras, la distancia mínima del código o simplemente la distancia del código es la menor distancia entre las palabras del código, es decir

$$d(\mathcal{C}) = \min\{d(\mathbf{x}, \mathbf{y}) : \mathbf{x}, \mathbf{y} \in \mathcal{C}, \mathbf{x} \neq \mathbf{y}\}.$$

Definición 1.6. Sea $\mathbf{x}$ una palabra del código $\mathcal{C}$. El peso de Hamming, denotado por $\text{wt}(\mathbf{x})$, se define como el número de coordenadas diferentes de 0 en $\mathbf{x}$, esto es $\text{wt}(\mathbf{x}) = d(\mathbf{x}, \mathbf{0})$.

Observación 1.7. El peso mínimo de un código $\mathcal{C}$ es el menor de los pesos de Hamming de las palabras código diferentes del vector $\mathbf{0}$ y se denota con $\text{wt}(\mathcal{C})$.

En el siguiente ejemplo se ilustran las Definiciones 1.3, 1.5 y 1.6.

Ejemplo 1.8. Considere el código $\mathcal{C} = \{\mathbf{x} = 01010, \mathbf{y} = 01101, \mathbf{z} = 11101\}$. A continuación se calcula la distancia de Hamming, la distancia de $\mathcal{C}$ y peso de Hamming sobre $\mathcal{C}$. Las distancias de Hamming entre las palabras del código son:

- $d(\mathbf{x}, \mathbf{y}) = 3,$
- $d(\mathbf{x}, \mathbf{z}) = 4,$
- $d(\mathbf{y}, \mathbf{z}) = 1.$

Así la distancia mínima de $\mathcal{C}$ es 1; es decir $d(\mathcal{C}) = 1$. Por otro lado, los pesos de Hamming de cada palabra del código es:

- $\text{wt}(\mathbf{x}) = d(\mathbf{x}, \mathbf{0}) = 2,$
- $\text{wt}(\mathbf{y}) = d(\mathbf{y}, \mathbf{0}) = 3,$
- $\text{wt}(\mathbf{z}) = d(\mathbf{z}, \mathbf{0}) = 4.$

Así $\text{wt}(\mathcal{C}) = 2$.

A continuación se presenta un resultado que será de ayuda en la construcción de la familia de códigos pares, ver Teorema 1.28.

Lema 1.9. (Lema 4.3.5, [5]) *Si $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$, entonces*

$$\text{wt}(\mathbf{x} + \mathbf{y}) = \text{wt}(\mathbf{x}) + \text{wt}(\mathbf{y}) - 2 \text{wt}(\mathbf{x} \star \mathbf{y}),$$

donde $\mathbf{x} \star \mathbf{y} = (x_1 y_1, x_2 y_2, \dots, x_n y_n)$.

Definición 1.10. Un código $\mathcal{C}$ se llama código par si todas las palabras tienen peso de Hamming par.

Por otro lado, en la teoría de códigos un concepto importante es el del soporte de un vector $\mathbf{x} = x_1 x_2 \dots x_n \in \mathbb{F}_2^n$, denotado por $\text{supp}(\mathbf{x})$ y definido como el conjunto de coordenadas del vector que son diferentes de cero, esto es $\text{supp}(\mathbf{x}) = \{i : x_i \neq 0\}$. Esto significa que $\text{wt}(\mathbf{x}) = |\text{supp}(\mathbf{x})|$.

Ejemplo 1.11. Sea $010100 \in \mathbb{F}_2^6$. Note que $\text{supp}(010100) = \{2, 4\}$. En este trabajo los índices del vector se empiezan a tener en cuenta a partir del 1, pero en la literatura se puede encontrar documentos en los que el conjunto de índices empieza en 0, y en este caso el soporte estaría dado por $\text{supp}(010100) = \{1, 3\}$.

Al transmitir un vector se puede recibir un vector diferente al enviado; esto es, pueden ocurrir cambios en las componentes del vector original. Cada uno de estos cambios se conoce como un error. Por ejemplo, al enviar $\mathbf{x} \in \mathbb{F}_q^n$ se puede recibir un vector $\mathbf{y} \in \mathbb{F}_q^n$ tal que $\mathbf{x} \neq \mathbf{y}$. De esta forma, $\mathbf{x} = \mathbf{y} + \mathbf{e}$, donde $\mathbf{e}$ se denomina vector de errores y $\text{wt}(\mathbf{e})$ es la cantidad de errores cometidos. La distancia de un código tiene una relación directa con la capacidad de detección y corrección de errores. A continuación se presentan las definiciones de detección y corrección de errores.

Definición 1.12. (Código detector de t -errores) Un código $\mathcal{C}$ detecta t -errores si cada vez que se envía una palabra, ocurren entre 1 y t errores durante la transmisión, el vector resultante no es una palabra código. Un código $\mathcal{C}$ detecta exactamente t -errores, si este detecta t -errores, pero no detecta $t + 1$ errores, es decir, existe al menos una palabra la cual cambiando $t + 1$ coordenadas origina una nueva palabra código.

Teorema 1.13. (Teorema 1.3, [6]) *Un código $\mathcal{C}$ detecta exactamente t -errores si y solo si $d(\mathcal{C}) = t + 1$.*

Definición 1.14. Dado un código $\mathcal{C}$, al recibir un vector $\mathbf{x}$, la decodificación por distancia mínima consiste en reemplazar $\mathbf{x}$ por la palabra código $\mathbf{c}$ más cercana a ella; es decir

$$d(\mathbf{x}, \mathcal{C}) = \min\{d(\mathbf{x}, \mathbf{y}) : \mathbf{y} \in \mathcal{C}\}. \quad (1.1)$$

Existen dos tipos de decodificación por distancia mínima: completa e incompleta. Si una palabra $\mathbf{x}$ es recibida y dos o más palabras código satisfacen la ecuación (1.1), entonces la decodificación por distancia mínima completa elige una arbitrariamente, mientras que la incompleta pide retransmisión.

Definición 1.15. (Código corrector de t -errores) Un código $\mathcal{C}$ corrige t -errores si la decodificación por distancia mínima corrige todos los errores de tamaño t o menos en cualquier palabra.

Un código $\mathcal{C}$ corrige exactamente t -errores, si este corrige t -errores, pero no corrige $(t + 1)$ -errores. Dicho de otra forma, todos los errores de tamaño t son corregidos, pero al menos un error de tamaño $t + 1$ es decodificado incorrectamente.

Teorema 1.16. (Teorema 1.4, [6]) *Un código $\mathcal{C}$ corrige exactamente t -errores si y solo si $d(\mathcal{C}) = 2t + 1$ o $d(\mathcal{C}) = 2t + 2$.*

Corolario 1.17. (Corolario 1.1, [6]) *Sea $\mathcal{C}$ un código. Entonces $d(\mathcal{C}) = d$ si y solo si $\mathcal{C}$ corrige exactamente $\lfloor (d - 1)/2 \rfloor$ -errores.*

Con los anteriores resultados se resalta la importancia de saber la distancia mínima de un código para el proceso de decodificación. A continuación se presentan algunas definiciones adicionales.

Definición 1.18. Sean $\mathbf{x} \in \mathbb{F}_q^n$ y r un número real no negativo. La esfera de radio r y centro en $\mathbf{x}$ es el conjunto

$$S(\mathbf{x}, r) = \{\mathbf{y} \in \mathbb{F}_q^n : d(\mathbf{x}, \mathbf{y}) \leq r\}.$$

Definición 1.19. Sea $\mathcal{C} \subseteq \mathbb{F}_q^n$ un código. El radio de empaquetamiento de $\mathcal{C}$ es el mayor entero r tal que todas las esferas $S(\mathbf{c}, r)$ para cada $\mathbf{c} \in \mathcal{C}$ son disjuntas. El radio de cubrimiento de $\mathcal{C}$ es el menor entero positivo s tal que las esferas $S(\mathbf{c}, s)$ con centro en las palabras de $\mathcal{C}$ cubren $\mathbb{F}_q^n$, esto es

$$\mathbb{F}_q^n = \bigcup_{\mathbf{c} \in \mathcal{C}} S(\mathbf{c}, s).$$

Se denota el radio de empaquetamiento de $\mathcal{C}$ por $\text{pr}(\mathcal{C})$ y el radio de cubrimiento $\rho(\mathcal{C})$.

Proposición 1.20. *Sean $\mathcal{C}$ un código y $d = d(\mathcal{C})$. Entonces las esferas con centro en palabras del código y radio $t = \lfloor (d - 1)/2 \rfloor$ son disjuntas dos a dos; es decir si $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$, entonces $S(\mathbf{c}_1, t) \cap S(\mathbf{c}_2, t) = \emptyset$.*

Demostración. De la definición de esfera se sigue que si $\mathbf{x} \in S(\mathbf{c}_1, t) \cap S(\mathbf{c}_2, t)$, entonces $d(\mathbf{x}, \mathbf{c}_1) \leq t$ y $d(\mathbf{x}, \mathbf{c}_2) \leq t$. De la desigualdad triangular se tiene que

$$\begin{aligned} d(\mathbf{c}_1, \mathbf{c}_2) &\leq d(\mathbf{c}_1, \mathbf{x}) + d(\mathbf{x}, \mathbf{c}_2) \\ &\leq 2t = 2 \left\lfloor \frac{d-1}{2} \right\rfloor \leq 2 \left(\frac{d-1}{2} \right) = d-1, \end{aligned}$$

lo cual es una contradicción. □

Corolario 1.21. Sean $\mathcal{C}$ un código y $d = d(\mathcal{C})$. Entonces

$$\lfloor (d-1)/2 \rfloor \leq \text{pr}(\mathcal{C}).$$

Definición 1.22. Un código $\mathcal{C}$ se denomina perfecto si $\text{pr}(\mathcal{C}) = \rho(\mathcal{C})$. En otras palabras, un código $\mathcal{C} \subseteq \mathbb{F}_q^n$ es perfecto si existe un número r tal que las esferas $S(\mathbf{c}, r)$ con centro en las palabras código son disjuntas y cubren $\mathbb{F}_q^n$.

Ejemplo 1.23. Considere el código de Hamming $\mathcal{H}_2(3)$, que es un $[7, 4, 3]$ -código lineal (ver Subsección 1.2), como $d = 3 = 2 * 1 + 1$, entonces las esferas de radios $r = 1$ centradas en las palabras del código son disjuntas. Además, $|\mathcal{H}_2(3)| = 16$, $|S_2(\mathbf{c}, 1)| = 8$ y $|\mathbb{F}_2^7| = 2^7 = 128$. Como $128 = 16 \times 8$, entonces las esferas $S_2(\mathbf{c}, 1)$ cubren a $\mathbb{F}_2^7$. Así, $\mathcal{H}_2(3)$ es perfecto.

1.2. Códigos lineales binarios

En la teoría de códigos existen diversas familias de códigos. En el presente trabajo se estudian resultados particulares de los códigos lineales binarios. En general, si $\mathcal{C}$ es un subespacio vectorial de $\mathbb{F}_q^n$ se le llama código lineal de longitud n . Además, si $q = 2$, se dice que $\mathcal{C}$ es un código lineal binario y se denomina un $[n, k]$ -código lineal binario, donde n es la longitud del código y k es su dimensión como espacio vectorial. Si además d es la distancia mínima de $\mathcal{C}$, se dice que $\mathcal{C}$ es un $[n, k, d]$ -código lineal binario.

Algunas definiciones y resultados enunciados a partir de ahora también son válidos para códigos en espacios vectoriales sobre $\mathbb{F}_q$, aunque en este trabajo solo se considerará el caso de códigos lineales binarios.

Para cada $[n, k]$ código lineal binario $\mathcal{C}$ existe un conjunto finito que permite representar de forma única sus elementos. Este conjunto se conoce como una base de $\mathcal{C}$. De manera más formal, $B = \{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ es una base de $\mathcal{C}$ si:

1. Para todo $\mathbf{v} \in \mathcal{C}$, existen escalares $a_1, \dots, a_k \in \mathbb{F}_2$ tales que $\mathbf{v} = a_1\mathbf{v}_1 + \dots + a_k\mathbf{v}_k$.
2. Si $b_1\mathbf{v}_1 + \dots + b_k\mathbf{v}_k = \mathbf{0}$, entonces $b_1 = \dots = b_k = 0$;

es decir B es un conjunto que genera al espacio vectorial $\mathcal{C}$ y es linealmente independiente.

A partir de lo anterior, una matriz generadora para el código lineal binario $\mathcal{C}$ es una matriz G cuyas filas forman una base para $\mathcal{C}$. Además, una matriz de control de paridad del código $\mathcal{C}$ es una matriz generadora para el código dual $\mathcal{C}^\perp$, el cual se define de la siguiente manera.

Definición 1.24. El código dual de un código lineal binario $\mathcal{C}$, denotado por $\mathcal{C}^\perp$, es el espacio ortogonal con respecto al producto punto $\cdot$ en $\mathbb{F}_2^n$, esto es

$$\mathcal{C}^\perp = \{\mathbf{y} \in \mathbb{F}_2^n : \mathbf{y} \cdot \mathbf{x} = 0, \text{ para todo } \mathbf{x} \in \mathcal{C}\}.$$

Ejemplo 1.25. Para el código $\mathcal{C} = \{11101, 10110, 01011, 11010, 00111, 01100, 10001, 00000\}$ la matriz generadora y la matriz de control de paridad son respectivamente

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} \quad \text{y} \quad H = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix}.$$

De la matriz de control de paridad puede hallarse el síndrome de un vector $\mathbf{x}$.

Definición 1.26. Sea $\mathcal{C}$ un $[n, k, d]$ -código lineal binario con matriz de control de paridad H . El síndrome del vector $\mathbf{x} \in \mathbb{F}_2^n$ se define como $S(\mathbf{x}) = \mathbf{x}H^T \in \mathbb{F}_2^{n-k}$.

Proposición 1.27. (Theorem 4.8.11, [5]) Sean $\mathcal{C}$ un código lineal binario y $\mathbf{x} \in \mathbb{F}_2^n$. Entonces $S(\mathbf{x}) = \mathbf{0}$ si y solo si $\mathbf{x} \in \mathcal{C}$.

La matriz generadora de un código puede dar ciertas propiedades a los códigos, como se puede ver en el siguiente resultado:

Teorema 1.28. Si $\mathcal{C}$ es un código lineal binario con una matriz generadora cuyas filas tienen peso de Hamming par, entonces cada palabra del código tiene peso de Hamming par.

Demostración. Por hipótesis el peso de las filas es par, además, por la definición de matriz generadora y el Lema 1.9 la suma de las las filas tienen peso de Hamming par, es decir las palabras del código tienen peso de Hamming par. $\square$

Ejemplo 1.29. El código $\mathcal{C}$ con una matriz generadora

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

es el conjunto $\mathcal{C} = \{(0, 0, 0, 0, 0), (1, 0, 0, 0, 1), (0, 1, 0, 0, 1), (1, 1, 0, 0, 0), (0, 0, 1, 1, 0), (1, 0, 1, 1, 1), (0, 1, 1, 1, 1), (1, 1, 1, 1, 0)\}$, donde todas las palabras tienen peso de Hamming par, es decir $\mathcal{C}$ es un código par.

Con el anterior teorema se pueden construir de una forma fácil códigos pares. Por otra parte, la siguiente definición es indispensable en este trabajo, y luego de enunciarla se dará un teorema que la relaciona con el síndrome, dando uno de los resultados más importantes para los procesos de decodificación, como se ejemplificará más adelante.

Definición 1.30. Sean $\mathcal{C}$ un código lineal de longitud n sobre $\mathbb{F}_2$ y $\mathbf{x}$ un vector de longitud n . Se define la clase lateral de $\mathbf{x}$ sobre $\mathcal{C}$ como el conjunto

$$\mathbf{x} + \mathcal{C} = \{\mathbf{x} + \mathbf{y} : \mathbf{y} \in \mathcal{C}\}.$$

Además, una palabra con peso de Hamming mínimo de una clase lateral se denomina líder de clase lateral o simplemente líder. El peso de Hamming de un líder, será el peso de la clase lateral, esto es, si $\mathbf{x}$ es el líder de una clase lateral $\mathcal{L}$ de un código $\mathcal{C}$, entonces $\text{wt}(\mathcal{L}) = \text{wt}(\mathbf{x})$.

Teorema 1.31. Sean $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$. Las siguientes afirmaciones son equivalentes:

- (i) $\mathbf{x}$ y $\mathbf{y}$ tienen el mismo síndrome.
- (ii) $\mathbf{y} - \mathbf{x} \in \mathcal{C}$.
- (iii) $\mathbf{x}$ y $\mathbf{y}$ están en la misma clase lateral.

Demostración.

(i) $\Rightarrow$ (ii) Si $S(\mathbf{x}) = S(\mathbf{y})$ entonces $S(\mathbf{x} - \mathbf{y}) = (\mathbf{x} - \mathbf{y})H^T = \mathbf{x}H^T - \mathbf{y}H^T = S(\mathbf{x}) - S(\mathbf{y}) = 0$, lo que implica que $\mathbf{x} - \mathbf{y} \in \mathcal{C}$.

(ii) $\Rightarrow$ (iii) Suponga que $\mathbf{y} - \mathbf{x} \in \mathcal{C}$, entonces $\mathbf{y} - \mathbf{x} = \mathbf{c}$ para algún $\mathbf{c} \in \mathcal{C}$. Así

$$\mathbf{y} + \mathcal{C} = (\mathbf{x} + \mathbf{c}) + \mathcal{C} = \mathbf{x} + \mathcal{C},$$

porque $\mathcal{C}$ es un espacio vectorial. Por lo tanto, $\mathbf{x}$ y $\mathbf{y}$ están en la misma clase lateral.

(iii) $\Rightarrow$ (i) Si $\mathbf{x}$ y $\mathbf{y}$ están en la misma clase lateral, entonces $\mathbf{y} + \mathcal{C} = \mathbf{x} + \mathcal{C}$ y así $\mathbf{y} = \mathbf{x} + \mathbf{c}'$, con $\mathbf{c}' \in \mathcal{C}$. Luego

$$\begin{aligned} S(\mathbf{y}) &= (\mathbf{x} + \mathbf{c}')H^T \\ &= \mathbf{x}H^T + \mathbf{c}'H^T \\ &= \mathbf{x}H^T \\ &= S(\mathbf{x}). \end{aligned}$$

□

1.3. Orden de términos

Para lo que sigue es importante tener en cuenta el orden del producto. En el caso de una variable el orden lo define el grado más alto de la variable. En caso de tener varias variables se necesita un orden análogo al del caso de una variable.

Primero se define un conjunto de productos de potencias denotado por

$$\mathbb{T}^n = \left\{ x_1^{\beta_1} \cdots x_n^{\beta_n} : \beta_i \in \mathbb{N}, i = 1, \dots, n \right\}.$$

Tener en cuenta que el producto de potencias hace referencia al producto de los x_i y término se refiere al coeficiente del producto de potencias. Además $x_1^{\beta_1} \cdots x_n^{\beta_n}$ también se denotará como X^β .

Existen muchas formas de ordenar $\mathbb{T}^n$, sin embargo, se espera que el conjunto cumpla con las propiedades de un orden deseado. Como para el caso lineal o el caso de una variable, con una extensión de la relación de divisibilidad se definirá el orden. Los términos de los polinomios se organizan en orden creciente o decreciente, por lo tanto debe ser capaz de comparar dos productos de potencias. Para ello el orden debe ser un orden total, es decir para $X^\alpha, X^\beta \in \mathbb{T}^n$ se cumple exactamente una de las siguientes relaciones:

$$X^\alpha \prec X^\beta, X^\alpha = X^\beta, X^\alpha \succ X^\beta.$$

Con la siguiente definición se describen las condiciones que debe cumplir un orden para ser un orden de términos.

Definición 1.32. Un orden de términos de $\mathbb{T}^n$ es un orden total $\prec$ que satisface las siguientes condiciones:

- $1 \prec X^\beta$ para todo $X^\beta \in \mathbb{T}^n, X^\beta \neq 1$;
- Si $X^\alpha \prec X^\beta$, entonces $X^\alpha X^\gamma \prec X^\beta X^\gamma$, para todo $X^\gamma \in \mathbb{T}^n$.

Los siguientes enunciados son ejemplos de ordenes de términos.

Definición 1.33. El orden lexicográfico en $\mathbb{T}^n$ con $x_1 > x_2 > \cdots > x_n$ se define así:

para $\alpha = (\alpha_1, \dots, \alpha_n), \beta = (\beta_1, \beta_2, \dots, \beta_n) \in \mathbb{N}^n$ se define $X^\alpha \prec X^\beta$ si la primera coordenada α_i y β_i en α y β al leer de izquierda a derecha que son diferentes satisfacen que $\alpha_i < \beta_i$.

Definición 1.34. El orden lexicográfico inverso en $\mathbb{T}^n$ con $x_1 > x_2 > \cdots > x_n$ para $\alpha = (\alpha_1, \dots, \alpha_n), \beta = (\beta_1, \beta_2, \dots, \beta_n) \in \mathbb{N}^n$ se define $X^\alpha \prec X^\beta$ como la primera coordenada α_i y β_i en α y β desde la derecha, las cuales son diferentes y que satisfacen $\alpha_i < \beta_i$.

Definición 1.35. El orden lexicográfico gradual en $\mathbb{T}^n$ con $x_1 > x_2 > \cdots > x_n$ se define como sigue: para $\alpha = (\alpha_1, \dots, \alpha_n), \beta = (\beta_1, \dots, \beta_n) \in \mathbb{T}^n$ se tiene que: $X^\alpha \prec X^\beta$, si $\sum_{i=1}^n \alpha_i < \sum_{i=1}^n \beta_i$ o $\sum_{i=1}^n \alpha_i = \sum_{i=1}^n \beta_i$ y $X^\alpha \prec X^\beta$ con respecto al lexicográfico con $x_1 > x_2 > \cdots > x_n$.

Definición 1.36. El orden lexicográfico inverso gradual en $\mathbb{T}^n$ con $x_1 > x_2 > \cdots > x_n$ se define de la siguiente manera: para $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n), \beta = (\beta_1, \beta_2, \dots, \beta_n) \in \mathbb{N}^n$ se tiene que: $X^\alpha \prec X^\beta$, si $\sum_{i=1}^n \alpha_i < \sum_{i=1}^n \beta_i$ o $\sum_{i=1}^n \alpha_i = \sum_{i=1}^n \beta_i$ y la primera coordenada α_i y β_i en α y β desde la derecha, en las cuales son diferentes, satisface $\alpha_i > \beta_i$.

Ejemplo 1.37. Sean $001101, 011000 \in \mathbb{F}_2^6$.

- Respecto al orden lexicográfico $001101 \prec 011000$ porque al leer los términos desde la izquierda en la primera coordenada son iguales, pero en la segunda coordenada $0 < 1$.
- En el orden lexicográfico inverso $011000 \prec 001101$ ya que los términos desde la derecha en la primera coordenada $1 > 0$.
- En el orden lexicográfico gradual y lexicográfico inverso gradual, $011000 \prec 001101$, porque para 011000 la suma da grado 3 y para 001101 la suma da grado 2.

Ejemplo 1.38. Considere los vectores $1101, 0111 \in \mathbb{F}_2^4$.

- Con el orden lexicográfico $0111 \prec 1101$ porque los términos desde la izquierda en la primera coordenada $1 > 0$.
- En el orden lexicográfico inverso $1101 \prec 0111$ ya que los términos desde la derecha en la primera coordenada son iguales, pero en la segunda coordenada $0 < 1$.
- Para el orden lexicográfico gradual $0111 \prec 1101$, ya que el grado de los dos vectores es 3 y los términos desde la izquierda en la primera coordenada $1 > 0$.
- Para el orden lexicográfico inverso gradual $1101 \prec 0111$, ya que el grado de los dos vectores es 3 los términos desde la derecha en la primera coordenada son iguales, pero en la segunda coordenada $0 < 1$.

El algoritmo `ordering` (ver Algoritmo 9) se construyó como una función auxiliar del algoritmo `Comp_CL` (ver Algoritmo 6). Esta es una función que recibe dos vectores y orden de términos; y retorna una lista con los dos vectores ordenados teniendo en cuenta el orden de términos dado, donde el primer vector es el menor. El orden que se define con esta función es una función compatible con el peso de Hamming (ver Definición 2.41).

Ejemplo 1.39. Haciendo los cálculos para el anterior ejemplo con el orden lexicográfico, lexicográfico inverso y lexicográfico de grado en SageMath se obtiene:

```
ordering(vector(GF(2),(0,1,1,1)),vector(GF(2),(1,1,1,0)),'lex')
[(0, 1, 1, 1), (1, 1, 1, 0)]
ordering(vector(GF(2),(0,1,1,1)),vector(GF(2),(1,1,1,0)),'invlex')
[(1, 1, 1, 0), (0, 1, 1, 1)]
ordering(vector(GF(2),(0,1,1,1)),vector(GF(2),(1,1,1,0)),'deglex')
[(0, 1, 1, 1), (1, 1, 1, 0)]
```

Ejemplo 1.40. A continuación se calcula el orden de los vectores del Ejemplo 1.37 con `ordering`.

```
ordering(vector(GF(2),(0,1,1,0,0,0)),vector(GF(2),(0,0,1,1,0,1)),
'lex')
[(0, 1, 1, 0, 0, 0), (0, 0, 1, 1, 0, 1)]
ordering(vector(GF(2),(0,1,1,0,0,0)),vector(GF(2),(0,0,1,1,0,1)),
'invlex')
[(0, 1, 1, 0, 0, 0), (0, 0, 1, 1, 0, 1)]
```

Observación 1.41. La función auxiliar `ordering` (ver Algoritmo 9) fue construida con la teoría de orden de términos, pero es un orden de grado, donde el grado es el peso de Hamming de un vector. Por eso en el anterior ejemplo como los vectores tienen el mismo peso de Hamming el orden lexicográfico y lexicográfico inverso son iguales.

Capítulo 2

Líderes de clases de un código binario

En este capítulo se presenta el concepto de arreglo estándar para un código lineal binario. Además, se muestra cómo calcular los líderes de las clases laterales para este tipo de códigos. En particular, en la Sección 2.2 se define un orden parcial en el conjunto de las clases laterales y a partir de este orden se establece su diagrama de Hasse asociado. Esta construcción se utilizará en el desarrollo del Capítulo 4. En la Sección 2.3 se expone otra forma, basada en el artículo [1], para calcular estos líderes.

2.1. Arreglo estándar

Teniendo en cuenta las definiciones y resultados del Capítulo 1, a continuación se describe el arreglo estándar, que se usa en el proceso de decodificación.

El arreglo Slepiano ¹ o arreglo estándar para un $[n, k, d]$ -código lineal binario $\mathcal{C}$ es una tabla de 2^{n-k} filas, que contiene todos los vectores del espacio $\mathbb{F}_2^n$. Este arreglo se realiza de la siguiente manera: la primera fila del arreglo consta de las palabras del código, la segunda fila consta de la suma de cada palabra de la primera fila con un vector $\mathbf{x}_1 \in \mathbb{F}_2^n$ de menor peso que no esté en $\mathcal{C}$, es decir la clase lateral $\mathbf{x}_1 + \mathcal{C}$. De manera general, la i -ésima fila del arreglo se forma por la suma de cada palabra de la primera fila con un vector $\mathbf{x}_i$ de menor peso que no está aún en el arreglo, es decir la clase lateral $\mathbf{x}_i + \mathcal{C}$. Este proceso continúa hasta que el arreglo contenga todas las palabras de $\mathbb{F}_2^n$, es decir, cuando se tengan 2^{n-k} filas. Observe que por la

¹En honor a David D. Slepian (1923-2007), matemático americano.

forma de construir el arreglo estándar los elementos de la primera columna son líderes de las clases laterales del código $\mathcal{C}$.

Ejemplo 2.1. Consideremos el $[5, 2, 3]$ -código lineal binario $\mathcal{C} = \{00000, 10110, 01011, 11101\}$.

Las clases laterales de este código son:

$$\begin{aligned}
 00000 + \mathcal{C} &= \{00000, 10110, 01011, 11101\}, \\
 00001 + \mathcal{C} &= \{00001, 10111, 01010, 11100\}, \\
 00010 + \mathcal{C} &= \{00010, 10100, 01001, 11111\}, \\
 00100 + \mathcal{C} &= \{00100, 10010, 01111, 11001\}, \\
 01000 + \mathcal{C} &= \{01000, 11110, 00011, 10101\}, \\
 10000 + \mathcal{C} &= \{10000, 00110, 11011, 01101\}, \\
 00101 + \mathcal{C} &= \{00101, 10011, 01110, 11000\}, \\
 01100 + \mathcal{C} &= \{01100, 11010, 00111, 10001\}.
 \end{aligned}$$

De esta forma un arreglo estándar para el código $\mathcal{C}$ es:

$$\begin{array}{cccc}
 00000 & 10110 & 01011 & 11101 \\
 00001 & 10111 & 01010 & 11100 \\
 00010 & 10100 & 01001 & 11111 \\
 00100 & 10010 & 01111 & 11001 \\
 01000 & 11110 & 00011 & 10101 \\
 10000 & 00110 & 11011 & 01101 \\
 00101 & 10011 & 01110 & 11000 \\
 01100 & 11010 & 00111 & 10001
 \end{array}$$

Para el mismo ejemplo se utilizó el algoritmo `arreglo_estandar` implementado en `SageMath` (ver Algoritmo 1). Además del arreglo estándar el algoritmo, al final, muestra una lista con los líderes del código.

```

M1=matrix(GF(2),[[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
arreglo_estandar(C1)
[[ (0, 0, 0, 0, 0), (1, 0, 1, 1, 0), (1, 1, 1, 0, 1),
  (0, 1, 0, 1, 1)],
[(1, 0, 0, 0, 0), (0, 0, 1, 1, 0), (0, 1, 1, 0, 1),
  (1, 1, 0, 1, 1)],
[(0, 0, 0, 0, 1), (1, 0, 1, 1, 1), (1, 1, 1, 0, 0),
  (0, 1, 0, 1, 0)],
[(0, 1, 0, 0, 0), (1, 1, 1, 1, 0), (1, 0, 1, 0, 1),
  (0, 0, 0, 1, 1)],
[(0, 0, 0, 1, 0), (1, 0, 1, 0, 0), (1, 1, 1, 1, 1),
  (0, 1, 0, 0, 1)],
[(0, 0, 1, 0, 0), (1, 0, 0, 1, 0), (1, 1, 0, 0, 1),
  (0, 1, 1, 1, 1)],
[(1, 1, 0, 0, 0), (0, 1, 1, 1, 0), (0, 0, 1, 0, 1),
  (1, 0, 0, 1, 1)],
[(0, 1, 1, 0, 0), (1, 1, 0, 1, 0), (1, 0, 0, 0, 1),
  (0, 0, 1, 1, 1)],
[(0, 0, 0, 0, 0), (1, 0, 0, 0, 0),
  (0, 0, 0, 0, 1), (0, 1, 0, 0, 0),
  (0, 0, 0, 1, 0), (0, 0, 1, 0, 0),
  (1, 1, 0, 0, 0), (0, 1, 1, 0, 0)]

```

En este ejemplo se puede observar que el líder de una clase lateral no necesariamente es único. Por ejemplo, la clase $00101 + \mathcal{C}$ tiene dos líderes: 00101 y 11000.

A continuación se presentan otros ejemplos implementados en `arreglo_estandar` (ver Algoritmo 1).

Ejemplo 2.2. Sea C el $[5, 2, 3]$ -código binario con una matriz generadora

$$M_2 = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Usando el algoritmo `arreglo_estandar` se obtuvo:

```
M2=matrix(GF(2),[[1,1,1,1,0],[0,0,1,1,1]])
C2=LinearCode(M2)
arreglo_estandar(C2)
[[ (0, 0, 0, 0, 0), (1, 1, 1, 1, 0), (0, 0, 1, 1, 1),
  (1, 1, 0, 0, 1)],
 [(1, 0, 0, 0, 0), (0, 1, 1, 1, 0), (1, 0, 1, 1, 1),
  (0, 1, 0, 0, 1)],
 [(0, 0, 0, 0, 1), (1, 1, 1, 1, 1), (0, 0, 1, 1, 0),
  (1, 1, 0, 0, 0)],
 [(0, 1, 0, 0, 0), (1, 0, 1, 1, 0), (0, 1, 1, 1, 1),
  (1, 0, 0, 0, 1)],
 [(0, 0, 0, 1, 0), (1, 1, 1, 0, 0), (0, 0, 1, 0, 1),
  (1, 1, 0, 1, 1)],
 [(0, 0, 1, 0, 0), (1, 1, 0, 1, 0), (0, 0, 0, 1, 1),
  (1, 1, 1, 0, 1)],
 [(1, 0, 1, 0, 0), (0, 1, 0, 1, 0), (1, 0, 0, 1, 1),
  (0, 1, 1, 0, 1)],
 [(0, 1, 1, 0, 0), (1, 0, 0, 1, 0), (0, 1, 0, 1, 1),
  (1, 0, 1, 0, 1)],
 [(0, 0, 0, 0, 0), (1, 0, 0, 0, 0),
  (0, 0, 0, 0, 1), (0, 1, 0, 0, 0),
  (0, 0, 0, 1, 0), (0, 0, 1, 0, 0),
  (1, 0, 1, 0, 0), (0, 1, 1, 0, 0)]
```

Entonces el arreglo estándar de este código tiene 8 filas y 4 columnas, que contienen todos los elementos de $\mathbb{F}_2^5$. Igual que en el Ejemplo 2.1 se puede evidenciar en las filas 7 y 8 que los líderes de estas clases laterales no son únicos.

Ejemplo 2.3. Sea $\mathcal{C}$ el $[6, 3, 2]$ -código binario con una matriz generadora dada por

$$M_3 = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

Para este código usando el algoritmo `arreglo_estandar` se obtuvo:

```
M3=matrix(GF(2),[[1,0,0,0,1,1],[0,1,0,1,0,1],[0,0,1,1,1,0]])
C3=LinearCode(M3)
arreglo_estandar(C3)
((0, 0, 0, 0, 0, 0), (1, 0, 0, 0, 1, 1),
 (0, 1, 0, 1, 0, 1), (1, 1, 0, 1, 1, 0),
 (0, 0, 1, 1, 1, 0), (1, 0, 1, 1, 0, 1),
 (0, 1, 1, 0, 1, 1), (1, 1, 1, 0, 0, 0)],
[(1, 0, 0, 0, 0, 0), (0, 0, 0, 0, 1, 1),
 (1, 1, 0, 1, 0, 1), (0, 1, 0, 1, 1, 0),
 (1, 0, 1, 1, 1, 0), (0, 0, 1, 1, 0, 1),
 (1, 1, 1, 0, 1, 1), (0, 1, 1, 0, 0, 0)],
[(0, 0, 1, 0, 0, 0), (1, 0, 1, 0, 1, 1),
 (0, 1, 1, 1, 0, 1), (1, 1, 1, 1, 1, 0),
 (0, 0, 0, 1, 1, 0), (1, 0, 0, 1, 0, 1),
 (0, 1, 0, 0, 1, 1), (1, 1, 0, 0, 0, 0)],
[(0, 0, 0, 0, 0, 1), (1, 0, 0, 0, 1, 0),
 (0, 1, 0, 1, 0, 0), (1, 1, 0, 1, 1, 1),
 (0, 0, 1, 1, 1, 1), (1, 0, 1, 1, 0, 0),
 (0, 1, 1, 0, 1, 0), (1, 1, 1, 0, 0, 1)],
[(0, 1, 0, 0, 0, 0), (1, 1, 0, 0, 1, 1),
 (0, 0, 0, 1, 0, 1), (1, 0, 0, 1, 1, 0),
 (0, 1, 1, 1, 1, 0), (1, 1, 1, 1, 0, 1),
 (0, 0, 1, 0, 1, 1), (1, 0, 1, 0, 0, 0)],
[(0, 0, 0, 1, 0, 0), (1, 0, 0, 1, 1, 1),
```

```

(0, 1, 0, 0, 0, 1), (1, 1, 0, 0, 1, 0),
(0, 0, 1, 0, 1, 0), (1, 0, 1, 0, 0, 1),
(0, 1, 1, 1, 1, 1), (1, 1, 1, 1, 0, 0)],
[(0, 0, 0, 0, 1, 0), (1, 0, 0, 0, 0, 1),
(0, 1, 0, 1, 1, 1), (1, 1, 0, 1, 0, 0),
(0, 0, 1, 1, 0, 0), (1, 0, 1, 1, 1, 1),
(0, 1, 1, 0, 0, 1), (1, 1, 1, 0, 1, 0)],
[(0, 1, 0, 0, 1, 0), (1, 1, 0, 0, 0, 1),
(0, 0, 0, 1, 1, 1), (1, 0, 0, 1, 0, 0),
(0, 1, 1, 1, 0, 0), (1, 1, 1, 1, 1, 1),
(0, 0, 1, 0, 0, 1), (1, 0, 1, 0, 1, 0)]]],
[(0, 0, 0, 0, 0, 0), (1, 0, 0, 0, 0, 0),
(0, 0, 1, 0, 0, 0), (0, 0, 0, 0, 0, 1),
(0, 1, 0, 0, 0, 0), (0, 0, 0, 1, 0, 0),
(0, 0, 0, 0, 1, 0), (0, 1, 0, 0, 1, 0)]]

```

Este arreglo estándar del código tiene 8 filas y 8 columnas, se puede observar que tiene 8 líderes, uno de peso 0, 6 de peso 1 y 1 de peso 2; el líder de peso 2 no es único, en esta clase lateral están los líderes 100100, 010010 y 001001.

Como se observa en los siguientes ejemplos, los datos que se calculan con el Algoritmo `arreglo_estandar` pueden ser muy grandes y esto hace que la información sea muy extensa para escribir. De esta forma para realizar análisis en códigos más grandes se utilizan las funciones `len` y `fun_cont_lid`, ver Apéndice A y Algoritmo B.4, respectivamente.

Ejemplo 2.4. Sea $\mathcal{C}$ el $[10, 4, 4]$ -código binario con una matriz de control de paridad

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

Los cálculos realizados en SageMath son los siguientes:

```
H1=matrix(GF(2),[[1,0,0,0,1,0,0,0,0,0],[1,0,1,1,0,1,0,0,0,0],
                 [1,1,0,1,0,0,1,0,0,0],[1,1,1,0,0,0,0,1,0,0],
                 [1,1,1,1,0,0,0,0,1,0],[1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
ae5=arreglo_estandar(C5)
len(ae5[0][1])
16
len(ae5[1])
64
fun_cont_lid(ae5[1])
Counter({2: 30, 3: 23, 1: 10, 0: 1})
```

En consecuencia, se obtuvo que el arreglo estándar para este código tiene 64 filas y 16 columnas. Además, con el algoritmo `fun_cont_lid`, se obtuvo el número de líderes para cada peso:

Peso	0	1	2	3
Número de líderes	1	10	30	23

Tabla 2.1: Número de líderes según su peso.

Los códigos de Reed-Muller forman una familia de códigos muy conocidos en la literatura y tienen diversas aplicaciones en la actualidad. Para cada entero positivo m y cada entero r tal que $0 \leq r \leq m$, existe un código de Reed-Muller de orden r , denotado por $\mathcal{R}(r, m)$; este es un código binario con parámetros $[2^m, \binom{m}{0} + \binom{m}{1} + \cdots + \binom{m}{r}, 2^{m-r}]$. Mayor información acerca de estos códigos se puede encontrar en [5, Sección 6.2].

Ejemplo 2.5. Sea $\mathcal{C} = \mathcal{R}(1, 4)$ un código de Reed-Muller. Se realizan los siguientes cálculos en SageMath.

```

C6=codes.BinaryReedMullerCode(1, 4);
ae6=arreglo_estandar(C6)
len(ae6[0][1])
    32
len(ae[1])
    2048
fun_cont_lid(ae6[1])
    Counter({4: 875, 3: 560, 5: 448, 2: 120, 6: 28, 1: 16, 0: 1})

```

Utilizando el algoritmo `arreglo_estandar` se obtuvo una matriz de 2048 filas y 32 columnas. Con el algoritmo `fun_cont_lid` se obtiene que los líderes tienen las siguientes características.

Peso	0	1	2	3	4	5	6
Número de líderes	1	16	120	560	875	448	28

Tabla 2.2: Número de líderes según su peso.

Observación 2.6. En los Ejemplos 2.27 y 2.28 en los que los códigos tienen mayor longitud se hace difícil a partir del arreglo estándar verificar si los líderes son únicos o no.

2.2. Diagrama de Hasse de un código

Las siguientes definiciones, teoremas y lemas se utilizarán para definir un orden parcial en el conjunto de las clases laterales de un código lineal binario dado. A partir de este concepto en el Capítulo 4 se definirá el grafo asociado a un código lineal binario. Con estas ideas se implementó el algoritmo `diagrama_Hasse_codigo` (ver Algoritmo 2), esto con el objetivo de usarlo para realizar ejemplos que ayuden a analizar características del grafo construido. Esta sección está basada en el Capítulo 11 de [4].

Definición 2.7. Para $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$, se define la relación $\preceq$ de la siguiente manera: $\mathbf{x} \preceq \mathbf{y}$ siempre que $\text{supp}(\mathbf{x}) \subseteq \text{supp}(\mathbf{y})$. Si $\mathbf{x} \preceq \mathbf{y}$ se dice que $\mathbf{y}$ cubre a $\mathbf{x}$ o que $\mathbf{x}$ está cubierto por $\mathbf{y}$. Cuando $\text{supp}(\mathbf{x}) \subset \text{supp}(\mathbf{y})$, se escribirá $\mathbf{x} \prec \mathbf{y}$.

Dado que la relación $\preceq$ se definió a partir de la inclusión de conjuntos, es fácil probar que esta relación es un orden parcial; es decir que para cualquier $\mathbf{x}$, $\mathbf{y}$ y $\mathbf{z}$ en $\mathbb{F}_2^n$ se tiene que:

1. $\mathbf{x} \preceq \mathbf{x}$. (Reflexiva)
2. Si $\mathbf{x} \preceq \mathbf{y}$ y $\mathbf{y} \preceq \mathbf{x}$, entonces $\mathbf{x} = \mathbf{y}$. (Antisimétrica)
3. Si $\mathbf{x} \preceq \mathbf{y}$ y $\mathbf{y} \preceq \mathbf{z}$, entonces $\mathbf{x} \preceq \mathbf{z}$. (Transitiva)

Este orden se extiende al conjunto de clases laterales de un código lineal, como se describe en la siguiente definición.

Definición 2.8. Si $\mathcal{C}_1$ y $\mathcal{C}_2$ son dos clases laterales de $\mathcal{C}$, entonces $\mathcal{C}_1 \preceq \mathcal{C}_2$ siempre que existan líderes $\mathbf{x}_1$ de $\mathcal{C}_1$ y $\mathbf{x}_2$ de $\mathcal{C}_2$, tales que $\mathbf{x}_1 \preceq \mathbf{x}_2$. Respectivamente, se escribirá $\mathcal{C}_1 \prec \mathcal{C}_2$ siempre que existan líderes de clase $\mathbf{x}_1$ de $\mathcal{C}_1$ y $\mathbf{x}_2$ de $\mathcal{C}_2$, tales que $\mathbf{x}_1 \prec \mathbf{x}_2$.

1. Si $\mathcal{C}_1 \prec \mathcal{C}_2$, se dice que $\mathcal{C}_1$ es una clase lateral descendiente de $\mathcal{C}_2$ o simplemente se dirá que $\mathcal{C}_1$ es un descendiente de $\mathcal{C}_2$ o equivalentemente que $\mathcal{C}_2$ es un ancestro de $\mathcal{C}_1$.
2. Si $\mathcal{C}_1 \prec \mathcal{C}_2$ y $\text{wt}(\mathcal{C}_1) = \text{wt}(\mathcal{C}_2) - 1$, se dice que $\mathcal{C}_1$ es una clase lateral hija de $\mathcal{C}_2$ o simplemente que $\mathcal{C}_1$ es hijo de $\mathcal{C}_2$, o equivalentemente que $\mathcal{C}_2$ es un padre de $\mathcal{C}_1$.
3. Si $\mathcal{C}_1$ no tiene padres, entonces se llamará clase lateral huérfana o se dirá que $\mathcal{C}_1$ es un huérfano.

Se puede probar que $\preceq$ es un orden parcial en el conjunto de las clases laterales de $\mathcal{C}$. Se debe tener en cuenta que para esto se debe demostrar que este orden es independiente de los líderes escogidos en cada clase; este hecho se demostrará en el Corolario 2.14. Primero se presentarán algunos ejemplos y resultados preliminares necesarios para demostrar este resultado.

Ejemplo 2.9. Sea $\mathcal{C}$ el $[5, 2, 3]$ -código binario con una matriz generadora dada por

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Entonces para este código sus clases laterales junto con sus líderes son:

$$\begin{aligned}
 \mathcal{C}_0 &= 00000 + \mathcal{C}, & \mathcal{C}_4 &= 00010 + \mathcal{C}, \\
 \mathcal{C}_1 &= 10000 + \mathcal{C}, & \mathcal{C}_5 &= 00001 + \mathcal{C}, \\
 \mathcal{C}_2 &= 01000 + \mathcal{C}, & \mathcal{C}_6 &= 10100 + \mathcal{C} = 01010 + \mathcal{C}, \\
 \mathcal{C}_3 &= 00100 + \mathcal{C}, & \mathcal{C}_7 &= 10010 + \mathcal{C} = 01100 + \mathcal{C}.
 \end{aligned}$$

La relación entre padres e hijos, dada en la Definición 2.8, puede representarse gráficamente de la siguiente manera:

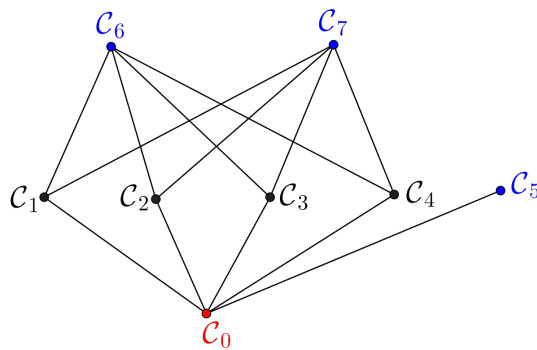


Figura 2.1: Representación gráfica de la relaciones en el código del Ejemplo 2.9.

Se puede observar que los vértices azules correspondientes a las clases laterales $\mathcal{C}_6$, $\mathcal{C}_7$ y $\mathcal{C}_5$ son huérfanos. Además, $\mathcal{C}_2$ es padre de $\mathcal{C}_0$, $\mathcal{C}_6$ es ancestro de $\mathcal{C}_0$, entre otras relaciones. En general, se denotará la clase lateral del vector $\mathbf{0}$ con $\mathcal{C}_0 = \mathbf{0} + \mathcal{C}$. Dado que $\text{supp}(\mathbf{0}) = \emptyset$, entonces $\mathcal{C}_0$ es descendiente de todas las otras clases laterales de $\mathcal{C}$. De esta forma, en las representaciones gráficas, como la de la Figura 2.1, el vértice rojo siempre corresponderá a la clase $\mathcal{C}_0$.

A partir de ahora, se usará la Definición 2.8 para representar las relaciones existentes entre los líderes de las clases laterales en un código lineal binario. La representación gráfica de estas relaciones se conoce como diagrama de Hasse. Mayor información sobre la construcción de los diagramas de Hasse puede encontrarse en la Sección 8.5 del libro [3].

Para probar los resultados enunciados a continuación, serán importantes las propiedades del peso de Hamming vistas en el Capítulo 1.

Teorema 2.10. (Theorem 11.1.6, [4]) *Sea $\mathcal{C}$ un $[n, k]$ -código lineal binario. Si existe una clase lateral de peso s , entonces existe una clase lateral con peso t , para cualquier $0 < t < s$.*

Demostración. Sea $\mathcal{C}'$ una clase lateral de peso s y $\mathbf{x} = x_1x_2 \dots x_n$ uno de sus líderes. Sea $i \in \text{supp}(\mathbf{x})$ (fijo) y considere $\mathbf{x}' = x'_1x'_2 \dots x'_n$ tal que

$$x'_j = \begin{cases} x_j, & \forall j \neq i, \\ 0, & \text{si } j = i. \end{cases}$$

Es claro que $\text{wt}(\mathbf{x}) = \text{wt}(\mathbf{x}') + 1$. Suponga que $\mathbf{x}'$ no es un líder de su clase lateral. Entonces existe una palabra $\mathbf{c} \in \mathcal{C}$ tal que $\mathbf{x}' + \mathbf{c}$ es un líder de $\mathbf{x}' + \mathcal{C}$. Así

$$\text{wt}(\mathbf{x}' + \mathbf{c}) \leq \text{wt}(\mathbf{x}') - 1 = \text{wt}(\mathbf{x}) - 2. \quad (2.1)$$

Dado que $\mathbf{x}$ y $\mathbf{x}'$ se diferencian en una sola componente entonces

$$\text{wt}(\mathbf{x} + \mathbf{c}) \leq \text{wt}(\mathbf{x}' + \mathbf{c}) + 1. \quad (2.2)$$

En efecto, si $i \in \text{supp}(\mathbf{c})$, entonces $\text{wt}(\mathbf{x} + \mathbf{c}) = \text{wt}(\mathbf{x}' + \mathbf{c}) - 1$, mientras que si $i \notin \text{supp}(\mathbf{c})$ $\text{wt}(\mathbf{x} + \mathbf{c}) = \text{wt}(\mathbf{x}' + \mathbf{c}) + 1$. Luego de (2.1) y (2.2) se sigue que

$$\text{wt}(\mathbf{x} + \mathbf{c}) \leq \text{wt}(\mathbf{x}' + \mathbf{c}) + 1 \leq \text{wt}(\mathbf{x}) - 1,$$

lo que contradice la hipótesis de que $\mathbf{x}$ es un líder de su clase. Así $\mathbf{x}' + \mathcal{C}$ es una clase de peso $s - 1$. Repitiendo este proceso se consigue una clase con peso t , para cada entero t tal que $0 < t < s$. $\square$

Teorema 2.11. (Theorem 11.7.3, [4]) Sean $\mathcal{C}$ un código lineal binario y $\mathcal{C}'$ una clase lateral con peso w de $\mathcal{C}$. Sea a un entero tal que $0 < a < w$. Entonces el número de descendientes de $\mathcal{C}'$ de peso a es igual al número de descendientes de $\mathcal{C}'$ de peso $w - a$.

Demostración. Por la demostración del Teorema 2.10, los líderes de los descendientes de peso a (respectivamente de peso $w - a$) de $\mathcal{C}'$ son los vectores de peso a (respectivamente de peso $w - a$) que están cubiertos por algún líder de $\mathcal{C}'$. Sea $\mathbf{x}$ un líder de peso a de un descendiente de $\mathcal{C}'$. Entonces existe un líder $\mathbf{x}'$ de $\mathcal{C}'$ tal que $\mathbf{x} \prec \mathbf{x}'$. Como $\mathbf{x}'$ cubre a $\mathbf{x}$, entonces

$$\text{wt}(\mathbf{x}' - \mathbf{x}) = \text{wt}(\mathbf{x}') - \text{wt}(\mathbf{x}) = w - a.$$

Además, $\mathbf{x}' - \mathbf{x}$ es un líder de su clase, pues en caso contrario existiría $\mathbf{c} \in \mathcal{C}$ tal que $(\mathbf{x}' - \mathbf{x}) + \mathbf{c}$ es un líder de $(\mathbf{x}' - \mathbf{x}) + \mathcal{C}$, es decir

$$\text{wt}(\mathbf{x}' - \mathbf{x} + \mathbf{c}) < \text{wt}(\mathbf{x}' - \mathbf{x}).$$

Esta desigualdad implica que $\text{supp}(\mathbf{c}) \subseteq \text{supp}(\mathbf{x}' - \mathbf{x}) \subset \text{supp}(\mathbf{x}')$ y entonces

$$\text{wt}(\mathbf{x}' + \mathbf{c}) < \text{wt}(\mathbf{x}'),$$

lo que es una contradicción. Por lo tanto, $\mathbf{x}' - \mathbf{x}$ es un líder de peso $w - a$ de un descendiente de $\mathcal{C}'$. Ahora si $\mathbf{y}$ es un líder de peso $w - a$ de un descendiente de $\mathcal{C}'$, entonces existe un líder $\bar{\mathbf{x}}$ de $\mathcal{C}'$ tal que $\mathbf{y} \prec \bar{\mathbf{x}}$ y así $\mathbf{y} = \bar{\mathbf{x}} - \mathbf{x}$; donde $\mathbf{x} = \bar{\mathbf{x}} - \mathbf{y}$ es de peso a . Esto indica que todos los líderes de peso $w - a$ de un descendiente de $\mathcal{C}'$ se pueden escribir de la forma $\bar{\mathbf{x}}' - \mathbf{x}$, donde $\bar{\mathbf{x}}'$ es un líder de $\mathcal{C}'$ y $\mathbf{x}$ es de peso a . De esta forma existe una correspondencia inyectiva entre los líderes de peso a de descendientes de $\mathcal{C}'$ y aquellos de peso $w - a$.

Supóngase que $\mathbf{x}$ y $\mathbf{y}$ son líderes de peso a del mismo descendiente de $\mathcal{C}'$. Sean $\mathbf{x}'$ y $\mathbf{y}'$ vectores de peso w en $\mathcal{C}'$ que cubren a $\mathbf{x}$ y $\mathbf{y}$, respectivamente. Entonces $\mathbf{y} - \mathbf{x} \in \mathcal{C}$ y dado que $\mathbf{x}' - \mathbf{y}'$ también está en $\mathcal{C}$, se tiene que $(\mathbf{x}' - \mathbf{x}) - (\mathbf{y}' - \mathbf{y}) = (\mathbf{x}' - \mathbf{y}') + (\mathbf{y} - \mathbf{x}) \in \mathcal{C}$. Por lo tanto, $\mathbf{x}' - \mathbf{x}$ y $\mathbf{y}' - \mathbf{y}$ son líderes de la misma clase lateral. Así, hay tantos descendientes de $\mathcal{C}'$ de peso a como los hay de peso $w - a$. De nuevo intercambiando los roles de a y $w - a$, se demuestra que hay tantos descendientes de $\mathcal{C}'$ de peso $w - a$ como los hay de peso a . $\square$

Teorema 2.12. (Theorem 11.7.5, [4]) Sean $\mathcal{C}_1$ y $\mathcal{C}_2$ dos clases laterales de un código $\mathcal{C}$ y asuma que $\mathcal{C}_1 \prec \mathcal{C}_2$. Sean $\mathbf{x}_1$ y $\mathbf{x}_2$ líderes de $\mathcal{C}_1$ y $\mathcal{C}_2$ respectivamente, tales que $\mathbf{x}_1 \prec \mathbf{x}_2$. Sea $\mathbf{u} = \mathbf{x}_2 - \mathbf{x}_1$, entonces un vector $\mathbf{x}$ es un líder de $\mathcal{C}_1$ si y solo si existe un líder $\mathbf{y}$ de $\mathcal{C}_2$ tal que $\mathbf{u} \preceq \mathbf{y}$ y $\mathbf{x} = \mathbf{y} - \mathbf{u}$.

Demostración. Supongáse que $\mathbf{x}$ es un líder de $\mathcal{C}_1$, entonces

$$\begin{aligned} (\mathbf{x} + \mathbf{u}) - \mathbf{x}_2 &= (\mathbf{x} + \mathbf{x}_2 - \mathbf{x}_1) - \mathbf{x}_2 \\ &= \mathbf{x} - \mathbf{x}_1 \in \mathcal{C}. \end{aligned}$$

Así $\mathbf{x} + \mathbf{u} \in \mathcal{C}_2$, además $\text{wt}(\mathbf{u}) = \text{wt}(\mathbf{x}_2) - \text{wt}(\mathbf{x}_1)$ porque $\mathbf{x}_1 \prec \mathbf{x}_2$. Con esto y por el Lema 1.9, se tiene que

$$\begin{aligned} \text{wt}(\mathbf{x} + \mathbf{u}) &\leq \text{wt}(\mathbf{x}) + \text{wt}(\mathbf{u}) \\ &= \text{wt}(\mathbf{x}_1) + \text{wt}(\mathbf{u}) \\ &= \text{wt}(\mathbf{x}_2). \end{aligned}$$

Por lo tanto, $\mathbf{y} = \mathbf{x} + \mathbf{u}$ es un líder de $\mathcal{C}_2$, con $\mathbf{u} \preceq \mathbf{y}$. Recíprocamente, sea $\mathbf{y}$ un líder de $\mathcal{C}_2$ con $\mathbf{u} \preceq \mathbf{y}$, entonces

$$\begin{aligned} (\mathbf{y} - \mathbf{u}) - \mathbf{x}_1 &= \mathbf{y} - (\mathbf{u} + \mathbf{x}_1) \\ &= \mathbf{y} - \mathbf{x}_2 \in \mathcal{C}. \end{aligned}$$

Así $\mathbf{y} - \mathbf{u}$ está en $\mathcal{C}_1$. Como $\mathbf{u} \preceq \mathbf{y}$, se tiene que $\text{wt}(\mathbf{u}) \leq \text{wt}(\mathbf{y})$. Con esto se sigue que

$$\begin{aligned} \text{wt}(\mathbf{y} - \mathbf{u}) &= \text{wt}(\mathbf{y}) - \text{wt}(\mathbf{u}) \\ &= \text{wt}(\mathbf{x}_2) - \text{wt}(\mathbf{u}) \\ &= \text{wt}(\mathbf{x}_2) - \text{wt}(\mathbf{x}_2 - \mathbf{x}_1) \\ &= \text{wt}(\mathbf{x}_2) - \text{wt}(\mathbf{x}_2) + \text{wt}(\mathbf{x}_1) \\ &= \text{wt}(\mathbf{x}_1). \end{aligned}$$

En consecuencia, $\mathbf{x} = \mathbf{y} - \mathbf{u}$ es un líder de $\mathcal{C}_1$. □

Proposición 2.13. *Si $\mathbf{u} \preceq \mathbf{y}$ y $\mathbf{x} = \mathbf{y} - \mathbf{u}$, entonces $\mathbf{x} \preceq \mathbf{x} + \mathbf{u}$.*

Demostración. Si $\mathbf{u} \preceq \mathbf{y}$ entonces $\text{supp}(\mathbf{y} - \mathbf{u}) \subseteq \text{supp}(\mathbf{y})$, pero como $\mathbf{x} = \mathbf{y} - \mathbf{u}$, $\text{supp}(\mathbf{x}) \subseteq \text{supp}(\mathbf{x} + \mathbf{u})$. Por lo tanto, $\mathbf{x} \preceq \mathbf{x} + \mathbf{u}$. □

El siguiente resultado demuestra que la relación de orden $\preceq$ no depende de los líderes que se tomen.

Corolario 2.14. *Sean $\mathcal{C}_1$ y $\mathcal{C}_2$ dos clases laterales de un código lineal binario $\mathcal{C}$, con $\mathcal{C}_1 \preceq \mathcal{C}_2$. Si $\mathbf{x}$ es un líder de $\mathcal{C}_1$, entonces existe $\mathbf{y}$ un líder de $\mathcal{C}_2$ tal que $\mathbf{x} \preceq \mathbf{y}$, pero no existe un líder $\mathbf{z}$ de $\mathcal{C}_2$ tal que $\mathbf{z} \prec \mathbf{x}$.*

Demostración. Sea $\mathbf{x}$ un líder de $\mathcal{C}_1$, por la hipótesis de que $\mathcal{C}_1 \preceq \mathcal{C}_2$, existen $\mathbf{x}_1$ y $\mathbf{x}_2$ líderes de $\mathcal{C}_1$ y $\mathcal{C}_2$ respectivamente tales que $\mathbf{x}_1 \preceq \mathbf{x}_2$. Por el Teorema 2.12, existe un líder $\mathbf{y}$ de $\mathcal{C}_2$, tal que si $\mathbf{u} = \mathbf{x}_2 - \mathbf{x}_1$ entonces $\mathbf{u} \preceq \mathbf{y}$ y además $\mathbf{x} = \mathbf{y} - \mathbf{u}$. Por la proposición anterior $\mathbf{x} \preceq \mathbf{y}$.

Asuma que existe un líder $\mathbf{z}$ de $\mathcal{C}_2$ tal que $\mathbf{z} \prec \mathbf{x}$ y que $\mathbf{y}$ es un líder de $\mathcal{C}_2$, tal que $\mathbf{x} \preceq \mathbf{y}$; dado que $\mathbf{z} \prec \mathbf{x}$, esto significa que $\text{supp}(\mathbf{z}) \subset \text{supp}(\mathbf{x})$ lo que implica que $\text{wt}(\mathbf{z}) < \text{wt}(\mathbf{x})$. Sin embargo, por hipótesis $\mathbf{x} \preceq \mathbf{y}$, es decir $\text{wt}(\mathbf{x}) \leq \text{wt}(\mathbf{y})$, lo que lleva a una contradicción. □

Del anterior corolario se puede concluir que la relación $\preceq$ definida en el conjunto de clases laterales está bien definida.

Observación 2.15. Para $1 \leq i \leq n$, $\mathbf{e}_i \in \mathbb{F}_2^n$ denota el i -ésimo vector canónico, es decir, el vector que tiene 1 en la coordenada i y 0 en las demás coordenadas.

Corolario 2.16. (Corollary 11.7.7, [4]) Sean $\mathcal{C}$ un código lineal binario y $\mathcal{C}'$ una clase lateral de $\mathcal{C}$ con un líder $\mathbf{x}$. Si $i \in \text{supp}(\mathbf{x})$, entonces $\mathbf{x} - \mathbf{e}_i$ es un líder de un hijo de $\mathcal{C}'$, y ningún líder de $(\mathbf{x} - \mathbf{e}_i) + \mathcal{C}$ tiene a i en su soporte.

Demostración. Dado que $\mathbf{x} - \mathbf{e}_i \preceq \mathbf{x}$, como $\mathbf{e}_i \preceq \mathbf{x}$, se tiene que $\text{wt}(\mathbf{x} - \mathbf{e}_i) = \text{wt}(\mathbf{x}) - 1$. Además, los elementos de la clase lateral $(\mathbf{x} - \mathbf{e}_i) + \mathcal{C}$ son de la forma $\mathbf{x} - \mathbf{e}_i + \mathbf{c}$, $\mathbf{c} \in \mathcal{C}$. Luego, $\text{wt}(\mathbf{x} - \mathbf{e}_i + \mathbf{c}) = \text{wt}(\mathbf{x} + \mathbf{c}) - 1$, pero $\mathbf{x}$ es líder de $\mathcal{C}'$, así $\mathbf{x} - \mathbf{e}_i$ es un líder de un hijo de $\mathcal{C}'$. Por el Teorema 2.12 cada líder de este hijo tiene la forma $\mathbf{y} - \mathbf{e}_i$, donde $\mathbf{y}$ es un líder de $\mathcal{C}'$ y $\mathbf{e}_i \preceq \mathbf{y}$. En particular, ningún líder del hijo tiene a i en su soporte. $\square$

Ejemplo 2.17. Sea $\mathcal{C}$ el $[6, 2, 4]$ -código binario con una matriz generadora

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Para la clase lateral $\mathcal{C}' = \{101001, 010101, 100110, 011010\}$ del código $\mathcal{C}$, uno de sus líderes es 101001, cuyo soporte es $\text{supp}(101001) = \{1, 3, 6\}$. Por el corolario anterior $101001 - \mathbf{e}_6 = 101000$ es un líder de uno de los hijos de la clase lateral $\mathcal{C}'$. Además, se puede observar que este hijo también tiene como líder el vector $\mathbf{x} = 010100$ y $6 \notin \text{supp } \mathbf{x}$.

El corolario anterior da un método para encontrar los líderes de los hijos de una clase lateral, a partir de sus líderes.

Dada una clase lateral $\mathcal{C}'$ de un código lineal binario, se denota con $\mathcal{CL}(\mathcal{C}')$ el conjunto de los líderes de $\mathcal{C}'$ y con $S_{\mathcal{C}'}$ el conjunto de todos los índices $i \in \{1, \dots, n\}$ tales que $\mathbf{e}_i$ está cubierto por algún líder de $\mathcal{C}'$, es decir

$$S_{\mathcal{C}'} = \{i : \exists \mathbf{x} \in \mathcal{CL}(\mathcal{C}') \text{ tal que } \mathbf{e}_i \preceq \mathbf{x}\} = \bigcup_{\mathbf{x} \in \mathcal{CL}(\mathcal{C}')} \text{supp}(\mathbf{x}).$$

Adicionalmente, con $\overline{S}_{\mathcal{C}'}$ se denota el complemento de $S_{\mathcal{C}'}$.

Teorema 2.18. (Theorem 11.7.10, [4]) *Sea $\mathcal{C}$ un $[n, k, d]$ -código lineal binario. Sea $\mathcal{C}'$ una clase lateral de $\mathcal{C}$ con peso w . Entonces se cumplen las siguientes afirmaciones:*

- (i) $\mathcal{C}'$ es huérfano si y solo si para todo $1 \leq i \leq n$, existe $\mathbf{x} \in \mathcal{C}'$ tal que $\mathbf{e}_i \preceq \mathbf{x}$ con $\text{wt}(\mathbf{x}) \in \{w, w+1\}$.
- (ii) Si $d > 2$, entonces existe una correspondencia uno a uno entre los hijos de $\mathcal{C}'$ y $S_{\mathcal{C}'}$.
- (iii) Si $d > 2$ y $\mathcal{C}$ es un código par, entonces existe una correspondencia uno a uno entre los padres de $\mathcal{C}'$ y $\overline{S}_{\mathcal{C}'}$. En particular, $\mathcal{C}'$ es un huérfano si y solo si $\overline{S}_{\mathcal{C}'} = \emptyset$.

Demostración.

- (i) Por el Corolario 2.14 cada padre de $\mathcal{C}'$ es de la forma $\mathbf{e}_i + \mathcal{C}'$ para algún $i \in \{1, \dots, n\}$. Para que $\mathcal{C}'$ sea huérfano, para cada i la clase lateral $\mathbf{e}_i + \mathcal{C}'$ debe contener vectores de peso menor o igual que w . Así $\mathcal{C}'$ es huérfano si y solo si para cada i existe un vector en $\mathcal{C}'$ de peso w o $w+1$, con un 1 en la i -ésima coordenada.
- (ii) Como $d > 2$, entonces para $i \neq j$, las clases laterales $\mathbf{e}_i + \mathcal{C}'$ y $\mathbf{e}_j + \mathcal{C}'$ son distintas, pues si fueran iguales $\mathbf{x} + \mathbf{e}_i = \mathbf{x} + \mathbf{e}_j + \mathbf{c}$ para algún $\mathbf{c} \in \mathcal{C}$. Entonces $\mathbf{e}_i + \mathbf{e}_j \in \mathcal{C}$ y como $\text{wt}(\mathbf{e}_i + \mathbf{e}_j) = 2$ se contradice la hipótesis. De esta forma, $\mathbf{e}_i + \mathcal{C}'$ es un hijo de $\mathcal{C}'$ si y solo si existe $\mathbf{x}$ un líder de $\mathcal{C}'$ tal que $i \in \text{supp}(\mathbf{x})$ o equivalentemente $\mathbf{e}_i \preceq \mathbf{x}$.
- (iii) Suponga que $\mathcal{C}$ es un código par y $d > 2$. Cada padre de $\mathcal{C}'$ es igual a $\mathbf{e}_i + \mathcal{C}'$ para algún $i \in \{1, \dots, n\}$. Para que $\mathbf{e}_i + \mathcal{C}'$ sea un padre de $\mathcal{C}'$, por el item (i) se debe tener que cada vector de $\mathcal{C}'$ de peso w o $w+1$ no tenga un 1 en la coordenada i . Como $\mathcal{C}$ es un código par, una clase lateral tiene solo vectores de peso par o solo vectores de peso impar. En efecto, para $\mathbf{y} = \mathbf{x} + \mathbf{c} \in \mathbf{x} + \mathcal{C}$, se tiene que

$$\text{wt}(\mathbf{y}) = \text{wt}(\mathbf{x} + \mathbf{c}) = \text{wt}(\mathbf{x}) + \text{wt}(\mathbf{c}) - 2\text{wt}(\mathbf{x} \star \mathbf{c}).$$

Como $\text{wt}(\mathbf{c})$ es par, esto implica que $\text{wt}(\mathbf{y})$ tiene la misma paridad de $\text{wt}(\mathbf{x})$ y por tanto todos los elementos de la clase $\mathbf{x} + \mathcal{C}$ tienen la misma paridad. En consecuencia, como $\text{wt}(\mathcal{C}') = w$, entonces $\mathcal{C}'$ no tiene vectores de peso $w+1$. Observe que $\mathbf{e}_i + \mathcal{C}' \neq \mathbf{e}_j + \mathcal{C}'$, si $i \neq j$. Si $i \in \overline{S}_{\mathcal{C}'}$, entonces $i \notin \text{supp}(\mathbf{x})$ para ningún $\mathbf{x} \in \mathcal{CL}(\mathcal{C}')$. Esto implica que

$\text{wt}(\mathbf{e}_i + \mathbf{x}) = w + 1$, para todo $\mathbf{x} \in \mathcal{CL}(\mathcal{C}')$ y por lo tanto $\mathbf{e}_i + \mathcal{C}'$ es un padre de $\mathcal{C}'$. Recíprocamente, si $\mathbf{e}_j + \mathcal{C}'$ es un padre de $\mathcal{C}'$ entonces $\text{wt}(\mathbf{e}_j + \mathcal{C}') = w + 1$. Por el Corolario 2.14, para cada líder $\mathbf{x}$ de $\mathcal{C}'$ existe un líder $\mathbf{y}$ de $\mathbf{e}_j + \mathcal{C}'$ tal que $\mathbf{x} \preceq \mathbf{y}$. Entonces $\mathbf{y} = \mathbf{e}_j + \mathbf{x}$, donde $j \in \overline{S}_{\mathcal{C}'}$. Para probar la segunda parte, observe por la correspondencia uno a uno entre padres de $\mathcal{C}'$ y $\overline{S}_{\mathcal{C}'}$, se tiene que $\mathcal{C}'$ es huérfano si y solo si $\overline{S}_{\mathcal{C}'} = \emptyset$.

□

Observación 2.19. En el Ejemplo 2.9 se evidencia que para que se cumpla el item (iii) del Teorema 2.18 es necesario que el código sea par, porque los líderes $\mathcal{C}_6$ y $\mathcal{C}_7$ son huérfanos pero no cubren todos los vectores canónicos.

Corolario 2.20. (Corollary 11.7.11, [4]) Sea $\mathcal{C}$ un $[n, k, d]$ -código lineal binario par, con $d > 2$. Se tienen las siguientes afirmaciones:

- (i) Un huérfano de $\mathcal{C}$ tiene n descendientes de peso 1 y n hijos.
- (ii) Una clase lateral de peso w , la cual tiene exactamente t líderes tal que $tw < n$, no es un huérfano.

Demostración.

- (i) Suponga que $\mathcal{C}'$ es huérfano de $\mathcal{C}$. Por hipótesis $\mathcal{C}$ es un código lineal binario par y $d > 2$, entonces por Teorema 2.18(iii), para todo $1 \leq i \leq n$, existe un líder $\mathbf{x}$ de $\mathcal{C}'$ tal que $\mathbf{e}_i \preceq \mathbf{x}$, por esto $\mathbf{e}_i + \mathcal{C}$ es un descendiente de $\mathcal{C}'$ y por lo tanto $\mathcal{C}'$ tiene n descendientes de peso 1. Ahora, por el Corolario 2.16 cada hijo de $\mathcal{C}'$ es de la forma $\mathbf{e}_i + \mathcal{C}'$ para algún $1 \leq i \leq n$. Por la parte (ii) del Teorema 2.18 existe una correspondencia uno a uno entre los hijos de $\mathcal{C}'$ y los vectores $\mathbf{e}_i$ cubiertos por un líder de $\mathcal{C}'$. Por lo tanto, para todo $i \in \{1, \dots, n\}$, $\mathbf{e}_i + \mathcal{C}'$ es un hijo de $\mathcal{C}'$.
- (ii) Por el Teorema 2.18 parte (ii), existe una correspondencia uno a uno entre los hijos de $\mathcal{C}'$ y los vectores canónicos $\mathbf{e}_i$ cubiertos por los líderes de $\mathcal{C}'$, por definición de peso de Hamming se tiene que cada líder de $\mathcal{C}'$ tiene w coordenadas iguales a 1, o de forma equivalente cubre w vectores $\mathbf{e}_i$ y como son t líderes entonces a lo más tw vectores canónicos son cubiertos por estos líderes. Ahora por hipótesis se tiene que $tw < n$ y esto significa que no todas las coordenadas están cubiertas por los líderes de $\mathcal{C}'$, esto implica que $\mathcal{C}'$ no es huérfano.

□

El algoritmo `diagrama_Hasse_codigo` (ver Algoritmo 2) se construyó con base en algunos de los resultados anteriores, el Corolario 2.16 en su enunciado y prueba proporcionó la manera de encontrar los líderes de los hijos de una clase lateral de un código. Además, el algoritmo auxiliar `huerfanos` (ver Algoritmo 5) está basado en la parte (i) del Teorema 2.18.

A continuación se presentan algunos ejemplos que se calcularon con ayuda del Algoritmo 2.

Ejemplo 2.21. Sea $\mathcal{C}$ el $[5, 2, 3]$ -código binario con una matriz generadora dada por

$$M_1 = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix}.$$

```
M1=matrix(GF(2),[[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
diagrama_Hasse_codigo(C1)
{'(0, 0, 0, 0, 0)': [],
 '(1, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 1, 0, 0, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 0, 1, 0, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 0, 0, 1, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0)'],
 '(1, 0, 0, 0, 1)': ['(0, 0, 0, 0, 1)'],
 '(1, 0, 0, 0, 0)',
 '(0, 0, 1, 0, 0)',
 '(0, 1, 0, 0, 0)'],
 '(0, 0, 1, 0, 1)': ['(0, 0, 0, 0, 1)'],
 '(0, 0, 1, 0, 0)',
 '(0, 1, 0, 0, 0)',
 '(1, 0, 0, 0, 0)']}]
```

Este ejemplo implementado en el algoritmo `diagrama_Hasse_codigo` (ver Algoritmo 2), muestra que el código tiene 8 líderes y que 00010, 00101 y 10001 son huérfanos. Según las relaciones

que tienen los líderes 00101 y 10001 con sus hijos, se sabe que no son líderes únicos en su clase lateral porque por el Corolario 2.16 el soporte de los padres contienen el soporte de los hijos.

El siguiente diagrama de Hasse (ver Definición 2.8) muestra las relaciones de las clases laterales con respecto a $\prec$ en el código $\mathcal{C}$.

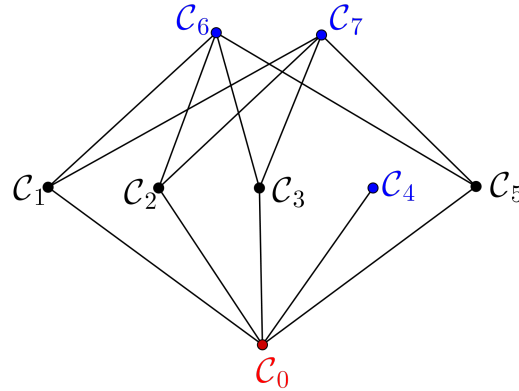


Figura 2.2: Diagrama de Hasse del código del Ejemplo 2.21.

Con esta figura se evidencia la relación entre padres e hijos del código. Además, los huérfanos son elementos maximales en el gráfico o que no tienen padres; es de anotar que el código tiene 3 huérfanos, $\mathcal{C}_4$ que tiene como líder un vector con peso $\text{wt}(\mathcal{C}_4) = 2$, $\mathcal{C}_6$ cuyo líder tiene peso 3 y $\mathcal{C}_7$ su líder con $\text{wt}(\mathcal{C}_7) = 3$.

Ejemplo 2.22. Para el Ejemplo 2.9 usando el algoritmo `diagrama_Hasse_codigo` en SageMath, se obtienen los siguientes resultados.

```
M2=matrix(GF(2),[[1,1,1,1,0],[0,0,1,1,1]])
C2=LinearCode(M2)
diagrama_Hasse_codigo(C2)
{'(0, 0, 0, 0, 0)': [],
 '(1, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 1, 0, 0, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 0, 1, 0, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 0, 0, 1, 0)': ['(0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0)'],
```

```
'(1, 0, 0, 1, 0)': ['(0, 0, 0, 1, 0)',
'(1, 0, 0, 0, 0)',
'(0, 0, 1, 0, 0)',
'(0, 1, 0, 0, 0)'],
'(0, 1, 0, 1, 0)': ['(0, 0, 0, 1, 0)',
'(0, 1, 0, 0, 0)',
'(0, 0, 1, 0, 0)',
'(1, 0, 0, 0, 0)']}]}
```

Este código implementado en el Algoritmo `diagrama_Hasse_codigo` (ver Algoritmo 2) tiene 8 líderes y su gráfico con las conexiones correspondientes (ver Ejemplo 2.9). Tiene 1 líder de peso 0, 5 líderes de peso 1 y 2 líderes de peso de 2. Las clases laterales $\mathcal{C}_5$, $\mathcal{C}_6$ y $\mathcal{C}_7$ son huérfanas, el líder de $\mathcal{C}_5$ es de peso 1 y para las clases laterales $\mathcal{C}_6$ y $\mathcal{C}_7$ los líderes tienen peso 2. Por el Corolario 2.16, las clases laterales $\mathcal{C}_6$ y $\mathcal{C}_7$ tienen más de un líder, porque tienen 4 hijos y $\text{wt}(\mathcal{C}_6) = \text{wt}(\mathcal{C}_7) = 2$.

Ejemplo 2.23. Sea $\mathcal{C}$ el $[6, 3, 3]$ -código binario con matriz generadora

$$M_3 = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

Los cálculos realizados en SageMath, son los siguientes:

```
M3=matrix(GF(2),[[1,0,0,0,1,1],[0,1,0,1,0,1],[0,0,1,1,1,0]])
C3=LinearCode(M3)
diagrama_Hasse_codigo(C3)
{'(0, 0, 0, 0, 0, 0)': [],
'(1, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
'(0, 1, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
'(0, 0, 1, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 1, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 0, 1, 0)': ['(0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0, 0)'],
```

```

'(0, 0, 1, 0, 0, 1)': ['(0, 0, 0, 0, 0, 1)',
'(0, 0, 1, 0, 0, 0)',
'(0, 0, 0, 1, 0, 0)',
'(1, 0, 0, 0, 0, 0)',
'(0, 0, 0, 0, 1, 0)',
'(0, 1, 0, 0, 0, 0)']}]

```

En la Figura 2.3 se representa el diagrama de Hasse del código $\mathcal{C}$ con las relaciones encontradas con el Algoritmo `diagrama_Hasse_codigo`, ver p.p. 81.

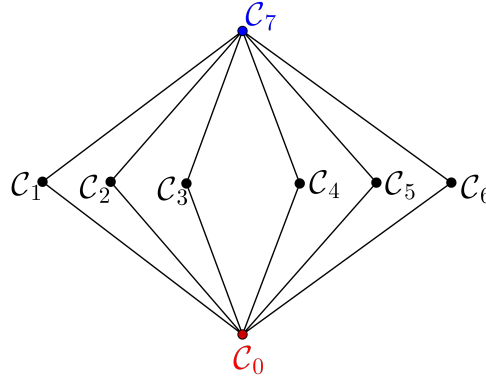


Figura 2.3: Diagrama de Hasse del código del Ejemplo 2.23.

Con estos cálculos se puede ver que este código tiene 8 líderes, 6 líderes son los vectores canónicos, tiene un único líder de peso 2. Por el Corolario 2.16 la clase lateral $\mathcal{C}_7$ no tiene un líder único, porque esta cubierta por 6 clases laterales pero $\text{wt}(\mathcal{C}_7) = 2$, así se puede concluir que no es único y que tiene 3 líderes.

Con la Figura 2.3 es evidente que la única clase lateral huérfana es $\mathcal{C}_7$ y que cubre las clases laterales de los vectores canónicos de $\mathbb{F}_2^6$.

Ejemplo 2.24. Sea $\mathcal{C}$ el $[6, 2, 4]$ -código binario con una matriz generadora

$$M_4 = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Al realizar los cálculos respectivos con ayuda de **SageMath** se obtuvieron los siguientes resultados.

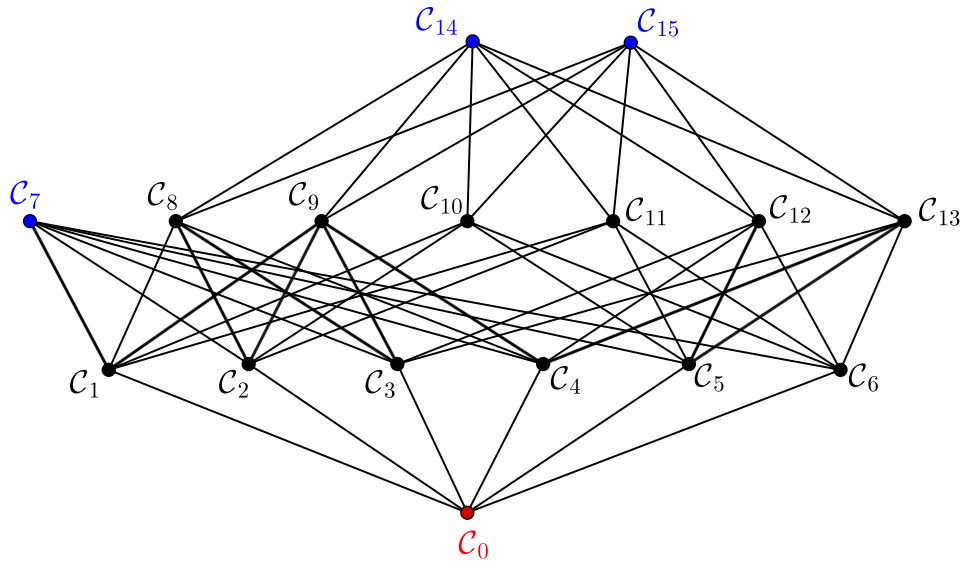


Figura 2.4: Diagrama de Hasse del código del Ejemplo 2.24.

```

M4=matrix(GF(2),[[1,1,1,1,0,0],[0,0,1,1,1,1]])
C4=LinearCode(M4)
diagrama_Hasse_codigo(C4)
{'(0, 0, 0, 0, 0, 0)': [],
 '(1, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
 '(0, 1, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
 '(0, 0, 1, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 1, 0, 0)': ['(0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 1, 0)': ['(0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0, 0)'],
 '(1, 0, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0, 1)',
 '(1, 0, 0, 0, 0, 0)',
 '(0, 0, 0, 0, 1, 0)',
 '(0, 1, 0, 0, 0, 0)'],
 '(0, 1, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0, 1)',
 '(0, 1, 0, 0, 0, 0)',
 '(0, 0, 0, 0, 1, 0)'],

```

```

'(1, 0, 0, 0, 0, 0)' ],
'(0, 0, 1, 0, 0, 1)': ['(0, 0, 0, 0, 0, 1)',
'(0, 0, 1, 0, 0, 0)',
'(0, 0, 0, 0, 1, 0)',
'(0, 0, 0, 1, 0, 0)'],
'(0, 0, 0, 1, 0, 1)': ['(0, 0, 0, 0, 0, 1)',
'(0, 0, 0, 1, 0, 0)',
'(0, 0, 0, 0, 1, 0)',
'(0, 0, 1, 0, 0, 0)'],
'(0, 0, 0, 0, 1, 1)': ['(0, 0, 0, 0, 0, 1)',
'(0, 0, 0, 0, 1, 0)',
'(0, 0, 0, 1, 0, 0)',
'(0, 0, 1, 0, 0, 0)',
'(0, 1, 0, 0, 0, 0)',
'(1, 0, 0, 0, 0, 0)'],
'(1, 0, 0, 1, 0, 1)': ['(0, 0, 0, 1, 0, 1)',
'(1, 0, 0, 0, 0, 1)',
'(0, 0, 1, 0, 0, 1)',
'(0, 1, 0, 0, 0, 1)'],
'(0, 1, 0, 1, 0, 1)': ['(0, 0, 0, 1, 0, 1)',
'(0, 1, 0, 0, 0, 1)',
'(0, 0, 1, 0, 0, 1)',
'(1, 0, 0, 0, 0, 1)'],
'(1, 0, 0, 1, 0, 0)': ['(0, 0, 0, 1, 0, 0)',
'(1, 0, 0, 0, 0, 0)',
'(0, 0, 1, 0, 0, 0)',
'(0, 1, 0, 0, 0, 0)'],
'(0, 1, 0, 1, 0, 0)': ['(0, 0, 0, 1, 0, 0)',
'(0, 1, 0, 0, 0, 0)',

```

```
'(0, 0, 1, 0, 0, 0)' ,
'(1, 0, 0, 0, 0, 0)']}]}
```

El código tiene 16 líderes de pesos 0, 1, 2 y 3. La Figura 2.4 resume la información encontrada en SageMath. En este ejemplo se evidencia el ítem (iii) del Teorema 2.18, ya que el código es par y tiene $d > 2$. La clase lateral $\mathcal{C}_7$ que tiene $\text{wt}(\mathcal{C}_7) = 2$ es un huérfano y cubre las clases laterales de las que los líderes son dos vectores canónicos. La clase lateral $\mathcal{C}_{14}$, que tiene $\text{wt}(\mathcal{C}_{14}) = 3$, es un huérfano y tiene como descendientes las 6 clases laterales de las cuales los líderes son los vectores canónicos, al igual que la clase lateral $\mathcal{C}_{15}$. Es así como se puede ver que los huérfanos de este código tienen 6 descendientes de peso 1.

Ejemplo 2.25. El código del Ejemplo 1.23 es un código perfecto con matriz generadora:

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Los cálculos para este código en SageMath son:

```
CH=codes.HammingCode(GF(2), 3)
diagrama_Hasse_codigo(CH)
{'(0, 0, 0, 0, 0, 0, 0)': [],
 '(1, 0, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0)'],
 '(0, 1, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0)'],
 '(0, 0, 1, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 1, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 1, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 0, 1, 0)': ['(0, 0, 0, 0, 0, 0, 0)'],
 '(0, 0, 0, 0, 0, 0, 1)': ['(0, 0, 0, 0, 0, 0, 0)']}
```

Este código de Hamming tiene 8 líderes, 7 líderes son los vectores canónicos y el vector $\mathbf{0}$. El diagrama de Hasse (ver, Figura 2.5) se construyó con la información del Algoritmo 2.

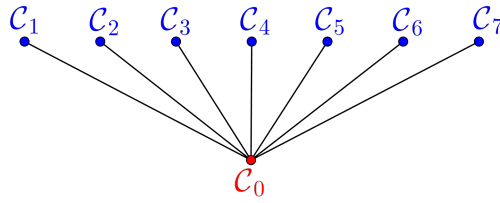


Figura 2.5: Diagrama de Hasse del código del Ejemplo 2.25.

Para los siguientes ejemplos solo se consignan los datos que relacionan el peso y número de líderes debido la longitud de los códigos que se usaron.

Ejemplo 2.26. Sea $\mathcal{C}$ el $[8, 5, 2]$ -código binario con matriz generadora

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Los cálculos realizados para este código en SageMath fueron:

```
M=matrix(GF(2),[[1,0,0,0,0,1,1,0],[0,1,0,0,0,1,0,1],
[0,0,1,0,0,0,1,0],[0,0,0,1,0,0,0,1],[0,0,0,0,1,1,1,1]])
C=LinearCode(M)
diagrama_hasse_codigo(C)
{'(0, 0, 0, 0, 0, 0, 0, 0)': [],
'(1, 0, 0, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0, 0)'],
'(0, 1, 0, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0, 0)'],
'(0, 0, 1, 0, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 1, 0, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 0, 1, 0, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 0, 0, 1, 0, 0)': ['(0, 0, 0, 0, 0, 0, 0, 0)'],
'(0, 0, 0, 0, 1, 1, 0, 0)': ['(0, 0, 0, 0, 0, 1, 0, 0)',
'(0, 0, 0, 0, 1, 0, 0, 0)',
'(0, 1, 0, 0, 0, 0, 0, 0)'],
'(0, 1, 0, 0, 0, 0, 0, 0)']
```



```
'(1, 0, 0, 0, 0, 0, 0, 0)',
'(0, 0, 1, 0, 0, 0, 0, 0)',
'(0, 0, 0, 1, 0, 0, 0, 0)']}]}
```

Se puede verificar que el diagrama de Hasse para este código es igual al diagrama del Ejemplo 2.23.

Ejemplo 2.27. Sea $\mathcal{C}$ un $[10, 4, 4]$ -código binario cuya matriz generadora es

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

Los cálculos hechos en SageMath:

```
H1=matrix(GF(2), [[1,0,0,0,1,0,0,0,0,0], [1,0,1,1,0,1,0,0,0,0],
[1,1,0,1,0,0,1,0,0,0], [1,1,1,0,0,0,0,1,0,0],
[1,1,1,1,0,0,0,0,1,0], [1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
R5=diagrama_Hasse_codigo(C5)
len(R5.keys())
16
fun_cont_lid(R5)
Counter({2: 30, 3: 23, 1: 10, 0: 1})
```

El diccionario resultante luego de aplicar la función `diagrama_Hasse_codigo` tiene 16 llaves, y en cada llave se encuentra un líder con sus respectivos hijos. A partir de la información de este diccionario se puede formar un grafo y él mostrará las relaciones que existen entre hijos y padres. Además, la información del número de líderes y sus pesos coincide con la información del algoritmo `arreglo_estandar`.

Peso	0	1	2	3
Número de líderes	1	10	30	23

Tabla 2.3: Número de líderes según su peso.

Ejemplo 2.28. Sea $\mathcal{C}$ el código binario Reed-Muller (1,4). Su implementación en SageMath es la siguiente:

```
C6=codes.BinaryReedMullerCode(1, 4);
R6=diagrama_Hasse_codigo(C6)
len(R6.keys())
2048
fun_cont_lid(R6)
Counter({4: 875, 3: 560, 5: 448, 2: 120, 6: 28, 1: 16, 0: 1})
```

Con el algoritmo `diagrama_Hasse_codigo` se obtuvo un diccionario que contiene 2048 llaves. La siguiente tabla consigna los pesos de los líderes y el número de líderes con ese peso. La tabla corrobora la información que se tenía con el algoritmo `arreglo_estandar` (ver Ejemplo 2.28).

Peso	0	1	2	3	4	5	6
Número de líderes	1	16	120	560	875	448	28

Tabla 2.4: Número de líderes según su peso.

A continuación se presentan algunos resultados que dan información adicional acerca del diagrama de Hasse de un código.

Teorema 2.29. Sea $\mathcal{C}$ un código lineal binario en $\mathbb{F}_2^n$ y $\rho(\mathcal{C})$ su radio de cubrimiento. Se tienen las siguientes igualdades:

$$(i) \quad \rho(\mathcal{C}) = \max_{\mathbf{x} \in \mathbb{F}_2^n} \min_{\mathbf{c} \in \mathcal{C}} d(\mathbf{x}, \mathbf{c}).$$

$$(ii) \quad \rho(\mathcal{C}) = \max_{\mathbf{x} \in \overline{H}(\mathcal{C})} \text{wt}(\mathbf{x}), \text{ donde } \overline{H}(\mathcal{C}) = \{\mathbf{x} \in \mathbb{F}_2^n : d(\mathbf{x}, \mathbf{c}) \geq d(\mathbf{x}, \mathbf{0}); \forall \mathbf{c} \in \mathcal{C}\}.$$

Demostración.

(i) Sea

$$t = \max_{\mathbf{x} \in \mathbb{F}_2^n} \min_{\mathbf{c} \in \mathcal{C}} d(\mathbf{x}, \mathbf{c}). \quad (2.3)$$

Sea $\mathbf{y} \in \mathbb{F}_2^n$, entonces existe $\bar{\mathbf{c}} \in \mathcal{C}$ tal que $d(\mathbf{y}, \bar{\mathbf{c}})$ es la menor distancia de una palabra del código a $\mathbf{y}$. Así de (2.3) se tiene que $d(\mathbf{y}, \bar{\mathbf{c}}) \leq t$ y por lo tanto $\mathbf{y} \in S(\bar{\mathbf{c}}, t)$. Esto implica

que $\mathbb{F}_2^n = \bigcup_{\mathbf{c} \in \mathcal{C}} S(\mathbf{c}, t)$ y por la Definición 1.19 se sigue que $\rho(\mathcal{C}) \leq t$. Ahora, sea $\mathbf{z} \in \mathbb{F}_2^n$ tal que $t = d(\mathbf{z}, \mathbf{c})$ para algún $\mathbf{c} \in \mathcal{C}$. Como $\mathbb{F}_2^n = \bigcup_{\mathbf{c} \in \mathcal{C}} S(\mathbf{c}, \rho(\mathcal{C}))$, entonces existe $\bar{\mathbf{c}} \in \mathcal{C}$ tal que $\mathbf{z} \in S(\bar{\mathbf{c}}, \rho(\mathcal{C}))$, esto es $d(\mathbf{z}, \bar{\mathbf{c}}) \leq \rho(\mathcal{C})$. Entonces de la definición de t se tiene que

$$t = d(\mathbf{z}, \mathbf{c}) \leq d(\mathbf{z}, \bar{\mathbf{c}}) \leq \rho(\mathcal{C}).$$

Por lo tanto, $\rho(\mathcal{C}) = t$.

- (ii) Sea $s = \max_{\mathbf{x} \in \overline{H}(\mathcal{C})} \text{wt}(\mathbf{x})$ y $\bar{\mathbf{x}} \in \overline{H}(\mathcal{C})$ tal que $s = \text{wt}(\bar{\mathbf{x}}) = d(\bar{\mathbf{x}}, \mathbf{0})$. Por la definición de $\rho(\mathcal{C})$ existe $\mathbf{c} \in \mathcal{C}$ tal que $\bar{\mathbf{x}} \in S(\mathbf{c}, \rho(\mathcal{C}))$. Así,

$$s \leq d(\bar{\mathbf{x}}, \mathbf{c}) \leq \rho(\mathcal{C}). \quad (2.4)$$

Ahora sean $\mathbf{x}', \mathbf{c}' \in \mathbb{F}_2^n$ tales que $\rho(\mathcal{C}) = d(\mathbf{x}', \mathbf{c}')$. Entonces por el item (i) $d(\mathbf{x}', \mathbf{c}') \leq d(\mathbf{x}', \mathbf{c}), \forall \mathbf{c} \in \mathcal{C}$. Como $\mathcal{C}$ es lineal entonces $\mathbf{c} + \mathbf{c}' \in \mathcal{C}$ y así

$$d(\mathbf{x}', \mathbf{c}') \leq d(\mathbf{x}', \mathbf{c} + \mathbf{c}') \quad \forall \mathbf{c} \in \mathcal{C}. \quad (2.5)$$

Dado que $d(\mathbf{x}', \mathbf{c} + \mathbf{c}') = d(\mathbf{x}' - \mathbf{c}', \mathbf{c}')$, (2.5) implica que

$$d(\mathbf{x}' - \mathbf{c}', \mathbf{0}) = d(\mathbf{x}', \mathbf{c}') \leq d(\mathbf{x}' - \mathbf{c}', \mathbf{c}), \forall \mathbf{c} \in \mathcal{C}.$$

Esto significa que $\mathbf{x}' - \mathbf{c}' \in \overline{H}(\mathcal{C})$ y por lo tanto

$$\rho(\mathcal{C}) = \text{wt}(\mathbf{x}' - \mathbf{c}') \leq s. \quad (2.6)$$

De (2.4) y (2.6) se sigue el resultado.

□

Definición 2.30. Sea $\mathcal{C}$ un $[n, k]$ -código lineal binario. Para un vector $\mathbf{x} \in \mathbb{F}_2^n$, un falso vecino es una palabra del código, diferente de cero la cual está tan cerca a $\mathbf{x}$ como a la palabra $\mathbf{0}$. Esto es, $\mathbf{c} \in \mathcal{C}$ es un falso vecino de $\mathbf{x}$ si y solo si $d(\mathbf{x}, \mathbf{c}) \leq d(\mathbf{x}, \mathbf{0}) = \text{wt}(\mathbf{x})$. Un error $\mathbf{e}$ es corregible (únicamente) si no tiene falsos vecinos. Un error que no es corregible (únicamente) se llama un error incorregible.

Se puede demostrar que $\mathbf{e}$ es corregible si y solo si es el único líder de su clase lateral. Sea $E(\mathcal{C})$ el conjunto de todos los errores incorregibles y

$$E_t(\mathcal{C}) = \{\mathbf{x} \in E(\mathcal{C}) : \text{wt}(\mathbf{x}) = t\}$$

el conjunto de errores incorregibles de peso t . Además, sea $\overline{E}(\mathcal{C})$ el conjunto de todos los errores corregibles y $\overline{E}_t(\mathcal{C})$ el conjunto de todos los errores corregibles de peso t . Sean

$$N_t = N_t(\mathcal{C}) = |E_t(\mathcal{C})|,$$

$$\overline{N}_t = \overline{N}_t(\mathcal{C}) = |\overline{E}_t(\mathcal{C})| = \binom{n}{t} - N_t(\mathcal{C}).$$

Definición 2.31. Sea $\mathcal{C}$ un código. El radio de Newton es el mayor valor $\nu(\mathcal{C}) = \nu$ tal que existe una clase lateral de peso ν con un único líder.

De esta manera, $\nu(\mathcal{C})$ es el peso más grande de tal forma que cualquier error que puede ser corregido de forma única. Así, el radio de Newton $\nu(\mathcal{C})$ de $\mathcal{C}$ es el error corregible de mayor peso.

Además de la definición resulta que para un código $\mathcal{C}$, donde,

$$\overline{E}(\mathcal{C}) = \{\mathbf{x} \in \mathbb{F}_2^n : d(\mathbf{x}, \mathbf{c}) > \text{wt}(\mathbf{x}), \forall \mathbf{0} \neq \mathbf{c} \in \mathcal{C}\}, \text{ el radio de Newton es}$$

$$\nu(\mathcal{C}) = \max_{\mathbf{x} \in \overline{E}(\mathcal{C})} \text{wt}(\mathbf{x})$$

Teorema 2.32. (Theorem 11.1.1, [4]) Sea $\mathcal{C}$ un código con distancia mínima d . Entonces $\rho(\mathcal{C}) \geq \lfloor \frac{d-1}{2} \rfloor$. Además se obtiene la igualdad si y solo si $\mathcal{C}$ es perfecto.

Demostración. Sea $\overline{\mathbf{c}} = c_1 c_2 \dots c_n \in \mathcal{C}$ y sea $t = \lfloor \frac{d-1}{2} \rfloor$, considere $\overline{\mathbf{x}}$ tal que $\overline{\mathbf{x}} = x_1 x_2 \dots x_n$, $x_i \neq c_i \forall 1 \leq i \leq t$ y $x_i = c_i$ para $t+1 \leq i \leq n$. De esta forma $d(\overline{\mathbf{x}}, \overline{\mathbf{c}}) = t$.

Así,

$$\min_{\mathbf{c} \in \mathcal{C}} d(\overline{\mathbf{x}}, \mathbf{c}) = t. \quad (2.7)$$

Esto porque si hubiese otra palabra $\mathbf{c}' \in \mathcal{C}$ tal que, $d(\overline{\mathbf{x}}, \mathbf{c}') < t$, entonces $\overline{\mathbf{x}} \in S(\overline{\mathbf{c}}, t) \cap S(\mathbf{c}', t)$ y esto contradice el hecho de que las esferas son disjuntas, ver Proposición 1.20. Por tanto de (2.7)

$$\rho(\mathcal{C}) = \max_{\mathbf{x} \in \mathbb{F}_2^n} \min_{\mathbf{c} \in \mathcal{C}} d(\mathbf{x}, \mathbf{c}) \geq t.$$

Si $t = \rho(\mathcal{C})$, entonces por definición de radio de cubrimiento $\mathbb{F}_2^n = \bigcup_{\mathbf{c} \in \mathcal{C}} S(\mathbf{c}, t)$. Por lo tanto $\rho(\mathcal{C})$ es perfecto.

Si $\mathcal{C}$ es perfecto, entonces las esferas con centro en las palabras del código y radio $\rho(\mathcal{C})$ son disjuntas. Esto significa que $\mathcal{C}$ es un código corrector de $\rho(\mathcal{C})$ -errores. Ahora como $\mathcal{C}$ corrige exactamente t -errores, entonces $\rho(\mathcal{C}) \leq t$ (ver Corolario 1.17). Como ya se probó que $\rho(\mathcal{C}) \geq t$, entonces $\rho(\mathcal{C}) = \lfloor \frac{d-1}{2} \rfloor$. $\square$

Proposición 2.33. *Si $\mathcal{C}$ es un código lineal binario y denotamos con $\mathcal{CL}(\mathcal{C})$ el conjunto de líderes. Entonces $\overline{H}(\mathcal{C}) = \mathcal{CL}(\mathcal{C})$.*

Demostración. Por definición,

$$\begin{aligned} \mathbf{x} \in \overline{H}(\mathcal{C}) &\Leftrightarrow d(\mathbf{x}, \mathbf{c}) \geq d(\mathbf{x}, \mathbf{0}), \forall \mathbf{c} \in \mathcal{C} \\ &\Leftrightarrow d(\mathbf{x}, -\mathbf{c}) \geq d(\mathbf{x}, \mathbf{0}), \\ &\Leftrightarrow \text{wt}(\mathbf{x} + \mathbf{c}) \geq \text{wt}(\mathbf{x}), \forall \mathbf{c} \in \mathcal{C} \\ &\Leftrightarrow \mathbf{x} \in \mathcal{CL}(\mathcal{C}). \end{aligned}$$

$\square$

Teorema 2.34. (Theorem 11.1.2, [4]) *Sea $\mathcal{C}$ un código. Entonces $\rho(\mathcal{C})$ es el máximo entre los pesos mínimos de las clases laterales de $\mathcal{C}$.*

Demostración. Por la proposición anterior $\overline{H}(\mathcal{C}) = \mathcal{CL}(\mathcal{C})$, aplicando la segunda parte del Teorema 2.29, $\rho(\mathcal{C}) = \max_{\mathbf{x} \in \overline{H}(\mathcal{C})} \text{wt}(\mathbf{x}) = \max_{\mathbf{c} \in \mathcal{CL}(\mathcal{C})} \text{wt}(\mathbf{x})$, se sigue el resultado. $\square$

Observación 2.35. *Del Teorema 2.34 se concluye que el radio de cubrimiento de un código corresponde al peso de las clases laterales huérfanas de mayor peso.*

Lema 2.36. (Lema 11.7.13, [4]) *Sea $\mathcal{C}$ $[n, k, d]$ -código lineal binario. Se cumplen las siguientes afirmaciones:*

- (i) $\lfloor \frac{d-1}{2} \rfloor \leq \nu(\mathcal{C}) \leq \rho(\mathcal{C})$,
- (ii) Si $\mathcal{C}$ es perfecto, $\lfloor \frac{d-1}{2} \rfloor = \nu(\mathcal{C}) = \rho(\mathcal{C})$,
- (iii) Si $\mathcal{C}$ es par y $d > 2$, entonces $\nu(\mathcal{C}) < \rho(\mathcal{C})$.

Demostración.

- (i) Sean $t = \lfloor (d-1)/2 \rfloor$ y $\mathcal{C}'$ una clase lateral de $\mathcal{C}$ tal que, $\text{wt}(\mathcal{C}') = t$. Ahora, suponga que $\mathcal{C}'$ tiene dos líderes $\mathbf{x}$ y $\mathbf{y}$, por definición, esto significa que, $\text{wt}(\mathbf{x}) = \text{wt}(\mathbf{y}) = t$. Sea $\mathbf{c} \in \mathcal{C}$ tal que $\mathbf{y} = \mathbf{x} + \mathbf{c}$, se tiene que

$$\text{wt}(\mathbf{c}) = d(\mathbf{c}, \mathbf{0}) = d(\mathbf{y} - \mathbf{x}, \mathbf{0}) = d(\mathbf{x}, \mathbf{y}).$$

Luego,

$$\begin{aligned} d(\mathbf{x}, \mathbf{y}) &\leq d(\mathbf{x}, \mathbf{0}) + d(\mathbf{0}, \mathbf{y}) \\ &= \text{wt}(\mathbf{x}) + \text{wt}(\mathbf{y}) \\ &= \left\lfloor \frac{d-1}{2} \right\rfloor + \left\lfloor \frac{d-1}{2} \right\rfloor \\ &= 2 \left\lfloor \frac{d-1}{2} \right\rfloor \\ &\leq 2 \left(\frac{d-1}{2} \right) \\ &= d-1 \end{aligned}$$

Por tanto $\text{wt}(\mathbf{c}) \leq d-1$ lo que es una contradicción. Así, una clase con peso t tiene un único líder y $t \leq \nu(\mathcal{C})$.

Ahora se prueba que $\nu(\mathcal{C}) \leq \rho(\mathcal{C})$. Dado que $\overline{E}(\mathcal{C}) \subseteq \overline{H}(\mathcal{C})$, por ítem (ii) del Teorema 2.29 y la definición de $\rho(\mathcal{C})$ se tiene $\nu(\mathcal{C}) \leq \rho(\mathcal{C})$.

- (ii) Si $\mathcal{C}$ es perfecto, entonces $\rho(\mathcal{C}) = \lfloor (d-1)/2 \rfloor$ y de (i) se sigue el resultado.
- (iii) Sean $\mathcal{C}$ un código par y $d > 2$. Por el Teorema 2.18 parte (iii), se tiene que los huérfanos tienen más de un líder. En particular, la clase lateral de peso $\rho(\mathcal{C})$ no tiene un único líder. Por tanto $\nu(\mathcal{C}) < \rho(\mathcal{C})$.

□

Teorema 2.37. *Si $\mathcal{C}$ es un $[n, k]$ -código lineal binario, entonces $0 \leq \rho(\mathcal{C}) - \nu(\mathcal{C}) \leq k$.*

Demostración. La primera parte de la desigualdad, se obtiene de (i) del lema anterior. Sea $\mathbf{x}_1$ el líder de una clase lateral $\mathcal{C}_1$ de $\mathcal{C}$ con peso $\rho = \rho(\mathcal{C})$. Reorganizando las coordenadas, se puede

asumir que $\text{supp}(\mathbf{x}_1) = \{1, \dots, \rho\}$. Si $\nu(\mathcal{C}) = \rho$ la prueba termina. Ahora, si $\nu(\mathcal{C}) \leq \rho - 1$, entonces existe otro líder $\mathbf{x}_2$ de $\mathcal{C}_1$, por tanto $\mathbf{x}_1 + \mathbf{x}_2 = \mathbf{c}_1 \in \mathcal{C}$. Reorganizando las coordenadas se puede asumir que $1 \in \text{supp}(\mathbf{c}_1)$, ya que $\mathbf{x}_1$ y $\mathbf{x}_2$ deben tener alguna coordenada diferente. Por el Corolario 2.16, existe un hijo $\mathcal{C}_2$ de $\mathcal{C}_1$ tal que todos sus líderes tienen su primera coordenada igual a 0, uno de los cuales es $\mathbf{x}_3 = \mathbf{e}_1 + \mathbf{x}_1$. Si $\nu(\mathcal{C}) \leq \rho - 2$, hay un líder $\mathbf{x}_4$ de $\mathcal{C}_2$ que debe tener una coordenada diferente de $\mathbf{x}_3$ en $\text{supp}(\mathbf{x}_3) = \{2, 3, \dots, \rho\}$. Reorganizando coordenadas se puede asumir que $\mathbf{c}_2 = \mathbf{x}_3 + \mathbf{x}_4 \in \mathcal{C}$ tiene un 1 en la segunda coordenada. Dado que todos los líderes de $\mathcal{C}_2$ tienen 0 en la primera coordenada, entonces $\mathbf{c}_2$ también cumple esta propiedad. Como $\mathbf{c}_1$ empieza por $1\dots$ y $\mathbf{c}_2$ empieza por $01\dots$, entonces son linealmente independientes. Por el Corolario 2.16, existe un hijo $\mathcal{C}_3$ de $\mathcal{C}_2$ cuyos líderes tienen 0 en la segunda coordenada, uno de los cuales es $\mathbf{x}_5 = \mathbf{e}_2 + \mathbf{x}_3$. Si $\mathcal{C}_3$ tiene un líder que tiene 1 en la primera coordenada, entonces $\mathcal{C}_2$ debe tener un líder con un 1 en la primera coordenada por el Corolario 2.14, lo cual es una contradicción, por lo que los líderes de $\mathcal{C}_3$ tienen a 0 en la primera y segunda coordenada. Si $\nu(\mathcal{C}) \leq \rho - 3$, hay un líder $\mathbf{x}_6$ de $\mathcal{C}_3$ la cual difiere de $\mathbf{x}_5$ en alguna coordenada en $\text{supp}(\mathbf{x}_5) = \{3, 4, \dots, \rho\}$, reorganizando coordenadas se puede asumir que $\mathbf{c}_3 = \mathbf{x}_5 + \mathbf{x}_6 \in \mathcal{C}$ tiene 1 en la tercera coordenada. Dado que $\mathbf{c}_1$ empieza $1\dots$, $\mathbf{c}_2$ empieza $01\dots$ y $\mathbf{c}_3$ empieza $001\dots$, ellos son linealmente independientes. Se puede continuar este proceso inductivamente hasta encontrar s palabras del código linealmente independientes siempre que $\nu(\mathcal{C}) < \rho - s$. Dado que el valor más grande que puede tomar s es k , se tiene que $\nu(\mathcal{C}) \geq \rho - k$. $\square$

2.3. Cálculo de líderes

Para esta sección se utilizarán definiciones y resultados enunciados anteriormente en los preliminares y en la Sección 2.2. El objetivo de esta sección es presentar la teoría matemática necesaria para sustentar el funcionamiento del algoritmo `Comp_CL` (ver Algoritmo 6).

Definición 2.38. La región de Voronoi de una palabra $\mathbf{c} \in \mathcal{C}$, denotada por $D(\mathbf{c})$, se define de la siguiente manera:

$$D(\mathbf{c}) = \{\mathbf{y} \in \mathbb{F}_2^n : d(\mathbf{y}, \mathbf{c}) \leq d(\mathbf{y}, \mathbf{c}'), \forall \mathbf{c}' \in \mathcal{C} \setminus \{\mathbf{c}\}\}.$$

El conjunto de todas las regiones de Voronoi de un código binario, cubre todo el espacio $\mathbb{F}_2^n$.

Además, $D(\mathbf{0}) = \mathcal{CL}(\mathcal{C})$ (ver Proposición 2.33).

A partir de las definiciones de órdenes de términos dadas en la Sección 1.3 se puede definir un orden para los elementos en $\mathbb{F}_2^n$, como se muestra a continuación. La función característica $\blacktriangle : \mathbb{F}_2^s \rightarrow \mathbb{Z}^s$, denota la aplicación que reemplaza los elementos $0, 1 \in \mathbb{F}_2$ por sus símbolos respectivos en el conjunto de los enteros.

Definición 2.39. Un orden en $\mathbb{N}^n$ es admisible si cumple las siguientes propiedades:

1. Para algún vector $\mathbf{u} \in \mathbb{N}^n \setminus \{0\}$, $\mathbf{0} \preceq_1 \mathbf{u}$.
2. Para vectores $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{N}^n$, si $\mathbf{u} \preceq_1 \mathbf{v}$, entonces $\mathbf{u} + \mathbf{w} \preceq_1 \mathbf{v} + \mathbf{w}$.

Un orden $\prec$ en $\mathbb{F}_2^n$ es una relación compatible con el peso de Hamming si para $\mathbf{a}, \mathbf{b} \in \mathbb{F}_2^n$ se dice que $\mathbf{a} \prec \mathbf{b}$ si $\text{wt}(\mathbf{a}) < \text{wt}(\mathbf{b})$ o si $\text{wt}(\mathbf{a}) = \text{wt}(\mathbf{b})$ y $\blacktriangle \mathbf{a} \prec_1 \blacktriangle \mathbf{b}$, donde $\prec_1$ es un orden admisible en $\mathbb{N}^n$.

Observación 2.40.

- En general un orden en $\mathbb{F}_2^n$ no satisface la definición de un orden admisible, esto porque no siempre se cumple la segunda condición de la definición de orden admisible.
- Para cada vector $\mathbf{a}, \mathbf{b} \in \mathbb{F}_2^n$, si $\text{supp}(\mathbf{a}) \subset \text{supp}(\mathbf{b})$, entonces $\mathbf{a} \prec \mathbf{b}$.

Además, para cada vector $\mathbf{a} = (a_1, \dots, a_n) \in \mathbb{F}_2^n$ se tiene que $X^{\mathbf{a}} = x_1^{a_1} \dots x_n^{a_n}$. Así $\deg(X^{\mathbf{a}}) = \text{wt}(\mathbf{a})$. Esto implica que un orden en $\mathbb{F}_2^n$ puede ser visto como un orden de términos en $\mathbb{K}[x_1, \dots, x_n]$. En este trabajo se usa esta interpretación para utilizar herramientas que permiten definir órdenes de términos disponibles en SageMath; con ayuda de estas herramientas se construyó la función `ordering` (ver Algoritmo 9) para implementar ordenes en $\mathbb{F}_2^n$. En la Sección 1.3 se presentó la definición de orden de términos y se introdujeron los ordenes de términos: lexicográfico (`lex`), lexicográfico inverso (`invlex`), lexicográfico gradual (`deglex`) y lexicográfico inverso gradual (`degrevlex`).

Definición 2.41. El objeto `List` se define como un conjunto ordenado de elementos en $\mathbb{F}_2^n$ con respecto a un orden de peso compatible $\prec$ verificando las siguientes propiedades:

1. $\mathbf{0} \in \text{List}$

2. Si $\mathbf{v} \in \text{List}$ y $\text{wt}(\mathbf{v}) = \text{wt}(N(\mathbf{v}))$, entonces $\{\mathbf{v} + \mathbf{e}_i : i \notin \text{supp}(\mathbf{v})\} \subset \text{List}$, donde $N(\mathbf{v}) = \min_{\prec} \{\mathbf{w} : \mathbf{w} \in \text{List} \cap (\mathbf{v} + \mathcal{C})\}$.

El siguiente teorema muestra que el conjunto List incluye el conjunto de líderes de un código lineal binario.

Teorema 2.42. (Theorem 3.1, [1]) Sea $\mathbf{w} \in \mathbb{F}_2^n$. Si $\mathbf{w} \in \mathcal{CL}(\mathcal{C})$, entonces $\mathbf{w} \in \text{List}$.

Demostración. Por definición de List , se tiene que la afirmación se cumple para $\mathbf{0} \in \mathbb{F}_2^n$. Ahora se asume que la propiedad se cumple con respecto a un orden de peso compatible $\prec$ para cualquier palabra $\mathbf{u} \in \mathcal{CL}(\mathcal{C})$ menor que una palabra arbitraria pero fija $\mathbf{w} \in \mathcal{CL}(\mathcal{C}) \setminus \{\mathbf{0}\}$, esto es:

$$\text{Si } \mathbf{u} \in \mathcal{CL}(\mathcal{C}) \text{ y } \mathbf{u} \prec \mathbf{w}, \text{ entonces } \mathbf{u} \in \text{List}. \quad (2.8)$$

Ahora se demuestra que $\mathbf{w} \in \text{List}$. Primero, note que $\mathbf{w}$ puede escribirse así $\mathbf{w} = \mathbf{v} + \mathbf{e}_i$ con $i \in \text{supp}(\mathbf{w})$ e $i \notin \text{supp}(\mathbf{v})$, o equivalentemente $\text{supp}(\mathbf{v}) \subset \text{supp}(\mathbf{w})$, esto es $\mathbf{v} \prec \mathbf{w}$. Además, dado que $\mathbf{w} \in \mathcal{CL}(\mathcal{C})$ entonces por el Corolario 2.16, $\mathbf{v}$ también está en $\mathcal{CL}(\mathcal{C})$. Así por (2.8) se tiene que $\mathbf{v} \in \text{List}$ y entonces $\text{wt}(\mathbf{v}) = \text{wt}(N(\mathbf{v}))$. Por el ítem 2 de la Definición 2.41, se tiene el resultado, es decir $\mathbf{w} = \mathbf{v} + \mathbf{e}_i \in \text{List}$. $\square$

Observación 2.43. En el Algoritmo `Comp_CL` (ver p.p. 85) primero se utiliza la función $\mathbf{t} = \text{NextTerm}(\text{Listing})$, donde el elemento $\mathbf{t}$ se borra del conjunto Listing . Entonces en la función $\text{InsertNext}(\mathbf{t}, \text{Listing})$ inserta en Listing todos los elementos de la forma: $\mathbf{t}' = \mathbf{t} + \mathbf{e}_k$, con $k \notin \text{supp}(\mathbf{t})$; es decir $\mathbf{t} \prec \mathbf{t}'$. De esta forma, todos los elementos nuevos que se insertan en Listing son mayores, con respecto a $\prec$, que aquellos que ya habían sido borrados.

En el Teorema 3.2 de [4] se demuestra que el Algoritmo `Comp_CL` calcula el conjunto de líderes de un código binario. En la implementación realizada en este trabajo a diferencia de la propuesta del artículo, solamente se calculan los líderes de un código binario. A continuación se exponen algunos ejemplos de su uso.

Ejemplo 2.44. Sea $\mathcal{C}$ un $[5, 2, 3]$ -código binario con matriz generadora

$$M_1 = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix}.$$

Los cálculos realizados en SageMath primero con el orden lexicográfico.

```

M1=matrix(GF(2),[[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
Comp_CL(C1,'lex')
{1: [(0, 0, 0, 0, 0)],
 2: [(0, 0, 0, 0, 1)],
 3: [(0, 0, 0, 1, 0)],
 4: [(0, 0, 1, 0, 0)],
 5: [(0, 1, 0, 0, 0)],
 6: [(1, 0, 0, 0, 0)],
 7: [(0, 0, 1, 0, 1), (1, 1, 0, 0, 0)],
 8: [(0, 1, 1, 0, 0), (1, 0, 0, 0, 1)]}

```

El mismo ejemplo con el orden lexicográfico inverso.

```

M11=matrix(GF(2),[[1,0,1,1,0],[1,1,1,0,1]])
C11=LinearCode(M11)
D11=Comp_CL(C11,'invlex')
D11
{1: [(0, 0, 0, 0, 0)],
 2: [(1, 0, 0, 0, 0)],
 3: [(0, 1, 0, 0, 0)],
 4: [(0, 0, 1, 0, 0)],
 5: [(0, 0, 0, 1, 0)],
 6: [(0, 0, 0, 0, 1)],
 7: [(1, 1, 0, 0, 0), (0, 0, 1, 0, 1)],
 8: [(0, 1, 1, 0, 0), (1, 0, 0, 0, 1)]}

```

El ejemplo implementado en `Comp_CL` con `lex` muestra un diccionario el cual contiene 8 líderes del código, un líder de peso 0, 5 líderes únicos de peso 1. En la llave 7 y 8 se puede notar que los líderes de peso 2 no son únicos y se muestran todos los líderes para la clase lateral. Sin embargo, al cambiar el orden `lex` por `invlex` solo se ve un cambio en el orden de los líderes de la llave 7, pero no hay cambios en los líderes.

Observación 2.45. *El orden que se usará para los siguientes ejemplos es `lex` que corresponde al orden lexicográfico, internamente en el algoritmo afectan los órdenes para la construcción del diccionario y algunas veces el orden en que se presentarán estos líderes, sin embargo los líderes son los mismos.*

Ejemplo 2.46. El Ejemplo 2.9 implementado en SageMath mediante el algoritmo `Comp_CL`.

```
M2=matrix(GF(2),[[1,1,1,1,0],[0,0,1,1,1]])
C2=LinearCode(M2)
Comp_CL(C2,'lex')
{1: [(0, 0, 0, 0, 0)],
 2: [(0, 0, 0, 0, 1)],
 3: [(0, 0, 0, 1, 0)],
 4: [(0, 0, 1, 0, 0)],
 5: [(0, 1, 0, 0, 0)],
 6: [(1, 0, 0, 0, 0)],
 7: [(0, 1, 0, 1, 0), (1, 0, 1, 0, 0)],
 8: [(0, 1, 1, 0, 0), (1, 0, 0, 1, 0)]}
```

Para este código hay 8 llaves, es decir 8 líderes; de los cuales los líderes 7 y 8 no son únicos.

Ejemplo 2.47. Sea $\mathcal{C}$ el $[6, 3, 3]$ -código binario con matriz generadora

$$M_3 = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

Los cálculos realizados en SageMath para este código fueron los siguientes.

```
M3=matrix(GF(2),[[1,0,0,0,1,1],[0,1,0,1,0,1],[0,0,1,1,1,0]])
C3=LinearCode(M3)
Comp_CL(C3,'lex')
{1: [(0, 0, 0, 0, 0, 0)],
 2: [(0, 0, 0, 0, 0, 1)],
 3: [(0, 0, 0, 0, 1, 0)],
```

```

4: [(0, 0, 0, 1, 0, 0)],
5: [(0, 0, 1, 0, 0, 0)],
6: [(0, 1, 0, 0, 0, 0)],
7: [(1, 0, 0, 0, 0, 0)],
8: [(0, 0, 1, 0, 0, 1), (0, 1, 0, 0, 1, 0), (1, 0, 0, 1, 0, 0)]

```

La implementación del ejemplo en `Comp_CL` muestra algo particular y es una clase lateral con un líder de peso 2, pero no es único, esta clase lateral tiene 3 líderes. Además tiene 6 líderes de peso 1 y un líder de peso 0. Entonces el radio de Newton de este código es 1.

Ejemplo 2.48. Sea $\mathcal{C}$ el $[6,2, 4]$ -código binario con matriz generadora

$$M_4 = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

```

M4=matrix(GF(2),[[1,1,1,1,0,0],[0,0,1,1,1,1]])
C4=LinearCode(M4)
Comp_CL(C4,'lex')
{1: [(0, 0, 0, 0, 0, 0)],
2: [(0, 0, 0, 0, 0, 1)],
3: [(0, 0, 0, 0, 1, 0)],
4: [(0, 0, 0, 1, 0, 0)],
5: [(0, 0, 1, 0, 0, 0)],
6: [(0, 1, 0, 0, 0, 0)],
7: [(1, 0, 0, 0, 0, 0)],
8: [(0, 0, 0, 0, 1, 1), (0, 0, 1, 1, 0, 0), (1, 1, 0, 0, 0, 0)],
9: [(0, 0, 0, 1, 0, 1), (0, 0, 1, 0, 1, 0)],
10: [(0, 0, 0, 1, 1, 0), (0, 0, 1, 0, 0, 1)],
11: [(0, 1, 0, 0, 0, 1), (1, 0, 0, 0, 1, 0)],
12: [(0, 1, 0, 0, 1, 0), (1, 0, 0, 0, 0, 1)],
13: [(0, 1, 0, 1, 0, 0), (1, 0, 1, 0, 0, 0)],
14: [(0, 1, 1, 0, 0, 0), (1, 0, 0, 1, 0, 0)],
15: [(0, 1, 0, 1, 0, 1), (0, 1, 1, 0, 1, 0),

```

```
(1, 0, 0, 1, 1, 0), (1, 0, 1, 0, 0, 1)],  
16: [(0, 1, 0, 1, 1, 0), (0, 1, 1, 0, 0, 1),  
(1, 0, 0, 1, 0, 1), (1, 0, 1, 0, 1, 0)]}
```

Implementando el ejemplo en `Comp_CL` hay 16 llaves, es decir 16 líderes. Los líderes tienen las siguientes características. En las llaves 15 y 16 hay 2 líderes de peso 3 para los cuales las clases laterales tienen 3 líderes más; de la llave 9 a la 14 hay 6 líderes de peso 2, con clases laterales que tienen un líder más; en la llave 8 hay un líder de peso 2 y su clase lateral tiene dos líderes más; de la llave 2 a la 7 hay líderes únicos de peso 1 y en la llave 1 hay un líder de peso 0. Estos cálculos implican que el radio de cubrimiento del código es 3 y el radio de Newton es 1.

Capítulo 3

Decodificación por distancia mínima

En este capítulo se mostrará cómo usar el cálculo de líderes para desarrollar procedimientos para la decodificación por distancia mínima (ver Definición 1.14).

3.1. Clases laterales y síndromes

La decodificación es un método que convierte las n -uplas recibidas en la transmisión, en un mensaje que contenga palabras del código. El objetivo de este proceso es que los mensajes transmitidos lleguen sin errores al receptor.

Se dice que una decodificación es eficiente cuando este proceso devuelve los mensajes sin ningún error y que es de buen rendimiento sobre todo para códigos de distancia mínima grande. Se enunciarán algunos resultados que ayudarán más adelante en el proceso de decodificación.

Teorema 3.1. *Sea $\mathcal{C}$ un $[n, k, d]$ -código lineal binario en el espacio vectorial $\mathbb{F}_2^n$. Entonces se satisfacen las siguientes afirmaciones:*

- (i) *Todo vector de $\mathbb{F}_2^n$ está contenido en alguna clase lateral de $\mathcal{C}$,*
- (ii) *para todo $\mathbf{u} \in \mathbb{F}_2^n$, $|\mathbf{u} + \mathcal{C}| = |\mathcal{C}| = 2^k$,*
- (iii) *para todo $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$, $\mathbf{u} \in \mathbf{v} + \mathcal{C}$ implica que $\mathbf{u} + \mathcal{C} = \mathbf{v} + \mathcal{C}$,*
- (iv) *dos clases laterales son iguales o su intersección es vacía,*
- (v) *existen 2^{n-k} clases laterales diferentes.*

Demostración.

- (i) Sea $\mathbf{v} \in \mathbb{F}_2^n$, como $\mathbf{0} \in \mathcal{C}$ se tiene que $\mathbf{v} \in \mathbf{v} + \mathcal{C}$.
- (ii) Por definición $\mathbf{u} + \mathcal{C}$ tiene a lo más $|\mathcal{C}| = 2^k$ elementos. Claramente, dos elementos $\mathbf{u} + \mathbf{c}$ y $\mathbf{u} + \mathbf{c}'$ de $\mathbf{u} + \mathcal{C}$ son iguales si y solo si $\mathbf{c} = \mathbf{c}'$, por tanto $|\mathbf{u} + \mathcal{C}| = |\mathcal{C}| = 2^k$.
- (iii) Por hipótesis $\mathbf{u} \in \mathbf{v} + \mathcal{C}$, entonces $\mathbf{u} + \mathcal{C} \subseteq \mathbf{v} + \mathcal{C}$, pero por (ii) $|\mathbf{u} + \mathcal{C}| = |\mathbf{v} + \mathcal{C}|$, así $\mathbf{u} + \mathcal{C} = \mathbf{v} + \mathcal{C}$.
- (iv) Considere las clases laterales $\mathbf{u} + \mathcal{C}$ y $\mathbf{v} + \mathcal{C}$ y suponga que $\mathbf{x} \in (\mathbf{u} + \mathcal{C}) \cap (\mathbf{v} + \mathcal{C})$. Dado que $\mathbf{x} \in \mathbf{u} + \mathcal{C}$ por (iii) $\mathbf{x} + \mathcal{C} = \mathbf{u} + \mathcal{C}$, análogamente $\mathbf{x} + \mathcal{C} = \mathbf{v} + \mathcal{C}$. Por tanto, $\mathbf{u} + \mathcal{C} = \mathbf{v} + \mathcal{C}$.
- (v) Por (i) hay 2^n elementos en la unión de todas las clases laterales y por (iv) generan una partición en $\mathbb{F}_2^n$. Ahora por (ii) en cada clase lateral hay 2^k elementos, así $\frac{2^n}{2^k} = 2^{n-k}$ clases laterales.

□

Sea $\mathbf{v}$ el mensaje transmitido y $\mathbf{w}$ el mensaje recibido, donde se da un error $\mathbf{e}$ tal que,

$$\mathbf{e} = \mathbf{w} + \mathbf{v} \in \mathbf{w} + \mathcal{C},$$

así por el Teorema 1.31 el error $\mathbf{e}$ y la palabra recibida $\mathbf{w}$ están en la misma clase lateral. Un método para decodificar $\mathbf{w}$ (ver Definición 1.14) consiste en elegir el líder de la clase lateral $\mathbf{w} + \mathcal{C}$ y restarlo a $\mathbf{w}$. Se puede concluir que la palabra transmitida es $\mathbf{v} = \mathbf{w} + \mathbf{e}$, si $\mathbf{e}$ es el único líder de $\mathbf{w} + \mathcal{C}$.

Además, si $\text{wt}(\mathbf{e}) \leq \lfloor \frac{d-1}{2} \rfloor$, entonces $\mathbf{e}$ es el único líder de su clase.

Observación 3.2. *Cuando hay dos o más líderes en la clase lateral del mensaje recibido, el proceso de decodificación tiene dos o más opciones, entonces se elige aleatoriamente alguno de los líderes o se solicita reenviar la información.*

Ejemplo 3.3. Sea $\mathcal{C} = \{0000, 1001, 1100, 0101\}$ un código lineal binario; decodifique las siguientes palabras recibidas: $\mathbf{w}_1 = 1101$, $\mathbf{w}_2 = 0111$, $\mathbf{w}_3 = 0110$.

- La clase lateral de $\mathbf{w}_1$ es

$$\mathbf{w}_1 + \mathcal{C} = \{1101, 0100, 0001, 1000\}.$$

En esta clase lateral hay 3 líderes. Si el líder que se escoge es 0100, este se decodifica $\mathbf{w}_1 + \mathbf{e} = 1101 + 0100 = 1001$, pero si se escoge 0001, entonces $\mathbf{w}_1 + \mathbf{e} = 1101 + 0001 = 1100$, por último si se escoge 1000, entonces $\mathbf{w}_1 + \mathbf{e} = 1101 + 1000 = 0101$. En este caso no se puede decir cual fue realmente la palabra enviada si 1001, 1100 o 0101.

- La clase lateral de $\mathbf{w}_2$ es el conjunto,

$$\mathbf{w}_2 + \mathcal{C} = \{0111, 1110, 1011, 0010\}.$$

En esta clase lateral el líder es único, entonces se tiene la certeza de que la palabra enviada fue $\mathbf{x}_2 + \mathbf{e} = 0111 + 0010 = 0101$.

- La clase lateral de $\mathbf{w}_3$ es,

$$\mathbf{w}_3 + \mathcal{C} = \{0110, 1111, 1010, 0011\}.$$

Esta clase lateral tiene 3 líderes 0110, 1010, 0011, escogiendo el líder 0110 la palabra enviada es $\mathbf{w}_3 + \mathbf{e} = 0110 + 0110 = \mathbf{0}$; si el líder es 1010 la palabra enviada es $\mathbf{w}_3 + \mathbf{e} = 0110 + 1010 = 1100$ y si el líder es 0011, entonces $\mathbf{w}_3 + \mathbf{e} = 0110 + 0011 = 0101$. Hay tres posibilidades para la palabra enviada.

Para hacer la decodificación se utiliza el concepto de síndrome (ver Definición 1.26) con esto se podrá encontrar la clase lateral a la cual pertenece el vector transmitido.

Teorema 3.4. *Sea $\mathcal{C}$ un código lineal binario y H su matriz de control de paridad. Suponga que $\mathbf{x}$ es el mensaje recibido y $\mathbf{x} = \mathbf{c} + \mathbf{e}$, donde $\mathbf{c}$ es la palabra transmitida y $\mathbf{e}$ el error transmitido. Entonces $S(\mathbf{x}) = S(\mathbf{e})$.*

Demostración. Sea $\mathbf{x} = \mathbf{c} + \mathbf{e}$,

$$\begin{aligned}
 S(\mathbf{x}) &= \mathbf{x}H^T \\
 &= (\mathbf{c} + \mathbf{e})H^T \\
 &= \mathbf{c}H^T + \mathbf{e}H^T \\
 &= \mathbf{e}H^T \\
 &= S(\mathbf{e}).
 \end{aligned}$$

□

Observación 3.5. Si el mensaje transmitido es $\mathbf{x}$ y $S(\mathbf{x}) = \mathbf{0}$, entonces se asume que no ha ocurrido ningún error en la transmisión.

Los teoremas 1.31 y 3.1 proporcionan una relación biyectiva entre los líderes de las clases laterales de un código y sus respectivos síndromes; es decir a líderes de clases laterales diferentes les corresponden síndromes diferentes.

Definición 3.6. Una tabla que relaciona cada líder de una clase lateral con su síndrome se llama tabla de búsqueda de síndromes o matriz de decodificación (SDA).

Ejemplo 3.7. A continuación se presenta la tabla de búsqueda de síndromes para el Ejemplo 3.3.

$S(\mathbf{x})$	Líder
00	0000
10	0001
01	0010
11	0011

Tabla 3.1: Tabla de búsqueda de síndromes para el Ejemplo 3.3.

La anterior tabla es útil para saber en cuál clase lateral está la palabra que se ha recibido y el error que se le debe sumar. No siempre esta decodificación es correcta, ya que si el líder no es único, no se tendrá certeza de cual fue el mensaje que se quería transmitir.

3.2. Ejemplos de decodificación

En los siguientes ejemplos se utiliza el Algoritmo `decodificacion` (ver Algoritmo 11) para realizar la decodificación de las palabras recibidas. Para utilizar este algoritmo primero se calculan los líderes de clase usando uno de los siguientes algoritmos: `arreglo_estandar`, `diagrama_Hasse_codigo` y `Comp_C1`. Además, se hará una decodificación donde si el líder no es único, entonces el mensaje que se quería transmitir y el recibido no son necesariamente los mismos.

Ejemplo 3.8. Considere el $[5, 2, 3]$ -código binario $\mathcal{C}$ con matriz generadora dada por

$$M_2 = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{pmatrix}.$$

Para decodificar las palabras $\mathbf{w}_1 = 00011$, $\mathbf{w}_2 = 11100$ y $\mathbf{w}_3 = 10101$, se calculan los líderes de $\mathcal{C}$ usando el Algoritmo 1. La decodificación de cada palabra se muestra a continuación.

```
M1=matrix(GF(2), [[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
ae1=arreglo_estandar(C1)
decodificacion(vector(GF(2), [0,0,0,1,1]), C1, ae1[1])
    (0, 1, 0, 1, 1)
decodificacion(vector(GF(2), [1,1,1,0,0]), C1, ae1[1])
    (1, 1, 1, 0, 1)
decodificacion(vector(GF(2), [1,1,0,1,0]), C1, ae1[1])
    (1, 0, 1, 1, 0)
```

Ahora si se calculan los líderes del código usando el Algoritmo 2, la decodificación para cada palabra es la siguiente.

```
M1=matrix(GF(2), [[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
r=diagrama_Hasse_codigo(C1)
L1=[vector(GF(2), eval(x)) for x in list(r.keys())]
decodificacion(vector(GF(2), [0,0,0,1,1]), C1, L1)
```

```

(0, 1, 0, 1, 1)
decodificacion(vector(GF(2),[1,1,1,0,0]),C1,L1)
(1, 1, 1, 0, 1)
decodificacion(vector(GF(2),[1,1,0,1,0]),C1,L1)
(0, 1, 0, 1, 1)

```

Finalmente, usando el Algoritmo 6, la decodificación para cada palabra se muestra a continuación.

```

M1=matrix(GF(2),[[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
D1=Comp_CL(C1,'lex')
decodificacion(vector(GF(2),[0,0,0,1,1]),C1,D1)
(0, 1, 0, 1, 1)
decodificacion(vector(GF(2),[1,1,1,0,0]),C1,L1)
(1, 1, 1, 0, 1)
decodificacion(vector(GF(2),[1,1,0,1,0]),C1,L1)
(1, 0, 1, 1, 0)

```

En este ejemplo en los 3 algoritmos las dos primeras palabras fueron decodificadas de la misma manera, esto debido a que pertenecen a una clase lateral con líder único. Pero para el vector 11010 existen dos decodificaciones distintas, no se sabe si el mensaje que se deseaba enviar es 01011 o 10110, esto se debe a que a la clase lateral de 11010 tiene dos líderes.

Ejemplo 3.9. Sea $\mathcal{C}$ un $[10, 4, 4]$ -código binario cuya matriz generadora es

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

A continuación se presenta la decodificación de las siguientes palabras: $\mathbf{w}_1 = 1101010011$ y $\mathbf{w}_2 = 1111110011$.

Con el Algoritmo 1:

```

H1=matrix(GF(2),[[1,0,0,0,1,0,0,0,0,0],[1,0,1,1,0,1,0,0,0,0],
[1,1,0,1,0,0,1,0,0,0],[1,1,1,0,0,0,0,1,0,0],[1,1,1,1,0,0,0,0,1,0],
[1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
ae5=arreglo_estandar(C5)
decodificacion(vector(GF(2),[1,1,0,1,0,1,0,0,1,1]),C5,ae5[1])
(0, 0, 0, 1, 0, 1, 1, 0, 1, 1)
decodificacion(vector(GF(2),[1,1,1,1,1,1,0,0,1,1]),C5,ae5[1])
(1, 0, 1, 1, 1, 1, 0, 0, 1, 1)

```

Con el Algoritmo 2:

```

H1=matrix(GF(2),[[1,0,0,0,1,0,0,0,0,0],[1,0,1,1,0,1,0,0,0,0],
[1,1,0,1,0,0,1,0,0,0],[1,1,1,0,0,0,0,1,0,0],[1,1,1,1,0,0,0,0,1,0],
[1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
r5=diagrama_Hasse_codigo(C5)
L5=[vector(GF(2),eval(x)) for x in list(r5.keys())]
decodificacion(vector(GF(2),[1,1,0,1,0,1,0,0,1,1]),C5,L5)
(0, 0, 0, 1, 0, 1, 1, 0, 1, 1)
decodificacion(vector(GF(2),[1,1,1,1,1,1,0,0,1,1]),C5,L5)
(1, 0, 1, 1, 1, 1, 0, 0, 1, 1)

```

Con el Algoritmo 6:

```

H1=matrix(GF(2),[[1,0,0,0,1,0,0,0,0,0],[1,0,1,1,0,1,0,0,0,0],
[1,1,0,1,0,0,1,0,0,0],[1,1,1,0,0,0,0,1,0,0],[1,1,1,1,0,0,0,0,1,0],
[1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
D5=Comp_CL(C5,'lex')
L=list(D5.values())
L5=[L[i][0] for i in [0..len(L)-1]]
decodificacion(vector(GF(2),[1,1,0,1,0,1,0,0,1,1]),C5,L5)

```

```
(1, 1, 0, 1, 1, 0, 1, 0, 1, 1)
decodificacion(vector(GF(2), [1, 1, 1, 1, 1, 1, 0, 0, 1, 1]), C5, L5)
(1, 0, 1, 1, 1, 1, 0, 0, 1, 1)
```

En este ejemplo, la palabra 1101010011 se decodifica como: 0001011011 y 1101101011, es decir no se tiene certeza cual fue el mensaje que deseaba ser enviado. La palabra 1111110011 se decodifica de manera única como 1011110011, esto porque el líder de su clase es único.

El sistema de álgebra computacional **SageMath** tiene disponible la función `decode_to_code` para realizar el proceso de decodificación. Esta función retorna resultados similares a los que se obtuvieron con los algoritmos `decodificacion` y `decodificacion1` que se implementaron para el desarrollo de este trabajo. A continuación se muestra la forma de usar la función `decode_to_code` con algunos de los ejemplos previos.

Ejemplo 3.10. Para usar la función `decode_to_code` en el Ejemplo 3.8, se deben ingresar en **SageMath** las siguientes ordenes.

```
M1=matrix(GF(2), [[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
DE1=C1.decoder()
DE1.decode_to_code(vector(GF(2), [0,0,0,1,1]))
(0, 1, 0, 1, 1)
DE1.decode_to_code(vector(GF(2), [1,1,1,0,0]))
(1, 1, 1, 0, 1)
DE1.decode_to_code(vector(GF(2), [1,1,0,1,0]))
(0, 1, 0, 1, 1)
```

Ejemplo 3.11. Usando la función `decode_to_code` con el Ejemplo 3.9, se obtienen los siguientes resultados.

```
H1=matrix(GF(2), [[1,0,0,0,1,0,0,0,0,0],[1,0,1,1,0,1,0,0,0,0],
[1,1,0,1,0,0,1,0,0,0],[1,1,1,0,0,0,0,1,0,0],
[1,1,1,1,0,0,0,0,1,0],[1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
DE5=C5.decoder()
```

```

DE5.decode_to_code(vector(GF(2),[0,0,1,0,0,0,1,1,1,1]))
(0, 1, 0, 0, 0, 0, 1, 1, 1, 1)
DE5.decode_to_code(vector(GF(2),[1,1,1,1,1,1,0,0,1,1]))
(1, 0, 1, 1, 1, 1, 0, 0, 1, 1)

```

En los anteriores ejemplos, no siempre la decodificación obtenida corresponde el mensaje que realmente quería ser transmitido. Por el Corolario 1.17 un código con parámetros $[5, 2, 3]$ corrige 1 error, por esta razón dado que en las palabras 00011 y 11100 solamente hay un error, su decodificación se realiza correctamente. Sin embargo, dado que en 11010 hay más de un error, esto lleva a que no se tenga la certeza de que la decodificación sea correcta.

Para un $[10, 4, 4]$ -código, se corrigen hasta 1 error. Dado que en la palabra 0010001111 hay dos errores por tanto no puede ser corregido, es decir la decodificación no siempre es correcta, en cambio como 1111110011 solo tiene un error, se puede decodificar correctamente.

Con la implementación del Algoritmo 12, y en el cual no se escoge un líder aleatoriamente, si no que se toman en cuenta todos los líderes, se da una lista con los mensajes que posiblemente querían ser transmitidos.

Ejemplo 3.12. Para el Ejemplo 4.2 se tiene:

```

M1=matrix(GF(2),[[1,0,1,1,0],[1,1,1,0,1]])
C1=LinearCode(M1)
D1=Comp_CL(C1,'lex')
decodificacion1(vector(GF(2),[0,0,0,1,1]),C1,D1)
[(0, 1, 0, 1, 1)]
decodificacion1(vector(GF(2),[1,1,1,0,0]),C1,D1)
[(1, 1, 1, 0, 1)]
decodificacion1(vector(GF(2),[1,1,0,1,0]),C1,D1)
[(1, 0, 1, 1, 0), (0, 1, 0, 1, 1)]

```

La decodificación de 11010 tiene dos posibles mensajes, uno de ellos es el que quería ser transmitido. Para los mensajes 00011 y 11100 se tiene certeza de cuáles eran los mensajes que querían ser transmitidos.

Ejemplo 3.13. Para el Ejemplo 3.9 se obtuvo:

```

H1=matrix(GF(2),[[1,0,0,0,1,0,0,0,0,0],[1,0,1,1,0,1,0,0,0,0],
[1,1,0,1,0,0,1,0,0,0],[1,1,1,0,0,0,0,1,0,0],
[1,1,1,1,0,0,0,0,1,0],[1,1,1,1,0,0,0,0,0,1]])
C5=codes.from_parity_check_matrix(H1)
D5=Comp_CL(C5,'lex')
decodificacion1(vector(GF(2),[0,0,1,0,0,0,1,1,1,1]),C5,D5)
[(0, 0, 1, 0, 0, 1, 0, 1, 1, 1), (0, 1, 0, 0, 0, 0, 1, 1, 1, 1)]
decodificacion1(vector(GF(2),[1,1,1,1,1,1,0,0,1,1]),C5,D5)
[(1, 0, 1, 1, 1, 1, 0, 0, 1, 1)]

```

Se evidencia lo que se dijo en la observación del Ejemplo 3.9.

Con los anteriores ejemplos se nota que el Algoritmo 12 ofrece todas las posibilidades para el mensaje que deseaba ser transmitido.

Capítulo 4

Un grafo asociado a un código lineal binario

En este capítulo se usa el digrama de Hasse que se definió en la Sección 2.2 para estudiar el grafo construido, se analizan sus propiedades y sus relaciones con parámetros del código.

4.1. Teoría de grafos: definiciones y ejemplos

Definición 4.1. Un grafo $\Gamma = (V, E)$ consiste de un conjunto no vacío V y de E , un conjunto de pares no ordenados de elementos distintos de V . Los elementos de V se denominan vértices o nodos del grafo Γ y los de E se llaman aristas del grafo. Dos vértices u y v de Γ son adyacentes o vecinos si $e = \{u, v\} \in E$, en este caso se dirá que el nodo u es incidente con la arista e . Las aristas incidentes comunes a un nodo son aristas adyacentes. El grado de un vértice es el número de sus vecinos. Adicionalmente, $|V|$ es el orden de Γ y $|E|$ es el tamaño del grafo Γ .

Ejemplo 4.2. Considere el grafo $\Gamma_1 = (V_1, E_1)$, donde $V_1 = \{v_1, v_2, v_3, v_4, v_5\}$ y $E_1 = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_1, v_4\}, \{v_2, v_5\}, \{v_3, v_4\}, \{v_4, v_5\}\}$. Este es un grafo de orden 5 y tamaño 6. La representación gráfica de Γ_1 se muestra en la Figura 4.2b.

Definición 4.3. Un grafo $\Gamma' = (V', E')$ es un subgrafo de $\Gamma = (V, E)$, si $V' \subseteq V$ y $E' \subseteq E$. En este caso se escribe $\Gamma' \subseteq \Gamma$.

Ejemplo 4.4. Para el grafo del Ejemplo 4.2, un subgrafo es $\Gamma' = (V', E')$, donde $V' =$

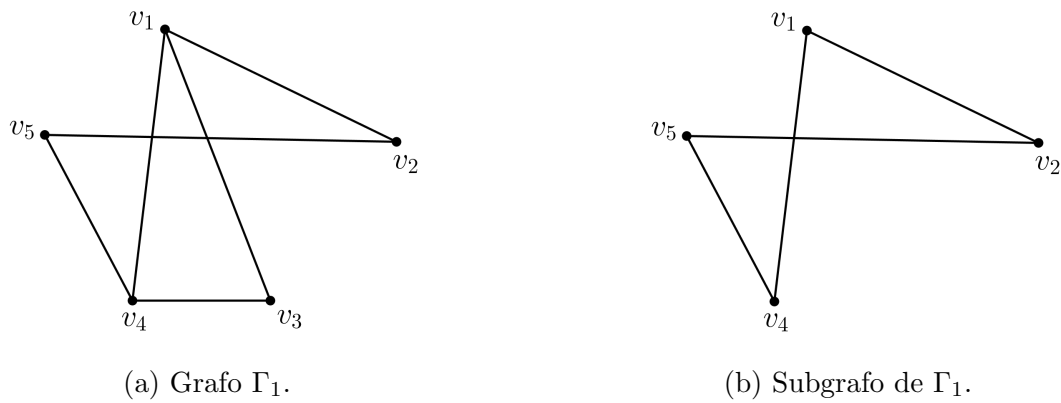


Figura 4.1: Ejemplo de grafo y un subgrafo.

$\{v_1, v_2, v_4, v_5\}$ y $E' = \{\{v_1, v_2\}, \{v_1, v_4\}, \{v_2, v_5\}, \{v_4, v_5\}\}$, este grafo tiene orden y tamaño 4 (ver Figura 4.1b).

Definición 4.5. Sean $\Gamma = (V, E)$ un grafo y $v, w \in V$.

1. Un camino de v a w , o un $v - w$ camino, es una sucesión finita alternada de vértices adyacentes y aristas de Γ . Por tanto, un camino W tiene la forma

$$W = v_0 e_1 v_1 e_2 \cdots v_{n-1} e_n v_n, \quad (4.1)$$

donde las v representan vértices, las e representan aristas, $v_0 = v$, $v_n = w$ y para toda $i = 1, 2, \dots, n$, $e_i = \{v_{i-1}, v_i\} \in E$. Para abreviar la notación también puede representarse el camino (4.1) como $W = v_0 v_1 \cdots v_n$, donde n se define como la longitud del camino W y $\{v_{i-1}, v_i\} \in E$, para todo $1 \leq i \leq n$. El camino trivial de v a v consiste del único vértice v .

2. Un sendero de v a w , o un $v - w$ sendero, es un camino de v a w que no contiene aristas repetidas.
3. Una trayectoria de v a w , o una $v - w$ trayectoria, es un sendero en el que no se repiten vértices.
4. Un camino cerrado es un camino que comienza y termina en el mismo vértice.
5. Un circuito es un camino cerrado que contiene al menos una arista y no contiene aristas repetidas.

6. Un circuito simple o ciclo es un circuito que no tiene cualquier otro vértice repetido excepto el primero y el último. Un ciclo con k vértices se denomina k -ciclo. Un grafo Γ se dice libre de triángulos si no contiene 3-ciclos.

Definición 4.6. Un grafo $\Gamma = (V, E)$ es conexo si para cualquier par de vértices u, v en V existe un camino de u a v , en otro caso se dirá que Γ es no conexo.

Ejemplo 4.7. En la Figura 4.2 se muestra un grafo conexo y uno no conexo, respectivamente.

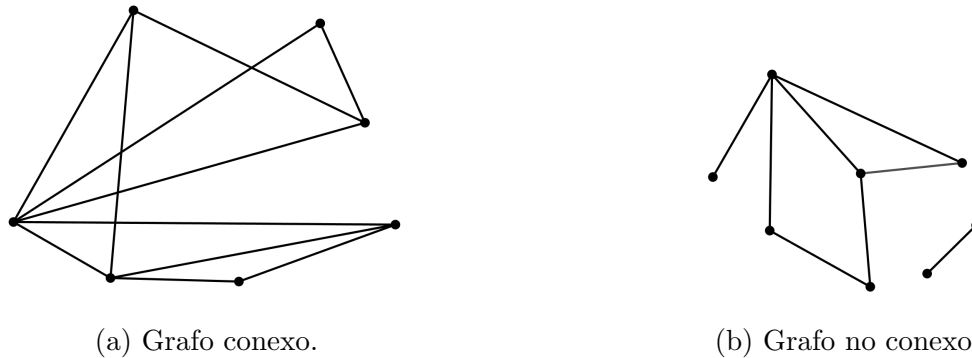


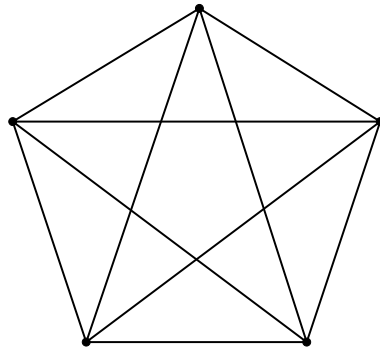
Figura 4.2: Grafo conexo y grafo no conexo.

Definición 4.8. Sea $\Gamma = (V, E)$ un grafo conexo. La distancia geodésica entre dos vértices $u, v \in V$, denotada con $d_\Gamma(u, v)$, es la longitud de la trayectoria más corta de u a v . El diámetro de Γ , denotado con $\text{diam}(\Gamma)$, es la mayor distancia geodésica entre todos los pares de vértices del grafo; es decir $\text{diam}(\Gamma) = \max\{d_\Gamma(u, v) : u, v \in V\}$.

4.2. Algunas familias de grafos

Existen diversas familias de grafos, a continuación se presentan las definiciones de aquellas que serán tenidas en cuenta en lo que resta de este capítulo.

Definición 4.9. El grafo completo de n vértices, que se denota con K_n , es el grafo que contiene exactamente una arista entre cada par de vértices distintos. En la Figura 4.3 se muestra el grafo completo K_5 .

Figura 4.3: Grafo completo K_5 .

Definición 4.10. El grafo camino con n vértices, que se denota con P_n , es el grafo en el que dos de sus vértices tienen grado 1 y los $n - 2$ vértices restantes tienen grado 2. Si sus nodos son $v_1, v_2, \dots, v_n$, entonces sus aristas están dadas por $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}$. En la Figura 4.4a se muestra el grafo camino P_4 .

Definición 4.11. El grafo ciclo C_n , con $n \geq 3$ vértices, es un grafo que consiste de un único ciclo. Este grafo consta de n vértices $v_1, v_2, \dots, v_n$ y sus aristas son $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}$ y $\{v_n, v_1\}$. Este es un grafo en el que todos sus vértices tienen grado 2. En la Figura 4.4a se muestra el grafo ciclo C_5 .

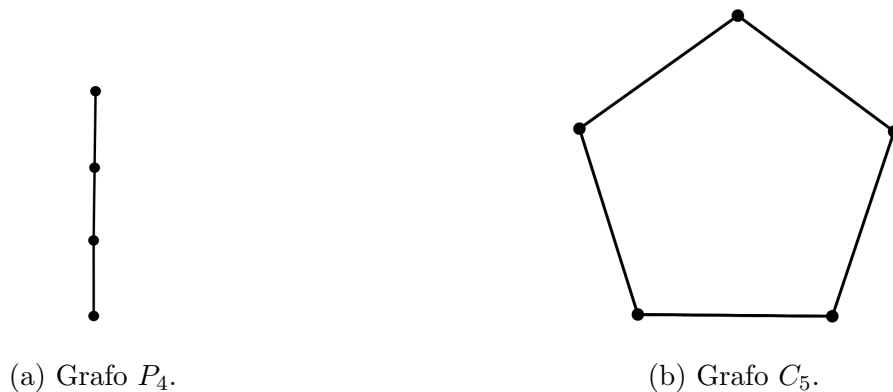
(a) Grafo P_4 .(b) Grafo C_5 .

Figura 4.4: Ejemplo de grafo camino y grafo ciclo.

Definición 4.12. Un grafo $\Gamma = (V, E)$ es bipartito si $V = U \cup W$, $U \cap W = \emptyset$ y $\{u, w\} \in E$ si y solo si $u \in U$ y $w \in W$.

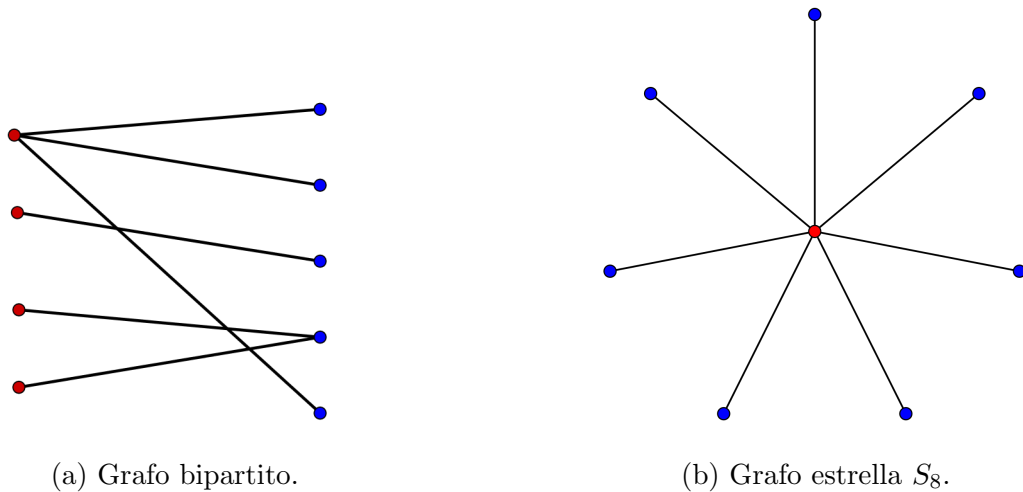


Figura 4.5: Ejemplos grafo bipartito y grafo estrella.

En la Figura 4.5a se muestra un ejemplo de un grafo bipartito con 9 vértices y 6 aristas, donde U es el conjunto de los nodos rojos y W es el conjunto de los nodos azules.

Definición 4.13. Un árbol es un grafo en el que cualquier par de vértices están conectados por exactamente un camino. En particular, un grafo estrella de orden n , que se denota con S_n , es un árbol de n vértices con un nodo de grado $n - 1$ y los $n - 1$ vértices restantes de grado 1.

La Figura 4.5b corresponde al grafo estrella S_8 , donde el vértice rojo tiene grado 7 y los vértices azules tienen grado 1.

4.3. Resultados

Vistos estos conceptos básicos en teoría de grafos se pueden establecer relaciones entre la teoría de códigos y teoría de grafos, en este documento se ilustrará una de ellas.

A partir del orden parcial que se presentó en la Definición 2.8 se establece una construcción de un grafo asociado con un código lineal binario.

Definición 4.14. Sea $\mathcal{C}$ un código lineal binario. Con $\Gamma(\mathcal{C}) = (V_{\mathcal{C}}, E_{\mathcal{C}})$ se denota el grafo asociado con el código $\mathcal{C}$, donde $V_{\mathcal{C}}$ es el conjunto de las clases laterales de $\mathcal{C}$ y $\{\mathcal{C}_1, \mathcal{C}_2\} \in E_{\mathcal{C}}$ si $\mathcal{C}_1 \prec \mathcal{C}_2$ y $\text{wt}(\mathcal{C}_1) = \text{wt}(\mathcal{C}_2) - 1$; es decir $\{\mathcal{C}_1, \mathcal{C}_2\}$ es una arista de $\Gamma(\mathcal{C})$ si $\mathcal{C}_1$ es hijo de $\mathcal{C}_2$.

El siguiente resultado se obtiene directamente de la Definición 4.14 y del item (v) del Teorema 3.1.

Corolario 4.15. Sea $\mathcal{C}$ un $[n, k, d]$ -código lineal binario y $\Gamma(\mathcal{C}) = (V_{\mathcal{C}}, E_{\mathcal{C}})$ su grafo asociado. Entonces $|V_{\mathcal{C}}| = 2^{n-k}$.

Ejemplo 4.16. Sea $\mathcal{C} = \mathbb{F}_2^n$. Entonces $\mathcal{C}$ es el código lineal del espacio completo. Este es un $[n, n, 1]$ -código con una matriz generadora

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix}$$

Como $\mathcal{C}$ consiste de todas las palabras del espacio, este código tiene una única clase lateral y su líder es el vector $\mathbf{0}$. Así, el grafo relacionado tiene un único nodo y no tiene aristas, a este tipo de grafos se le conoce como grafo Singleton.



Figura 4.6: Grafo del código $\mathbb{F}_2^n$.

Ejemplo 4.17. Sea $\mathcal{C}$ un $[3, 2, 1]$ -código con una matriz generadora $G = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$.

Su grafo corresponde al grafo camino P_2 , ver Figura 4.7



Figura 4.7: Grafo $\Gamma(\mathcal{C}) \simeq P_2$.

En los Ejemplos 2.21, 2.22, 2.23, 2.24 y 2.25 de la Sección 2.2 se pueden observar los grafos asociados para algunos códigos lineales binarios. Dado que el Algoritmo `diagrama_Hasse_codigo`

(ver Algoritmo 2) tiene como salida un diccionario y se obtienen las relaciones, con respecto a $\prec$, entre los líderes del código, se pueden construir los grafos $\Gamma(\mathcal{C})$ con ayuda de la función `Graph()` de SageMath.

El grafo $\Gamma(\mathcal{C})$ asociado con un código lineal binario $\mathcal{C}$ tiene ciertas características. A continuación se presentan los resultados que se obtuvieron durante el desarrollo de este trabajo.

Teorema 4.18. *Sea $\mathcal{C}$ un código lineal binario. Entonces $\Gamma(\mathcal{C})$ es conexo.*

Demostración. Sean $\mathcal{C}_k$ y $\mathcal{C}_l$ clases laterales del código $\mathcal{C}$, tales que $\text{wt}(\mathcal{C}_k) \leq \text{wt}(\mathcal{C}_l)$, con $\mathbf{x}_k$ y $\mathbf{x}_l$ líderes de estas clases laterales, respectivamente. Considere dos casos:

- (i) Si $\mathbf{x}_k \prec \mathbf{x}_l$, entonces $\text{supp}(\mathbf{x}_k) \subset \text{supp}(\mathbf{x}_l)$. Por el Corolario 2.14, $\mathcal{C}_k$ es descendiente de $\mathcal{C}_l$. Sean $A_1 = \text{supp}(\mathbf{x}_l) \setminus \text{supp}(\mathbf{x}_k)$, $t = |A_1|$, $i_1 = \min A_1$, $\mathbf{x}_{i_1} = \mathbf{x}_l + \mathbf{e}_{i_1}$ y para $2 \leq s \leq t$

$$A_s = A_{s-1} \setminus \{i_{s-1}\},$$

$$i_s = \min A_s,$$

$$\mathbf{x}_{i_s} = \mathbf{x}_{i_{s-1}} + \mathbf{e}_{i_s}.$$

Adicionalmente, considere las clases laterales $\mathcal{C}_{i_s} = \mathbf{x}_{i_s} + \mathcal{C}$, para $1 \leq s \leq t$. Por el Corolario 2.16, se tiene que $\mathbf{x}_{i_1}$ es líder de un hijo de $\mathcal{C}_l$ y respectivamente que $\mathcal{C}_{i_s}$ es hijo de $\mathcal{C}_{i_{s-1}}$. Así se tiene la trayectoria $\mathcal{C}_l \mathcal{C}_{i_1} \cdots \mathcal{C}_{i_t}$, donde $\mathcal{C}_{i_t} = \mathcal{C}_k$. Esto quiere decir que existe un camino de $\mathcal{C}_k$ a $\mathcal{C}_l$. Observe que en realidad $\mathcal{C}_l \mathcal{C}_{i_1} \cdots \mathcal{C}_{i_t}$ es el camino más corto, aunque no necesariamente es único. En consecuencia, $d_\Gamma(\mathbf{x}_k, \mathbf{x}_l) = t$.

- (ii) Supóngase que $\mathcal{C}_k$ no es descendiente de $\mathcal{C}_l$. Sea

$$\Omega = \{\mathbf{y} + \mathcal{C} : \mathbf{y} + \mathcal{C} \prec \mathcal{C}_k \text{ y } \mathbf{y} + \mathcal{C} \prec \mathcal{C}_l\},$$

es decir Ω es el conjunto de los descendientes comunes de $\mathcal{C}_k$ y $\mathcal{C}_l$. Dado que $\mathcal{C}_0 = \mathbf{0} + \mathcal{C}$ es descendiente de cualquier clase lateral de $\mathcal{C}$, se tiene que $\Omega \neq \emptyset$. Sea $\mathcal{C}''$, tal que $\text{wt}(\mathcal{C}'') = \max\{\text{wt}(\mathcal{C}') : \mathcal{C}' \in \Omega\}$.

Entonces como $\mathcal{C}''$ es descendiente de $\mathcal{C}_k$, por el item (i) existe una trayectoria $\mathcal{C}_k \cdots \mathcal{C}''$ en $\Gamma(\mathcal{C})$ que une estos dos vértices. Similarmente, existe una $\mathcal{C}'' - \mathcal{C}_l$ trayectoria: $\mathcal{C}'' \mathcal{C}_1'' \cdots \mathcal{C}_l$ que une a $\mathcal{C}''$ y $\mathcal{C}_l$. Así el camino $\mathcal{C}_k \cdots \mathcal{C}'' \mathcal{C}_1'' \cdots \mathcal{C}_l$ es una trayectoria, pues la existencia de vértices repetidos en ella sería contradictorio con la escogencia de $\mathcal{C}''$.

□

De la demostración del anterior teorema, se obtiene el siguiente resultado.

Corolario 4.19. Sean $\mathcal{C}$ un código lineal binario, $\mathcal{C}_k$ y $\mathcal{C}_l$ clases laterales de $\mathcal{C}$ y $\mathbf{x}_k \in \mathcal{CL}(\mathcal{C}_k)$, $\mathbf{x}_l \in \mathcal{CL}(\mathcal{C}_l)$. Entonces

$$d_{\Gamma}(\mathcal{C}_k, \mathcal{C}_l) = \begin{cases} d(\mathbf{x}_k, \mathbf{x}_l), & \text{si } \mathcal{C}_k \prec \mathcal{C}_l \text{ y } \mathbf{x}_k \prec \mathbf{x}_l, \\ d(\mathbf{x}_k, \mathbf{x}'') + d(\mathbf{x}'', \mathbf{x}_l), & \text{si } \mathcal{C}_k \not\prec \mathcal{C}_l \text{ y } \mathbf{x}'' \in \mathcal{CL}(\mathcal{C}''), \mathcal{C}'' \in \Omega \text{ es} \\ & \text{tal que } \text{wt}(\mathcal{C}'') = \max\{\text{wt}(\mathcal{C}') : \mathcal{C}' \in \Omega\}. \end{cases}$$

Adicionalmente, por el Teorema 2.34 se obtiene el siguiente resultado.

Corolario 4.20. Sea $\mathcal{C}$ un código lineal binario. Entonces $\text{diam}(\Gamma(\mathcal{C})) \geq \rho(\mathcal{C})$.

A partir de diferentes observaciones realizadas con ayuda de **SageMath**, se encontraron otras propiedades en los grafos construidos. A continuación se estudia una de ellas.

Teorema 4.21. Sea $\mathcal{C}$ un código lineal binario, $\Gamma(\mathcal{C})$ es libre de triángulos.

Demostración. Supongáse que existe un triángulo en el grafo formado por los vértices $\mathcal{C}_k, \mathcal{C}_q$ y $\mathcal{C}_m$. Sin pérdida de generalidad, suponga que $\mathcal{C}_k$ es hijo de $\mathcal{C}_q$, es decir $\mathcal{C}_k \prec \mathcal{C}_q$ y $\text{wt}(\mathcal{C}_q) - 1 = \text{wt}(\mathcal{C}_k)$. Como en el triángulo también existe $\{\mathcal{C}_k, \mathcal{C}_m\} \in E$, entonces $\mathcal{C}_k$ es hijo de $\mathcal{C}_m$ o $\mathcal{C}_k$ es padre de $\mathcal{C}_m$. Sean $\mathbf{x}_k$ y $\mathbf{x}_m$ líderes de $\mathcal{C}_k$ y $\mathcal{C}_m$, respectivamente. Si $\mathcal{C}_k$ es hijo de $\mathcal{C}_m$, entonces $\text{wt}(\mathcal{C}_m) = \text{wt}(\mathcal{C}_q)$ y por el Corolario 2.16 $\mathcal{C}_q = \mathcal{C}_m$, lo que es una contradicción.

Por otra parte, si $\mathcal{C}_m$ es hijo de $\mathcal{C}_k$, entonces $\text{wt}(\mathcal{C}_m) = \text{wt}(\mathcal{C}_k) - 1$, además $\text{wt}(\mathcal{C}_k) = \text{wt}(\mathcal{C}_q) - 1$, así $\text{wt}(\mathcal{C}_m) = \text{wt}(\mathcal{C}_q) - 2$, por lo tanto no existe ninguna arista que una a $\mathcal{C}_m$ y $\mathcal{C}_q$, nuevamente una contradicción a nuestra suposición inicial. Así no hay una arista que una a $\mathcal{C}_k$ y $\mathcal{C}_m$. Por lo tanto el grafo $\Gamma(\mathcal{C})$ es libre de triángulos. □

El siguiente resultado generaliza, el teorema anterior.

Teorema 4.22. Sea un código $\mathcal{C}$ lineal binario, entonces $\Gamma(\mathcal{C})$ es un grafo bipartito.

Demostración. Sean

$$V_1 = \{\mathcal{C}_i : \text{wt}(\mathcal{C}_i) = 2m + 1, m \in \mathbb{N}\},$$

$$V_2 = \{\mathcal{C}_i : \text{wt}(\mathcal{C}_i) = 2m, m \in \mathbb{N}\}.$$

De esta forma, $V = V_1 \cup V_2$ y $V_1 \cap V_2 = \emptyset$. Ahora, por la definición de $\Gamma(\mathcal{C})$ los vértices de peso par son vecinos únicamente de vértices de peso impar y viceversa. Por lo tanto, toda arista en $E_{\mathcal{C}}$ relaciona nodos de V_1 con nodos de V_2 , pero no conecta nodos en el mismo conjunto, así $\Gamma(\mathcal{C})$ es bipartito. $\square$

Observación 4.23. *El siguiente resultado muestra que el Teorema 4.22 es una generalización del Teorema 4.21, puesto que si un grafo no tiene ciclos impares, claramente es libre de triángulos.*

Proposición 4.24. (Theorem 1.12, [2]) *Un grafo Γ es bipartito si y solo si Γ no contiene ciclos impares.*

Ahora, se enunciarán dos definiciones importantes para el resultado que les sigue.

Definición 4.25. Dos (n, M) -códigos binarios $\mathcal{C}$ y $\mathcal{D}$ son equivalentes, si existe una permutación π del conjunto $\{1, \dots, n\}$ y biyecciones $\sigma_1, \dots, \sigma_n$ de $\mathbb{F}_2$, tales que $c_1 c_2 \dots c_n \in \mathcal{C}$ si y solo $\sigma_1(c_{\pi(1)}) \sigma_2(c_{\pi(2)}) \dots \sigma_n(c_{\pi(n)}) \in \mathcal{D}$.

Para abreviar la notación usada en la Definición 4.25, se considera la función P de $\mathbb{F}_2^n$ en $\mathbb{F}_2^n$ dada por

$$P(u_1 u_2 \dots u_n) = \sigma_1(u_{\pi(1)}) \sigma_2(u_{\pi(2)}) \dots \sigma_n(u_{\pi(n)}), \quad (4.2)$$

donde π es una permutación de $\{1, \dots, n\}$ y $\sigma_1, \dots, \sigma_n$ son biyecciones de $\mathbb{F}_2$. De esta forma, dos (n, M) -códigos binarios $\mathcal{C}$ y $\mathcal{D}$ son equivalentes si existe P como en (4.2) tal que $\mathcal{D} = \{P(\mathbf{c}) : \mathbf{c} \in \mathcal{C}\}$. Se puede demostrar que dados dos vectores $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$, se satisface la igualdad

$$d(\mathbf{u}, \mathbf{v}) = d(P(\mathbf{u}), P(\mathbf{v})); \quad (4.3)$$

esto es, la función P preserva distancias. De (4.3) se puede verificar que dos códigos equivalentes tienen la misma distancia mínima.

Definición 4.26. Dos grafos Γ y Δ son isomorfos si existe una biyección de los vértices de un grafo en el otro, de modo tal que se preserve la adyacencia de los vértices; es decir que envíe vértices adyacentes de Γ en vértices adyacentes en Δ . De manera más formal, dos grafos $\Gamma = (V, E)$ y $\Delta = (U, D)$ son isomorfos si existe una biyección entre los conjuntos de vértices V y U , $\varphi : V \rightarrow U$ tal que para cualquier $\{u, v\} \in E$ se tiene que $\{\varphi(u), \varphi(v)\} \in D$.

Teorema 4.27. Sean $\mathcal{C}$ y $\mathcal{D}$ dos códigos lineales binarios. Si $\mathcal{C}$ y $\mathcal{D}$ son códigos equivalentes, entonces $\Gamma(\mathcal{C})$ y $\Gamma(\mathcal{D})$ son isomorfos.

Demostración. Por hipótesis se tiene que existe P como en (4.2) tal que $\mathcal{D} = \{P(\mathbf{c}) : \mathbf{c} \in \mathcal{C}\}$. Asuma que $\mathbf{u}$ es un líder de la clase lateral $\mathbf{u} + \mathcal{C}$. Entonces como $d(\mathbf{u}, \mathbf{v}) = d(P(\mathbf{u}), P(\mathbf{v}))$ se tiene que para cualquier $\mathbf{c} \in \mathcal{C}$, $\text{wt}(\mathbf{u} + \mathbf{c}) = \text{wt}(P(\mathbf{u}) + \mathbf{v})$, donde $\mathbf{v} = P(\mathbf{c})$. Entonces $P(\mathbf{u})$ es un líder de su clase lateral en $\mathcal{D}$. Es decir, $P(\mathbf{u}) + \mathcal{D}$ es un vértice en $\Gamma(\mathcal{D})$.

Ahora, si $\{\mathbf{u} + \mathcal{C}, \mathbf{u}' + \mathcal{C}\}$ es una arista en $\Gamma(\mathcal{C})$, entonces $\mathbf{u} + \mathcal{C} \prec \mathbf{u}' + \mathcal{C}$. Así por el Corolario 2.14, existe un líder $\mathbf{u}''$ de $\mathbf{u}' + \mathcal{C}$, tal que $\text{supp}(\mathbf{u}) \subset \text{supp}(\mathbf{u}'')$. Esta última afirmación implica que $\text{supp}(P(\mathbf{u})) \subset \text{supp}(P(\mathbf{u}''))$ o, equivalentemente, que $P(\mathbf{u}) + \mathcal{D} \prec P(\mathbf{u}'') + \mathcal{D}$ o sea que $\{P(\mathbf{u}) + \mathcal{D}, P(\mathbf{u}') + \mathcal{D}\}$ es una arista en $\Gamma(\mathcal{D})$. Así la función $\varphi : V_{\mathcal{C}} \rightarrow V_{\mathcal{D}}$ definida por $\varphi(\mathbf{u} + \mathcal{C}) = P(\mathbf{u}) + \mathcal{D}$ es un isomorfismo de grafos. $\square$

Sin embargo, el recíproco del Teorema 4.27 no es verdadero. Los Ejemplos 2.23 y 2.26 muestran dos de códigos que no son equivalentes, pero cuyos grafos claramente son isomorfos.

Como se puede ver en el Ejemplo 2.25, bajo ciertas circunstancias el grafo $\Gamma(\mathcal{C})$ puede tener forma de estrella. El siguiente resultado caracteriza completamente para cuáles códigos se obtiene esta figura.

Teorema 4.28. Sea $\mathcal{C}$ un código lineal binario. Entonces $\Gamma(\mathcal{C}) \simeq S_{2^{n-k}}$ si y solo si $\rho(\mathcal{C}) = 1$.

Demostración. Si $\Gamma(\mathcal{C})$ es un grafo estrella esto implica que todas las clases laterales tienen peso a lo más 1; esto es para todo $\mathbf{u} \notin \mathcal{C}$ existe $i \in \{1, \dots, n\}$ tal que $\mathbf{u} \in \mathbf{e}_i + \mathcal{C}$, o equivalentemente que $\mathbf{u} = \mathbf{e}_i + \mathbf{c}$ para algún $\mathbf{c} \in \mathcal{C}$, esto es $d(\mathbf{u}, \mathbf{c}) = 1$. Esto implica que para todo $\mathbf{u} \in \mathbb{F}_2^n$, existe $\mathbf{c} \in \mathcal{C}$ tal que $d(\mathbf{u}, \mathbf{c}) \leq 1$. Por lo tanto, $\mathbb{F}_2^n = \bigcup_{\mathbf{c} \in \mathcal{C}} S(\mathbf{c}, 1)$; es decir $\rho(\mathcal{C}) = 1$. Recíprocamente, si $\rho(\mathcal{C}) = 1$, las clases laterales de $\mathcal{C}$ son de la forma $\mathbf{e}_i + \mathcal{C}$, entonces $\Gamma(\mathcal{C})$ tiene la forma que se muestra en la Figura 4.8.

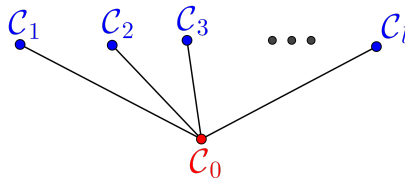


Figura 4.8: Grafo estrella.

Es decir $\Gamma(\mathcal{C})$ es un grafo estrella con $|V_{\mathcal{C}}| = 2^{n-k}$ vértices.

□

Apéndice A

Elementos básicos de SageMath (Python)

En este trabajo se usó el software libre **SageMath** creado por William Stein en 2005, este software está basado en el lenguaje de programación de **Python**. A continuación se darán algunos de los elementos más usados en este trabajo.

- **Lists**: son arreglos que contienen elementos no necesariamente del mismo tipo, también se les puede llamar contenedores dinámicos de datos. Para crear listas basta con poner los elementos entre corchetes.

```
Lista=['Hola',1,2]
print(Lista)
['Hola',1,2]
```

Para acceder a un elemento de una lista, se escribe el nombre de la lista seguido de la posición donde está el elemento entre corchetes y para el tamaño de la lista se utiliza `len()`. Por ejemplo

```
Lista[0]
Hola
len(Lista)
3
```

Para las **listas** en SageMath existen varios métodos integrados, a continuación se enlistan algunos de ellos:

- **append()**: Adiciona un elemento en la última posición de la lista.
 - **insert()**: Adiciona un elemento en la lista en la posición deseada.
 - **pop()**: Borra el último o el elemento en la posición deseada de una lista, además retorna ese elemento.
 - **index()**: Retorna el índice de un elemento en la lista.
- **Sets**: los conjuntos son listas sin entradas duplicadas y sus miembros pueden ser de diversos tipos; no obstante, no pueden incluir objetos mutables como listas, diccionarios o conjuntos. Para crear un conjunto se especifican los elementos entre llaves:

```
set={2,3,1,5,7,1,2}
print(set)
{1, 2, 3, 5, 7}
```

Algunos métodos integrados en los **sets** son:

- **union()**: Se agrupan los elementos de dos conjuntos y se eliminan las repeticiones, retorna un nuevo conjunto.
- **intersection()**: Se buscan elementos en común de dos conjuntos y se forma un nuevo conjunto.
- **difference()**: Se realiza la diferencia habitual entre dos conjuntos.

Observación A.1. *El método integrado `set_immutable()` permite convertir los elementos como listas, diccionario o conjuntos a elementos inmutables para poder utilizar algunas operaciones con los conjuntos.*

- **Dictionaries**: Los diccionarios son un tipo de arreglos estructurados por pares **keys** (llaves) –**values** (valores), las llaves solo permiten elementos inmutables, en cambio en valores se admiten todo tipo de objetos. Una vez definido el diccionario, se pueden incluir nuevos elementos, borrar y modificar elementos existentes con una sintaxis similar a la que se usa con listas.

```
diccionario={1:'uno',2:'cero',3:'tres'}  
print(diccionario)  
  
{1: 'uno', 2: 'cero', 3: 'tres'}
```

Para corregir un valor se ingresa el nombre del diccionario y la llave ya existente, así: `diccionario[key]=value`. Para agregar un nuevo elemento se ingresa el nombre del diccionario y la nueva llave.

```
diccionario[2]='dos'  
diccionario[4]='cuatro'  
print(diccionario)  
  
{1: 'uno', 2: 'dos', 3: 'tres', 4: 'cuatro'}
```

Los métodos integrados de los diccionarios usados en el trabajo son los siguientes:

- `keys()`: Guarda en una lista las llaves del diccionario.
 - `values()`: Guarda en una lista los valores del diccionario.
 - `append()`: Agrega un valor a una llave ya existente.
- **Códigos:** Para construir códigos lineales en SageMath primero se define el campo finito y la matriz generadora del código, así:

```
F=GF(2)  
MG=matrix(F,[[1,0,1,1,0],[1,1,1,0,1]])  
print(MG)  
  
[1 0 1 1 0]  
[1 1 1 0 1]  
  
C=LinearCode(MG)  
print(C)  
  
[5, 2] linear code over GF(2)
```

Algunos de los métodos integrados para la teoría de códigos son los siguientes:

- `length()`: Longitud del código.

- `dimension()`: Dimensión del código.
- `hamming_weight()`: Calcula el peso de Hamming de una palabra de un código.
- `support()`: En una lista guarda el soporte de un vector.
- `parity_check_matrix()`: Calcula la matriz de control de paridad de un código. A partir de esta matriz también se pueden crear códigos.
- `minimum_distance()`: Calcula la distancia mínima de un código.

Observación A.2. *Existen muchas más funciones y propiedades que aquí no se mencionan en la teoría de códigos dentro de SageMath. Por ejemplo, códigos ya establecidos como los códigos de Reed-Muller o los códigos de Hamming.*

- **functions:** Las funciones dentro de SageMath son útiles para dividir el código que se está escribiendo en bloques. Se definen usando la palabra reservada "`def`" seguida del nombre de la función, también pueden recibir argumentos. Por ejemplo:

```
def suma(a,b):  
    return a+b
```

Las funciones pueden ser llamadas dentro de una implementación, solo se escribe el nombre de la función y entre paréntesis los argumentos requeridos.

Apéndice B

Algoritmos y su implementación en SageMath

A continuación se presentan los algoritmos auxiliares, pseudocódigos e implementaciones en SageMath de los algoritmos construidos en el presente trabajo. Primero se dará una breve descripción, seguida de los algoritmos auxiliares, pseudocódigo y finalmente su implementación en SageMath.

B.1. Algoritmo arreglo estándar

Este algoritmo se construyó con la idea original del arreglo estándar o arreglo Slepiano (ver Sección 2.1). Mediante un ciclo `while` el cual espera hasta que una lista donde se guardan todos los elementos del espacio vectorial en el que se está trabajando quede vacía. Dentro de este ciclo hay un ciclo `for` en el cual se hacen cálculos de peso de Hamming y con ayuda de operaciones de conjuntos y propiedades de las `listas` se calcula un líder y su respectiva clase lateral, esta información se guarda en una matriz y cada una de estas clases laterales se le quita a la lista donde se guardan los elementos del espacio vectorial. Adicionalmente, se toma la primera columna de la matriz, en donde teóricamente se sabe que están los líderes del código.

Algoritmo 1: arreglo_estandar**Entrada:** Un código lineal binario $\mathcal{C}$.**Salida:** Matriz del arreglo estándar y líderes del código $\mathcal{C}$

```

1 begin
2    $m \leftarrow$  Longitud del código;
3    $FC \leftarrow$  Código del espacio  $\mathbb{F}_2^m$ ;
4    $L \leftarrow$  Lista de  $FC$ ;
5    $B \leftarrow \emptyset$ ;
6    $p \leftarrow 0$ ;
7    $AE \leftarrow \emptyset$ ;
8   Se hace un ciclo hasta que  $L = \emptyset$  con ayuda del siguiente proceso;
9   for  $i \in [0, \text{Longitud de } L \text{ menos } 1]$  do
10       $b \leftarrow$  Guarda el peso de Hamming de cada palabra del espacio  $\mathbb{F}_2^m$ ;
11      if  $p = b$  and  $FC[i] \in L$  then
12          $B \leftarrow$  Se guarda en B la clase lateral de  $FC[i]$ ;
13          $AE \leftarrow$  Se agrega B a la lista AE;
14          $L \leftarrow L - B$ ;
15          $B \leftarrow \emptyset$ 
16    $FC \leftarrow L$ ;
17    $p \leftarrow p + 1$ ;
18   columna  $\leftarrow$  Extrae la primera columna de la matriz AE

```

Para la implementación en **SageMath** el algoritmo recibe un código lineal, al final muestra una matriz en la que está guardado el arreglo estándar calculado, donde la primera columna de la matriz contiene los líderes de las clases laterales del código.

```

def arreglo_estandar(C):
    #Recibe un codigo lineal binario y retorna el arreglo estandar.
    m=C.length()
    F=GF(2)^m

```



```

A=F.basis_matrix()
FC=LinearCode(A)
L=FC.list()
LC=C.list()
B=[]
p=0
AE=[]
while L!=[]:
    for i in [0..len(L)-1]:
        b=FC[i].hamming_weight()
        if p==b and int(FC[i] in L)==1:
            for k in [0..len(LC)-1]:
                B.append(LC[k]+FC[i])
                B[k].set_immutable()
            AE.append(B)
            L=list(set(L).difference(B))
            B=[]
    FC=L
    p=p+1
    z=0
    columna = [fila[z] for fila in AE]
    return AE, columna

```

B.2. Algoritmo Diagrama de Hasse

El siguiente desarrollo se basó en las ideas proporcionadas por las definiciones, teoremas y demás resultados del Capítulo 11 del libro de Huffman [4] que aquí se exponen en la Sección 2.2. El algoritmo recibe un código y funciona con funciones auxiliares descritos después de mostrar cómo se implementó en SageMath este algoritmo. Un ciclo `while` que funciona hasta que en una lista se guarden todos los elementos del espacio vectorial en el que se está trabajando, con ayuda de algunas operaciones y con las funciones auxiliares se llena un diccionario. En las llaves del diccionario se guarda un líder por cada clase lateral. Además, en los valores se guardan las relaciones que tiene un líder con sus hijos. A partir del diccionario obtenido se puede realizar

la construcción del grafo de líderes del código.

Algoritmo 2: `diagrama_Hasse_codigo`

Entrada: Un código lineal binario $\mathcal{C}$

Salida: Diccionario de líderes

```

1 begin
2    $F \leftarrow \mathbb{F}_2^n$ ;
3    $diccionario\_lideres \leftarrow \{str(cero) : [ ]\}$ ;
4    $L \leftarrow$  Lista de las palabras del código  $\mathcal{C}$ ;
5    $r \leftarrow$  Longitud de  $L$ ;
6    $lideres \leftarrow [cero]$ ;
7   while  $r < 2^n$  do
8      $T \leftarrow$  clase_lateral( $hijo, \mathcal{C}$ ), donde  $hijo$  es el último elemento que se eliminará de
      la lista  $lideres$  ;
9     if  $not$   $huerfano(\mathcal{C}, T)$  then
10       $lid \leftarrow$  LISTA_LID[0], donde en LISTA_LID se han guardado los líderes de
        clases laterales de  $e_i + hijo$  en  $\mathcal{C}$ ;
11      if  $not$   $lid \in L$  then
12         $lid \leftarrow$  se agrega a la lista  $lideres$ ;
13         $L \leftarrow$  Se extiende la lista  $L$  con la lista de la clase lateral de  $e_i + hijo$  y se
          actualiza el tamaño de  $L$  en la variable  $r$ ;
14         $diccionario\_lideres[str(lid)] \leftarrow [ ]$ ;
15        for  $x \in LISTA\_LID$  do
16           $diccionario\_lideres[str(lid)] \leftarrow$  se agrega  $str(e_j + x)$  para establecer
            las conexiones de los líderes con cada uno de sus hijos;
```

Se ingresa un código lineal binario para la implementación en SageMath y el algoritmo retorna un diccionario que contiene los líderes del código y la relación con sus hijos.

```
def diagrama_Hasse_codigo(C):
    "Este algoritmo esta pensado para recibir un codigo C y encontrar su
    diagrama de Hasse de ancestros, huérfanos, padres e hijos"
    n=C.length()
    F=C.ambient_space()
    L=C.list()
    cero=zero_vector(GF(2),n)
    diccionario_lideres={str(cero):[]}
    r=len(L)
    lideres=[cero]
    lista_lid=[cero]
    while r<2^n:
        hijo=lideres.pop()
        T=clase_lateral(hijo,C)
        if not(huerfano(C,T)):
            for i in [0..n-1]:
                ei=F.matrix()[i]
                C_prima=clase_lateral(ei+hijo,C)
                LISTA_LID=liderex(C_prima)
                lid=LISTA_LID[0]
                if not(lid in L):
                    L.extend(C_prima)
                    r=len(L)
                    diccionario_lideres[str(lid)]=[]
                    for x in LISTA_LID:
                        for j in x.support():
                            ej=F.matrix()[j]
                            if str(ej+x) in diccionario_lideres and not(str(
                                ej+x) in diccionario_lideres[str(lid)]):
                                diccionario_lideres[str(lid)].append(str(ej+x
                                ))
                    lideres.append(lid)
                    lista_lid.append(lid)
    return diccionario_lideres
```

B.2.1. Funciones auxiliares

Las siguientes son las funciones auxiliares usadas en el anterior algoritmo.

Algoritmo 3: `clase_lateral`

Entrada: Una palabra $\mathbf{a} \in \mathbb{F}_2^n$ y un código lineal binario C .

Salida: Clase lateral de $\mathbf{a}$ relacionada con el código C .

```

1 begin
2   for  $\mathbf{c} \in C$  do
3      $\mathbf{a} + \mathbf{c} \leftarrow$  Palabras de la clase lateral que se guardarán en  $CL$ .
```

```

def clase_lateral(a,C):
    "Recibe un vector a y calcula su clase lateral a+C"
    return [a+c for c in C]
```

Algoritmo 4: `liderex`

Entrada: Recibe una clase lateral T .

Salida: Lista de líderes de una clase lateral.

```

1 begin
2    $p\_h \leftarrow$  Guarda la lista de los pesos de Hamming de una clase lateral;
3    $ind \leftarrow$  Busca los índices de los elementos de peso mínimo de la clase lateral;
4   for  $j \in ind$  do
5      $T[j] \leftarrow$  Guarda los líderes de la clase lateral  $T$  en una lista L;
```

```

def liderex(L):
    p_h=[L[i].hamming_weight() for i in range(len(L))]
    ind=[k for k,x in enumerate(list(p_h)) if x==min(p_h)]
    return [L[j] for j in ind]
```

Algoritmo 5: huerfano**Entrada:** Recibe una clase lateral T y un código C **Salida:** Bandera de verdadero o Falso

```

1 begin
2    $\mathbb{F}_2^n \leftarrow$  Espacio vectorial binario de dimensión  $n$ ;
3    $w \leftarrow$  Peso mínimo de la clase lateral  $T$ ;
4    $\mathbf{e}_i \leftarrow$  Vector canónico con 1 en la posición  $i \in [0, \dots, n-1]$  y 0 en las otras
      posiciones;
5    $C\_prima \leftarrow \text{clase\_lateral}(\mathbf{e}_i, T)$ ;
6    $bandera \leftarrow \text{False}$ ;
7   for  $\mathbf{y} \in C\_prima$  do
8      $bandera \leftarrow \text{True}$ , si el peso de Hamming de  $\mathbf{y} \leq w$ ;
9   if  $bandera = \text{False}$  then
10     $bandera \leftarrow \text{False}$ ;

```

```

def huerfano(C,T):
    "Este algoritmo recibe una clase lateral T y determina si es un
      huerfano del codigo C"
    F=C.ambient_space()
    n=C.length()
    x=liderex(T)[0]
    w=x.hamming_weight()
    for i in [0..n-1]:
        ei=F.matrix()[i]
        C_prima=clase_lateral(ei,T)
        bandera=False
        for y in C_prima:
            if y.hamming_weight()<=w:
                bandera=True
        if bandera==False:
            return False
    return(True)

```

B.3. Algoritmo cálculo de líderes

El algoritmo que se muestra a continuación, fue desarrollado con base en las ideas dadas en el artículo [1], ver Sección 2.3. Este algoritmo recibe un código lineal binario $\mathcal{C}$, utiliza las funciones auxiliares 7, 8, 9 y 10. El algoritmo en un ciclo **while** el cual para cuando se termine de vaciar una lista que ayuda a la construcción de los líderes, se divide principalmente en dos partes, si el síndrome de la palabra está o no calculado anteriormente. Si no está, se crea una nueva llave donde se guarda el nuevo líder encontrado y si el síndrome está, se agrega este líder a la llave correspondiente con el otro líder de la clase lateral. Así, el algoritmo construye un diccionario que contiene todos los líderes del código, con la particularidad que muestra cuáles pertenecen a la misma clase lateral.

Algoritmo 6: Comp_CL

Entrada: Un código lineal binario C y un tipo de ordenamiento $orden$

Salida: Lista de líderes de clases laterales

```

1  begin
2       $n \leftarrow$  Longitud del código  $C$ ;  $H \leftarrow$  Matriz de paridad;  $Listing \leftarrow [0]$ ,  $0 \in \mathbb{F}_2^n$ ;  $N \leftarrow []$ ;  $CL \leftarrow \{\}$ ;  $S \leftarrow []$ ;
3      while  $Listing \neq []$  do
4           $t \leftarrow$  Primer elemento eliminado de  $Listing$  con la función auxiliar  $NextTerm(Listing)$  ;
5           $s \leftarrow$  Cálculo de la multiplicación de  $t$  y  $H$  transpuesta;
6           $j \leftarrow$  Índice del elemento  $s$  en la lista  $S$ , calculado con la función auxiliar  $Member(s,S)$ ;
7          if  $j \neq False$  then
8              Si el peso de hamming de  $t$  es igual al peso de hamming de  $N[j]$  entonces, se agrega el elemento  $t$  al
                diccionario de  $CL$  en la posición  $j + 1$  y con la función auxiliar  $InsertNext$  se agregan elementos a
                 $Listing$  relacionados con  $t$  teniendo en cuenta el  $orden$ .
9          else if then
10              $r \leftarrow r + 1$ ;
11             Se agrega el elemento  $t$  al final de la lista  $N$ , análogamente  $s$  a la lista  $S$  y en la posición  $r$  del diccionario
                 $CL$  se inserta la lista  $[t]$ . Con la función auxiliar  $InsertNext$  se agregan elementos a  $Listing$ 
                relacionados con  $t$  y el  $orden$ .
```

Para su implementación en SageMath se debe ingresar un código lineal binario y un tipo de orden. La salida es un diccionario, donde en los valores se guardan los líderes del código.

```

def Comp_CL(C, orden):
    n=C.length()
    H=C.parity_check_matrix()
```

```

Listing=[zero_vector(GF(2),n)]
N=[]
r=0
CL={}
S=[]
while Listing!=[]:
    t=NextTerm(Listing)
    s=t*H.transpose()
    j=Member(s,S)
    if j!=-1:
        if t.hamming_weight()==N[j].hamming_weight():
            CL[j+1].append(t)
            InsertNext(t,Listing,n,orden)

    if j==-1:
        r=r+1
        N.append(t)
        CL[r]=[t]
        S.append(s)
        InsertNext(t,Listing,n,orden)

return CL

```

B.3.1. Funciones Auxiliares

Estas son las funciones auxiliares para la construcción del anterior algoritmo.

Algoritmo 7: NextTerm

Entrada: Lista de términos

Salida: Elemento eliminado de una lista

```

1 begin
2   L ← Lista ingresada;
3   if  $L \neq \emptyset$  then
4     Se elimina el primer elemento de la lista
5   else if then
6     Se retorna Falso a la salida

```

```

def NextTerm(L):
    if L != []:
        return L.pop(0)
    else:
        return False

```

Algoritmo 8: Member

Entrada: Recibe un elemento *obj* y una lista *G*

Salida: Índice de *obj* en *G*

```

1 begin
2   if  $obj \in G$  then
3     Encuentra el índice de obj en la lista G.
4   else if then
5     Se retorna Falso a la salida

```

```

def Member(obj, G):

```



```

    if obj in G:
        return G.index(obj)
    else:
        return -1

```

Algoritmo 9: ordering

Entrada: Dos vectores u, v y un orden

Salida: Lista con dos elementos ordenados, donde el primero es el menor con respecto al orden dado

```

1 begin
2   if wt(v) < wt(u) then
3     Se crea la lista [v, u]
4   else if then
5     if wt(u) < wt(v) then
6       Se crea la lista [u, v]
7     else if then
8        $n \leftarrow$  Tamaño del vector  $v$ ;
9        $R \leftarrow$  Pasar los elementos del campo binario  $n$  a un anillo de Polinomios, teniendo en cuenta el orden
          ingresado;
10       $var \leftarrow$  Invariante del anillo de polinomios  $R$ ;
11       $f \leftarrow 1$ ;
12       $g \leftarrow 1$ ;
13      for  $i \in [0..n-1]$  do
14         $f = var[i]^{u[i]} * f$ ;  $g = var[i]^{v[i]} * g$ ;
15      if  $f < g$  then
16        Se crea la lista [u, v]
17      else if then
18        Se crea la lista [v, u]

```

```

def ordering(u, v, orden):
    if v.hamming_weight() < u.hamming_weight():
        return [v, u]
    else:
        if u.hamming_weight() < v.hamming_weight():
            return [u, v]
        else:
            n = len(v)
            R = PolynomialRing(GF(2), 'x', n, order=orden)

```

```

power_product=1
var=R.gens()
f=1
g=1
for i in [0..n-1]:
    f=var[i]^u[i]*f
    g=var[i]^v[i]*g
if f<g:
    return[u,v]
else:
    return[v,u]

```

Algoritmo 10: InsertNext

Entrada: Recibe un vector $\mathbf{t}$, una lista L , un número n y un *orden*

Salida: Lista ordena L sin repeticiones

```

1 begin
2    $\mathbb{F}_2^n \leftarrow$  Espacio vectorial binario de dimensión  $n$ ; Soporte  $\leftarrow$  Soporte del vector  $\mathbf{t}$ ; Conj  $\leftarrow [0, \dots, n-1] - \text{Soporte}$ ;
3   for  $k \in \text{Conj}$  do
4        $\mathbf{tek} \leftarrow \mathbf{t} + \mathbf{e}_k$ ;
5        $b \leftarrow \text{False}$ ;
6        $i \leftarrow 1$ ;
7       if  $\mathbf{tek} \notin L$  then
8           if  $L \neq \emptyset$  then
9               while  $b = \text{False}$  end  $i \leq \text{Longitud de } L$  do
10                    $q + i \leftarrow \text{Longitud de } L$ ;
11                    $ad \leftarrow \text{ordering}(L[q], \mathbf{tek}, \text{orden})$ ;
12                   if  $ad[1] = \mathbf{tek}$  then
13                        $L \leftarrow$  Se inserta el elemento  $\mathbf{tek}$  en la posición de  $L[q] + 1$ ;  $b = \text{True}$ 
14                    $i = i + 1$ ;
15               if  $\mathbf{tek} \notin L$  then
16                    $L \leftarrow$  Se inserta  $\mathbf{tek}$  en la primera posición.
17           else if then
18                $L \leftarrow$  Se inserta  $\mathbf{tek}$  en la última posición de  $L$ .

```

```

def InsertNext(t,L,n,orden):
    F=GF(2)^n
    Soporte=t.support()

```

```

Conj=list(Set([0..n-1]).difference(t.support()))
for k in Conj:
    tek=t+F.matrix()[k]
    b=False
    i=1
    if not tek in L:
        if L!=[]:
            while b==False and i<=len(L):
                q=len(L)-i
                ad=ordering(L[q],tek,orden)
                if ad[1]==tek:
                    L.insert(L.index(L[q])+1,tek)
                    b=True
                i=i+1
            if not tek in L:
                L.insert(0,tek)
        else:
            L.append(tek)
return L

```

B.4. Algoritmos auxiliares para contar

El siguiente algoritmo cuenta el número de líderes de clases laterales de un determinado peso y los guarda en un diccionario. Se hace directamente con los vectores líderes previamente guardados en una lista.

```

from collections import Counter
def fun_cont_lid(L):
    PESOS=[y.hamming_weight() for y in L]
    c = Counter(PESOS)
    return c

```

Este algoritmo recibe un diccionario, extrae de las llaves las cadenas que representan a los líderes y los convierte en vectores, luego calcula el peso de Hamming de los líderes y guarda en

un diccionario el número de líderes de un código respecto a su peso.

```

from collections import Counter
def fun_cont_lid(D):
    L=D.keys()
    vectores_lideres=[vector(GF(2),eval(x)) for x in L]
    PESOS=[y.hamming_weight() for y in vectores_lideres]
    c = Counter(PESOS)
    return c

```

Igual que el anterior algoritmo, este recibe un diccionario, pero los líderes se guardan en los valores donde se conserva el formato de vector, luego se escoge un líder de cada clase lateral y se calcula su peso, se cuenta el número de líderes de cada peso y se guarda en un diccionario.

```

from collections import Counter
def fun_cont_lid(D):
    L=list(D.values())
    L1=[L[i][0] for i in range(0,len(L)-1)]
    PESOS=[y.hamming_weight() for y in L1]
    c = Counter(PESOS)
    return c

```

B.5. Algoritmos de decodificación

La decodificación por distancia mínima para códigos lineales binarios se basa en la teoría del Capítulo 3, el algoritmo calcula el síndrome del mensaje que se desea transmitir por el Teorema 3.4 el mensaje y el error tienen el mismo síndrome, y se suma el error al mensaje transmitido obteniendo el mensaje que se deseaba enviar siempre y cuando el líder sea único, si no es único el algoritmo escoge un líder aleatoriamente y puede que el mensaje decodificado sea incorrecto. Este algoritmo recibe la palabra que se ha transmitido, el código y la lista de líderes del código, la cual se ha calculado con anterioridad con alguno de los anteriores algoritmos para el cálculo de líderes: `arreglo_estandar` (ver Algoritmo 1), `diagrama_Hasse_codigo` (ver Algoritmo 2) y

Comp_CL (ver Algoritmo 6).

Algoritmo 11: decodificacion

Entrada: Recibe un vector $\mathbf{c}$, un código $\mathcal{C}$ y la lista de líderes del código, L

Salida: Palabra decodificada

```

1 begin
2    $H \leftarrow$  Matriz de control de paridad de  $\mathcal{C}$ ;
3    $cs \leftarrow$  Síndrome del vector  $\mathbf{c}$ ;
4   for  $i \in [0..longitud\ de\ L-1]$  do
5        $s \leftarrow$  Cálculo del síndrome  $L[i]$ ;
6       Si el síndrome  $s$  y  $cs$  son iguales, se retorna  $L[i] - \mathbf{c}$ 

```

El algoritmo usado en SageMath recibe un vector, un código binario y la lista de líderes.

```

def decodificacion (c,C,L):
    H=C.parity_check_matrix()
    S=[]
    cs=c*H.transpose()
    for i in [0..len(L)-1]:
        s=L[i]*H.transpose()
        if s==cs:
            return L[i]-c

```

Algoritmo 12: decodificacion1

Entrada: Recibe un vector $\mathbf{c}$, un código $\mathcal{C}$ y un diccionario D con la lista de líderes

Salida: Lista de palabras decodificadas

```

1 begin
2    $H \leftarrow$  Matriz de control de paridad de  $\mathcal{C}$ ;
3    $cs \leftarrow$  Síndrome del vector  $\mathbf{c}$ ;
4    $b \leftarrow \text{True}$ ;
5   while  $b$  do
6      $s \leftarrow$  Cálculo del síndrome de  $L[i][0]$ ;
7     Si el síndrome  $s$  y  $cs$  son iguales, se calcula la lista con los elementos  $L[i][j] - \mathbf{c}$ ,
       con  $j \in \{0 \dots \text{longitud de } L-1\}$ 

```

```

def decodificacion1(c,C,D):
    H=C.parity_check_matrix()
    S=[]
    cs=c*H.transpose()
    L=list(D.values())
    i=0
    b=True
    while b:
        s=L[i][0]*H.transpose()
        if s==cs:
            k=len(L[i])
            de=[L[i][j]-c for j in range(0,k)]
            b=False
        i=i+1
    return de

```

Referencias

- [1] Borges-Quintana, M., Borges-Trenard, M. A., Márquez-Corbella, I., Martínez-Moro, E. (2015). Computing coset leaders and leader codewords of binary codes. *J. Algebra Appl.*, 14(8):1550128, 19.
- [2] Chartrand, G.,Zhang, P. (2012). *A First Course in Graph Theory*. Mineola, New York: Dover Publications.
- [3] Epp, S. S. (2012). *Matemáticas discretas con aplicaciones*. México, D.F.: Cengage Learning.
- [4] Huffman, W. C.,Pless, V. (2003). *Fundamentals of error-correcting codes*. New York: Cambridge Univ. Press.
- [5] Ling, S.,Xing, C. (2004). *Coding Theory: A First Course*. New York: Cambridge University Press.
- [6] López, J. J.,Ruíz, H. M. (2013). La matemática de la teoría de códigos. Tesis de pregrado, Universidad de Nariño.
- [7] Shannon, C. E. (1948). A mathematical theory of communication. *Bell Syst. Tech. J.*, 27(3):379–423.